Pas à pas apprenant Sprint 3 CISIA InduSense

Version du 13 octobre 2026. Thomas Fesq, Formatome.

Ce livret accompagne les douze demi-journées du Sprint 3. Tu vas ouvrir le bon projet, intégrer le jalon annoncé, réaliser les exercices, puis conserver ton travail dans Git. Les dates d'août et septembre mentionnées dans les supports décrivent la cohorte d'origine ; le formateur annonce les dates et le rythme de ta session.

Préparer un dépôt dédié au Sprint 3

Un **dossier** contient tes fichiers. Un **dépôt Git** contient aussi leur historique dans un dossier caché `.git`. Un **commit** est une sauvegarde locale choisie ; un **push** envoie des commits vers un dépôt distant. Un **pull** récupère des commits distants. Fermer VS Code ne supprime pas les fichiers, mais un fichier enregistré n'est pas automatiquement commité ou envoyé.

Le dépôt officiel du cours est **https://github.com/thomasfesq/CISIA_24082026_Parcours**. Son remote doit garder le nom **origin** : les scripts de formation utilisent ce nom pour récupérer les jalons. Le remote **perso** servira à sauvegarder tes travaux dans ton dépôt personnel. Tu ne dois pas supposer que tu peux écrire dans le dépôt officiel.

Choisir le dossier et le terminal

Dans VS Code, ouvre **Terminal > Nouveau terminal**. Sous Windows, choisis **PowerShell** ; sous macOS ou Linux, les nouvelles séquences de ce guide utilisent **Bash** (tu peux taper `bash` dans le terminal zsh de macOS). Les blocs PowerShell ne se collent pas dans Bash. Exécute une commande à la fois et lis son résultat. Ne copie pas les trois accents qui entourent les blocs.

Note le chemin complet de ton futur dossier `CISIA_24082026_Parcours` dans ton journal. Il sera redemandé au début de chaque demi-journée et dans tout nouveau terminal. Si `code` est introuvable, utilise **Fichier > Ouvrir le dossier** dans VS Code ; il n'est pas nécessaire de réinstaller VS Code.

Si le dossier existe déjà

Ouvre le dossier dans VS Code et lance les contrôles suivants. Ce bloc convient aux trois systèmes :

```
pwd
git rev-parse --show-toplevel
git status --short
git remote -v
git branch --show-current
```

La racine Git doit être le dossier Parcours, et `origin` doit désigner l'URL officielle ci-dessus. Sous PowerShell, `pwd` est un alias de `Get-Location`. Si Git répond « not a git repository », vérifie le dossier avant d'initialiser quoi que ce soit. Si la racine Git est un autre projet parent, arrête : ne crée pas de dépôt imbriqué. Si des fichiers modifiés apparaissent, garde-les ; fais le checkpoint expliqué à la fin de la demi-journée avant de changer de branche ou de récupérer un jalon.

Si tu n as pas encore le dépôt

Place-toi dans le **dossier parent**, qui ne contient pas déjà un dossier Parcours. Le clone initialise Git et récupère les fichiers du cours ; il ne faut pas ajouter `git init` après un clone.

Windows PowerShell :

```
$parent = Read-Host 'Chemin complet du dossier parent de tes projets'
Set-Location -LiteralPath $parent -ErrorAction Stop
if (Test-Path -LiteralPath .\CISIA_24082026_Parcours) { throw 'Dossier déjà présent : utilise
la procédure de vérification.' }
git clone --branch main https://github.com/thomasfesq/CISIA_24082026_Parcours.git
if ($LASTEXITCODE -ne 0) { throw 'Clone non terminé : lire le message Git.' }
```

```
Set-Location -LiteralPath .\CISIA_24082026_Parcours -ErrorAction Stop
$branche = Read-Host 'Identifiant personnel sans espace par exemple prenom-nom'
if ($branche -notmatch '^[A-Za-z0-9][A-Za-z0-9._-]*$' -or $branche -eq 'main') { throw
'Identifiant invalide.' }
git switch --no-track -c $branche
git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
git status --short
git remote -v
git branch --show-current
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
code .
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier parent de tes projets : '
IFS= read -r parent
cd "$parent" || exit 1
if [ -e CISIA_24082026_Parcours ]; then
  printf 'Dossier déjà présent : utilise la vérification.\n'
  exit 1
else
  git clone --branch main https://github.com/thomasfesq/CISIA_24082026_Parcours.git
fi
cd CISIA_24082026_Parcours || exit 1
printf 'Identifiant personnel sans espace par exemple prenom-nom : '
IFS= read -r branche
git check-ref-format --branch "$branche" || exit 1
case "$branche" in main|jalon/*|'') printf 'Choisis une branche personnelle.\n'; exit 1;;
esac
git switch --no-track -c "$branche" || exit 1
git rev-parse --show-toplevel
git status --short
git remote -v
git branch --show-current
head -n 1 FORMATION/JALON_ACTUEL.md
code .
```

Attendu : ta branche personnelle, le remote `origin` officiel et `# Jalon actuel : 00-reconstruction-fin-sprint2`. Le formateur donne ensuite le signal pour charger le jalon 01. Si la branche personnelle existe déjà, utilise `git switch` avec son nom, sans `-c`, après avoir vérifié que le dépôt est propre.

Alternative pour un dossier réellement vide seulement. Si le formateur demande une initialisation manuelle au lieu du clone, vérifie que ce nouveau dossier n'a aucun fichier et n'appartient à aucun dépôt parent, puis exécute :

```
git init -b main
git remote add origin https://github.com/thomasfesq/CISIA_24082026_Parcours.git
git fetch origin main
```

Crée ensuite ta branche personnelle **depuis** `origin/main` : PowerShell `git switch --no-track -c $branche origin/main` ; Bash `git switch --no-track -c "$branche" origin/main`. Saisis d'abord ton identifiant comme dans le clone. Attends le succès de chaque commande. Cette alternative ne s'applique jamais à un dossier contenant ton travail.

Configurer la sauvegarde personnelle sans toucher à origin

Dans ton compte GitHub déjà utilisé pour la formation, vérifie ou crée un dépôt **dédié à tes travaux Sprint 3**. Pour un nouveau dépôt, laisse-le vide : pas de README, licence ou gitignore ajoutés par GitHub. Copie son URL HTTPS ou SSH depuis le bouton **Code**. N'y insère jamais de mot de passe ou de token. L'authentification se fait dans le mécanisme proposé par Git/GitHub. Si tu n'as pas les droits nécessaires, conserve les commits locaux et fais préciser la destination par le formateur.

Vérifie d'abord `git remote -v`. Si `perso` existe déjà, compare son URL avec celle de ton dépôt : ne relance pas `remote add` et ne remplace pas un remote existant sans avoir identifié son propriétaire.

Windows PowerShell, seulement si `perso` est absent :

```
$urlPerso = Read-Host 'URL HTTPS ou SSH du dépôt personnel sans token'
git remote add perso $urlPerso
git remote -v
git config --local --get user.name
git config --local --get user.email
```

Bash, seulement si `perso` est absent :

```
printf 'URL HTTPS ou SSH du dépôt personnel sans token : '
IFS= read -r url_perso
git remote add perso "$url_perso"
git remote -v
git config --local --get user.name
git config --local --get user.email
```

Si ton nom ou ton adresse Git manque, configure-les localement en saisissant ton propre nom et l'adresse associée à ton compte. PowerShell : `$nomGit = Read-Host 'Ton nom Git'; $emailGit = Read-Host 'Ton adresse Git'; git config --local user.name $nomGit; git config --local user.email $emailGit`. Bash : `IFS= read -r nom_git; IFS= read -r email_git; git config --local user.name "$nom_git"; git config --local user.email "$email_git"`. Aucune adresse n'est imposée par ce guide.

Le premier envoi de la branche utilise `git push -u perso` avec ton nom de branche ; les suivants utilisent `git push perso` avec le même nom. Ces commandes sont répétées à chaque clôture. **origin apporte le cours, perso conserve ton travail.** Un `pull --ff-only` personnel ne remplace pas la fusion pédagogique des jalons, qui peut créer un commit de fusion.

Utiliser les supports

Lis aussi `notice_preparation_et_controles_sprint3.md` et `memo_metriques_sans_confusion.md` du pack REX. Les blocs indiquent des résultats **attendus** : ils ne deviennent tes preuves qu'après exécution sur ton poste. Une erreur doit être lue et corrigée avant de poursuivre. Les scripts de jalon installent un environnement et exécutent tests et Ruff ; ce sont des actions distinctes de la seule récupération Git.

Ton terminal à lire une fois ça t évitera la moitié des blocages

Sous **Windows**, utilisez PowerShell dans le terminal intégré de VS Code (`Terminal > New Terminal`). Sous **macOS**, ouvre le terminal de VS Code puis tape `bash` pour les blocs `bash` ; sous **Linux**, utilise bash. Dans les trois cas, ouvrez le terminal dans le dossier du projet. Les blocs `powershell` sont la référence Windows ; leur équivalent macOS/Linux se trouve dans le guide multiplateforme lié ci-dessus.

Sais quelle version tu as (à faire maintenant) :

```
$PSVersionTable.PSVersion
```

- **Major = 5** → tu es sur **Windows PowerShell 5.1**, celui livré avec Windows. Il fonctionne très bien

pour tout le sprint, avec **une seule réserve** : il **ne connaît pas l'opérateur `&&`**. Si tu colles une commande contenant `&&`, il répond `Le jeton « && » n'est pas un séparateur d'instruction valide.`

- **Major = 7** → tu es sur **PowerShell 7**, `&&` fonctionne. Rien à faire.

Si une commande refuse `&&`, coupe la ligne en commandes séparées. Vérifie le résultat avant de lancer une action qui dépend de la précédente. Exemple :

Ce qui peut échouer	Ce qui marche toujours
`cd indusense-gameday && git switch -c reparation-x`	`cd indusense-gameday` *(entrée)* puis `git switch -c reparation-x`
`uv run pytest -q && uv run ruff check .`	`uv run pytest -q; uv run ruff check .`

⚠ **Nuance à connaître** : `&&` veut dire « **seulement si la précédente a réussi** », `;` veut dire « **ensuite, quoi qu'il arrive** ». Pour un **diagnostic** (voir toutes les versions), `;` est même préférable. Pour un **enchaînement où l'échec doit stopper la suite** (ex. « teste puis publie »), préviens le formateur plutôt que d'improviser.

💡 **Certains blocs sont marqués `` `bash` `` (surtout dans les fiches TD)** : ce sont des commandes pensées pour **macOS/Linux/WSL2/Git Bash**, ou pour un **runner de CI**. Les commandes `uv`, `git`, `docker`, `pytest` s'écrivent pareil dans les deux mondes ; ce qui diffère, ce sont les **enchaînements** (`&&`), les **chemins** (`\` contre `/`) et les commandes propres au shell (`printf`, `export`, `cat`). En cas de doute : demande.

Ta progression GitHub un jalon au début de chaque demi journée

Dépôt officiel du groupe : `https://github.com/thomasfesq/CISIA_24082026_Parcours`. `main` est l'état de départ reconstruit à la fin du Sprint 2. Les branches distantes s'appellent exactement `jalon/01` à `jalon/12`. **Attends toujours l'annonce du formateur. Ne fais jamais `git switch jalon/NN` : reste sur ta branche personnelle.** Les scripts `mettre_a_niveau` et `verifier_jalon` publiés le 24/08/2026 acceptent le numéro court `NN` et ciblent exactement `jalon/NN`. Un ancien nom long exact reste accepté par compatibilité, mais le numéro court est la notation officielle.

Quand le formateur donne le signal	Ce que tu charges	Première ligne attendue de `FORMATION/JALON_ACTUEL.md`
J1 24/08, à l'ouverture	clone de `main`	`# Jalon actuel : 00-reconstruction-fin-sprint2`
J1 matin, au signal	`jalon/01`, avant M23	`# Jalon actuel : 01-j1-matin-m23`
J1 après-midi, au signal	`jalon/02`, avant M24	`# Jalon actuel : 02-j1-apres-midi-m24`
J2 matin, au signal	`jalon/03`, avant M25	`# Jalon actuel : 03-j2-matin-m25`
J2 après-midi, au signal	`jalon/04`, avant M26	`# Jalon actuel : 04-j2-apres-midi-m26`
J3 matin, au signal	`jalon/05`, avant M27	`# Jalon actuel : 05-j3-matin-m27`
J3 après-midi, au signal	`jalon/06`, avant M28	`# Jalon actuel : 06-j3-apres-midi-m28`
J4 matin, au signal	`jalon/07`, avant M29 puis M30	`# Jalon actuel : 07-j4-matin-m29-m30`
J4 après-midi, au signal	`jalon/08`, avant PayGuard	`# Jalon actuel : 08-j4-apres-midi-m31-m32-payguard`
J5 matin, au signal	`jalon/09`, avant InduSense	`# Jalon actuel : 09-j5-matin-m31-m32-indusense`
J5 après-midi, au signal	`jalon/10`, avant M33 puis M34	`# Jalon actuel : 10-j5-apres-midi-m33-m34`
J6 matin, au signal	`jalon/11`, avant le Game Day	`# Jalon actuel : 11-j6-matin-gameday`

Quand le formateur donne le signal	Ce que tu charges	Première ligne attendue de `FORMATION/JALON_ACTUEL.md`
J6 après-midi, au signal	`jalon/12`, dans Parcours seulement	`# Jalon actuel : 12-j6-apres-midi-retex`

Avant chaque jalon : enregistre ton travail, ouvre le panneau **Contrôle de code source** de VS Code, relis les fichiers modifiés et fais un commit de checkpoint. Ne versionne jamais `.env`, une clé, un token ou un secret. Puis vérifie que `git status --short` ne renvoie rien.

Windows PowerShell — remplace `NN` par le numéro annoncé :

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon NN
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon NN
```

macOS zsh ou Linux bash — remplace `NN` par le numéro annoncé :

```
bash scripts/formation/mettre_a_niveau.sh NN
bash scripts/formation/verifier_jalon.sh NN
```

`mettre_a_niveau` contrôle ta branche et ton dépôt propre, crée une sauvegarde locale, récupère le jalon annoncé et le fusionne sans réécrire l'historique. `verifier_jalon` contrôle ensuite le marqueur, recrée l'environnement avec `uv`, lance les tests puis Ruff : c'est la preuve que ton poste est aligné, pas seulement que Git a fusionné.

☑ **Tu peux continuer seulement si** la sortie affiche `Jalon integre sur <ta-branche> : jalon/NN`, puis `Jalon verifie : jalon/NN`, si la première ligne du marqueur est exactement celle du tableau et si ta branche porte toujours ton nom. Ne relance pas les scripts simplement parce que tu as rouvert VS Code : attends le jalon annoncé. En cas de conflit, la fusion est annulée ; appelle le formateur. Il pourra relancer avec `-Rattrapage` sous Windows ou `--rattrapage` sous macOS/Linux. Ton travail reste dans ta branche personnelle et dans `sauvegarde/...`. Ne lance jamais `reset --hard`.

Plan B manuel, uniquement si le script est absent ou illisible et avec le formateur :

```
git branch --show-current
git status --short
git fetch origin
git merge --no-edit origin/jalon/NN
git branch --show-current
```

0 1 Accueil cap

- Caméras allumées si possible, partage d'écran prêt sur la slide de titre du module 23.

0 2 Contrôle d entrée des postes

Dans le terminal du projet, relève `uv --version`, `git --version` et `uv run python --version` (3.13.x attendu). Vérifie VS Code. Docker sera requis pour J3 ; `docker --version` prouve le client, `docker version` doit aussi montrer le moteur. WSL version 2 concerne les postes Windows qui l'utilisent. N'effectue pas un second clone après la préparation Git.

0 3 Point projet fil rouge

- Au tour de table, **présente où tu en es** de ton projet fil rouge en fin de Sprint 2.

☑ **Résultat attendu :** Checklist go/no-go (cf. note de cadrage §5) : model card v1 · contrat I/O · artefacts · seuil · runs MLflow · Gold Dataset. **Si l'un manque**, tu le combles en autonomie cette semaine.

0 4 Bascule vers le premier module

- 👀 La slide de titre du module 23.

23 Refactoring structure projet J1 matin

✓ **Preuve finale visée :** `uv run pytest -q` vert · `uv run ruff check .` propre · `uv run indusense --help` répond (train/predict).

Début de demi journée J1 matin M23

Terminal du dépôt Parcours. Même si tu y travaillais hier, redonne le chemin complet ; cela évite un `Set-Location -LiteralPath $projet` depuis un dossier déjà ouvert. Arrête d'abord le serveur qui occupe ce terminal avec Ctrl+C si nécessaire. Ne lance pas de Git dans le terminal occupé par un serveur.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
pwd
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
git status --short
git remote -v
git branch --show-current
if (git status --porcelain) { throw 'Travail non commité : fais un checkpoint avant la
récupération.' }
$branche = Read-Host 'Nom exact de ta branche personnelle du Sprint 3'
if ([string]::IsNullOrWhiteSpace($branche) -or $branche -eq 'main' -or $branche -like
'jalon/*') { throw 'Utilise ta branche personnelle.' }
git switch $branche
if ($LASTEXITCODE -ne 0) { throw 'Branche non ouverte : lire le message Git.' }
code .
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Remote perso absent : configure ta destination de
sauvegarde avant la suite.' }
$refPerso = git ls-remote --heads perso $branche
if ($LASTEXITCODE -ne 0) { throw 'Accès distant échoué : ne pas continuer comme si le pull
avait réussi.' }
if ($refPerso) {
    git pull --ff-only perso $branche
    if ($LASTEXITCODE -ne 0) { throw 'Pull refusé : conserve ton travail et appelle le
formateur.' }
} else { Write-Host 'Premier passage : branche encore absente de perso, premier push à la
clôture.' }
git status --short
git branch --show-current
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
pwd
racine_git="$(git rev-parse --show-toplevel)" || exit 1
```

```
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
git status --short
git remote -v
git branch --show-current
[ -z "$(git status --porcelain)" ] || { printf 'Fais un checkpoint avant la récupération.\n';
exit 1; }
printf 'Nom exact de ta branche personnelle du Sprint 3 : '
IFS= read -r branche
case "$branche" in main|jalon/*|'') printf 'Utilise ta branche personnelle.\n'; exit 1;; esac
git switch "$branche" || exit 1
code .
git remote get-url perso || exit 1
ref_perso="$(git ls-remote --heads perso "$branche")" || exit 1
if [ -n "$ref_perso" ]; then
  git pull --ff-only perso "$branche" || exit 1
else
  printf 'Branche absente de perso : premier push à la clôture.\n'
fi
git status --short
git branch --show-current
```

Attendu : racine Parcours, branche personnelle et remote de sauvegarde corrects. Si `status` n'est pas vide, enregistre tes fichiers, lis le diff et sélectionne ceux à commiter comme dans la clôture ci-dessous. Ne récupère pas un jalon par-dessus du travail non commité. Si le pull échoue pour authentification, vérifie le compte et les droits dans le mécanisme GitHub proposé ; ne colle aucun token dans une commande. Un refus `--ff-only` indique des historiques divergents : fais les contrôles avec le formateur, sans reset ni push forcé.

Au signal du formateur, intègre **jalon/01** sur cette même branche avec les deux commandes suivantes. Le script effectue la fusion nécessaire des commits du cours ; `--ff-only` ci-dessus concernait uniquement la sauvegarde personnelle.

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 01
if ($LASTEXITCODE -ne 0) { throw 'Jalon non intégré : lire le diagnostic, ne pas lancer le
module.' }
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 01
if ($LASTEXITCODE -ne 0) { throw 'Vérification du jalon échouée.' }
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
git branch --show-current
```

```
bash scripts/formation/mettre_a_niveau.sh 01 || exit 1
bash scripts/formation/verifier_jalon.sh 01 || exit 1
head -n 1 FORMATION/JALON_ACTUEL.md
git branch --show-current
```

Attendu : `Jalon integre sur … : jalon/01`, puis `Jalon verifie : jalon/01`, le marqueur correspondant au tableau de progression, et ta branche personnelle. En cas de conflit, le script annule sa fusion ; lis le diagnostic avec le formateur avant un éventuel rattrapage.

Préflight dépannage poste AVANT la théorie à faire dès l ouverture

- Lance le contrôle express. Sous Windows, `python --version` peut afficher **3.14** : ce n'est pas grave, on cible le venv du projet en 3.13.

☑ **Résultat attendu :** Ce qui fait foi, c'est `uv run python --version` dans le projet (doit dire **3.13.x**). `code --version` et `docker --version` ne sont pas bloquants pour le matin.

Les cas Windows fréquents à régler tout de suite :

```
# 1) uv absent du PATH (cas fréquent) :
winget install --id astral-sh.uv -e         # puis FERMER / ROUVRIR le terminal
uv --version

# 2) depuis la racine du dépôt déjà cloné :
if (-not (Test-Path -LiteralPath .\pyproject.toml) -or -not (Test-Path -LiteralPath
.\uv.lock)) {
    throw "Ouvre le terminal dans la racine CISIA_24082026_Parcours (pyproject.toml et
uv.lock attendus)."
}
uv venv --python 3.13
uv sync --frozen --extra dev
uv run python --version                  # DOIT afficher Python 3.13.x  <-- fait foi
git status --short -- uv.lock            # aucune sortie attendue

# 3) VS Code : commande 'code' introuvable ?
Get-Command code -ErrorAction SilentlyContinue   # si rien ne sort :
winget install --id Microsoft.VisualStudioCode -e   # puis rouvrir le terminal, ou ouvrir VS
Code à la main
```

Alignement local avant M23. Le dépôt `CISIA_24082026_Parcours` contient déjà le socle corrigé utilisé par ce livret. N'applique aucun ancien script `APPLIQUER_CORRECTIF_SPRINT3.ps1` provenant d'un kit antérieur : le marqueur du jalon 01 et les tests ci-dessous sont les preuves qui font foi.

💡 **Hors ligne.** Utilise la copie `CISIA_24082026_Parcours` et le `.venv` préparés la veille. Si le dépôt n'est pas présent, demande la copie locale du kit : ne relance pas un clone au milieu du dossier courant. M23 n'a besoin ni de GPU ni de Docker.

Ouvrir le Sprint 3 scénario B seulement

- Vérifie tes **prérequis Sprint 2**, un par un : modèle candidat retenu · **model card v1** · **contrat I/O JSON** · seuils de décision · artefacts modèle · Gold Dataset / données disponibles.
- 👀 `note_cadrage_formateur_sprint3` → section « Point projet fil rouge ».

Pourquoi on sort du notebook

- 👀 Le contraste à l'écran :

```
# avant — cellule de notebook (logique dupliquée, fragile)
df["temp_lag1"] = df.groupby("machine_id")["temperature"].shift(1)

# après — package (une seule source de vérité, testée)
from indusense.features.temporal import add_temporal_features
df = add_temporal_features(df)
```

Montrer le squelette

- Reste dans le dossier `CISIA_24082026_Parcours` validé au préflight.
- 👀 Précisément : `pyproject.toml` (identité, Python `>=3.13,<3.14`, deps, script `indusense`) · `src/indusense/features/temporal.py` (le `shift(1)` **avant** le `rolling`) · `src/indusense/data/loaders.py` (`normalize_machine_id`) · `tests/test_temporal.py` (suite anti-fuite, tri temporel et colonne manquante) · `src/indusense/cli.py` (train / predict).
- À lancer ensemble :

```
uv --version
uv sync --frozen --extra dev
Test-Path -LiteralPath .\pyproject.toml
```

```
Test-Path -LiteralPath .\uv.lock
uv run python -c "import indusense; print(indusense.__file__)"
```

☑ **Résultat attendu :** les deux contrôles renvoient `True` et l'import affiche `CISIA_24082026_Parcours\src\indusense\__init__.py`. Si `uv` est introuvable → cf. Préflight (`winget install --id astral-sh.uv -e`).

Pause obligatoire en visio

- Coupe caméra et partage d'écran, éloigne-toi du poste et reviens au signal du formateur. Ne lance aucune commande pendant la pause.

TP 1 Structure pyproject

- Explore `src/indusense/`, `tests/`, `data/`, `artifacts/` et classe ce qui relève de `data` / `features` / `models` / `api` / `cli`.
- 👀 `pyproject.toml` **bloc par bloc** (détail dans la fiche TD 23 / le manuel de révision) : `[build-system]` → `[project]` (nom, version, `requires-python`) → `dependencies` → `[project.optional-dependencies] dev` → `[project.scripts]` (`indusense = "indusense.cli:main"`) → `[tool.hatch…]` → `[tool.ruff]` / `[tool.pytest…]`.
- Audite le contrat sans le modifier : `Select-String -Path .\pyproject.toml -Pattern 'requires-python','optional-dependencies','indusense\s*='`, puis `git diff -- pyproject.toml uv.lock` (aucune sortie attendue).

TP 2 Anti leakage

- Ouvre `features/temporal.py`, repère le **tri par machine et timestamp**, comprends pourquoi `shift(1)` est **avant** `rolling`, puis complète / rejoue le test anti-fuite.
- `uv run pytest tests/test_temporal.py -q`

☑ **Résultat attendu :** 0 échec ; ce test prouve la feature décalée. Le split train/test temporel est un mécanisme distinct, traité dans l'exercice avancé.

TP 3 Normalisation des machines

- Prédit puis exécute :

```
$normalizationCheck = @'
from indusense.data.loaders import normalize_machine_id as n
ids = ("MACH-01", "MACH_01", "M-06", "M-2")
print([n(raw) for raw in ids])
'@
uv run python -c $normalizationCheck
uv run pytest tests/test_loaders.py -q -k normalize_machine_id
```

- Résultat exact : `['MACH-01', 'MACH-01', 'MACH-06', 'MACH-02']` et 0 échec.

TP 3 suite et extension facultative

- 👀 `normalize_machine_id` dans `loaders.py`.
- *(Si le groupe avance)* code **nouveau** à taper — extraire une fonction propre : créer `src/indusense/features/cleaning.py` + `tests/test_cleaning.py` (code fourni dans la **fiche TD 23**), puis `uv run pytest tests/test_cleaning.py -q`.

Preuve finale

- Tu produis :

```
uv run pytest -q
uv run ruff check .
uv run indusense --help
uv run python --version
```

☑ **Résultat attendu :** tests à **0 échec** (sans compteur imposé) · ruff propre · CLI qui répond (`train`/`predict`) · Python 3.13.x. **Partage tes sorties terminal** dans le chat.

À ne pas oublier FAQ définitions

📖 **Définition. Fuite de données** : info du **futur** qui entre dans l'entraînement → score faussement excellent. **Package** = dossier importable ; **module** = un fichier `.py` ; **refactoring** = réorganiser sans changer le comportement. ❓ « `src/` layout vs code à la racine ? » → avec `src/`, les tests s'exécutent contre le **package installé** (comme en prod) → on attrape les erreurs de packaging. ❓ « Pourquoi `uv` et pas `pip` ? » → plus rapide **et** un `uv.lock` (versions exactes) → environnements identiques partout. 🔑 À ne pas oublier de dire : **un score trop beau doit inquiéter** ; le découpage **temporel** n'est pas optionnel.

QCM express transition

- Réponds aux questions J1 1-3 : package `src/`, rôle de `pyproject.toml`, fuite par split non temporel. La correction attendue est **1-B · 2-C · 3-A**, avec justification.
- 👀 Supports à garder ouverts : `23_module_refactoring_structure_projet` · `fiches_TD_apprenant_sprint3` · deck apprenant publié `01_PRESENTATIONS_PPTX/23_slides_refactoring_structure.pptx`.

Fin de demi journée J1 matin M23

Enregistre les fichiers dans VS Code. Retourne à la **racine Parcours** avant de sauvegarder. Les chemins ci-dessous correspondent aux fichiers travaillés dans cette demi-journée : retire ceux que tu n'as pas modifiés et ajoute uniquement les nouveaux fichiers que tu as réellement créés et relus. Les dossiers temporaires, `.env`, clés, bases SQLite, captures contenant des données privées et environnements `.venv` restent hors Git. Si `git status` signale une suppression volontaire, inspecte-la avec le formateur avant son ajout.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
$brancheAttendue = Read-Host 'Nom exact de ta branche personnelle choisie en début de demi-
journée'
$branche = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheAttendue) -or $branche -cne
$brancheAttendue -or $branche -eq 'main' -or $branche -like 'jalon/*') { throw 'Branche
courante différente de ta branche personnelle attendue : arrêt avant ajout, commit et push.'
}
git status --short
git diff --stat
git diff
```

```
$fichiers = @('src/indusense/features/temporal.py', 'src/indusense/data/loaders.py',
'src/indusense/features/cleaning.py', 'tests/test_temporal.py', 'tests/test_loaders.py',
'tests/test_cleaning.py')
# Relis et adapte cette liste avant de poursuivre.
$existants = @($fichiers | Where-Object { Test-Path -LiteralPath $_ })
if ($existants.Count -gt 0) {
    git add -- $existants
    if ($LASTEXITCODE -ne 0) { throw 'Ajout Git échoué : ne pas commiter.' }
}
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Corrige les erreurs signalées avant le commit.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J1 matin M23 travail et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire identité Git, hook ou conflit.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Impossible de contrôler les changements indexés.' }
git status --short
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Sauvegarde distante non configurée : commits locaux
conservés.' }
$suivi = git for-each-ref --format="%(upstream:short)" "refs/heads/$branche"
if ($suivi -cne "perso/$branche") {
    git push -u perso $branche
} else { git push perso $branche }
if ($LASTEXITCODE -ne 0) { throw 'Push échoué : ta sauvegarde distante reste à faire.' }
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso $branche
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
printf 'Nom exact de ta branche personnelle choisie en début de demi-journée : '
IFS= read -r branche_attendue
branche="$(git branch --show-current)" || exit 1
[ -n "$branche_attendue" ] && [ "$branche" = "$branche_attendue" ] || { printf 'Branche
différente de la branche personnelle attendue.\n'; exit 1; }
case "$branche" in main|jalon/*|'') printf 'Branche personnelle requise avant ajout, commit
et push.\n'; exit 1;; esac
git status --short
git diff --stat
git diff
fichiers=("src/indusense/features/temporal.py" "src/indusense/data/loaders.py"
"src/indusense/features/cleaning.py" "tests/test_temporal.py" "tests/test_loaders.py"
"tests/test_cleaning.py")
# Relis et adapte cette liste avant de poursuivre.
for fichier in "${fichiers[@]}"; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
```

```
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then
  git commit -m "Sprint 3 J1 matin M23 travail et preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then
  printf 'Contrôle du diff impossible.\n'; exit 1
fi
git status --short
git remote get-url perso || exit 1
suivi="$(git for-each-ref --format='%(upstream:short)' "refs/heads/$branche")"
if [ "$suivi" != "perso/$branche" ]; then
  git push -u perso "$branche" || exit 1
else
  git push perso "$branche" || exit 1
fi
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso "$branche"
```

Attendu : un commit seulement si tu as des changements, un push terminé sans erreur et **le même hash** pour `HEAD` et la branche distante listée par `ls-remote`. Vérifie aussi ton dernier commit dans le dépôt personnel GitHub. `Everything up-to-date` est normal si tout était déjà envoyé. `nothing to commit` ne justifie pas un commit vide : vérifie et pousse les commits locaux éventuels. Si des fichiers restent dans `status`, conserve-les et identifie-les ; tu n'es pas encore prêt pour le prochain jalon.

Si le commit échoue à cause d'un hook qui reformate un fichier, relis le nouveau diff et indexe à nouveau le fichier avant un nouveau commit. Si le push répond `rejected` ou `non-fast-forward`, ne force pas : un autre historique distant doit être réconcilié avec le formateur. Si un conflit est déjà en cours, `git status` indique les fichiers concernés ; travaille avec le formateur avant d'ajouter et de commiter la résolution.

24 CI CD tests versioning J1 après midi

✓ **Preuve finale visée :** `uv run --frozen pytest -q` · `uv run --frozen ruff check .` · `uv run --frozen black --check .` · `uv run --frozen pre-commit run --all-files` · pipeline CI **quality + build** vert · deux pointeurs DVC · deux `run_id` + deux versions MLflow · `versioning_strategy.md` · **0 secret**.

Ce que le jalon 02 te donne — et ce qu'il ne fait pas à ta place. Il fournit le job CI `quality`, la configuration pre-commit non activée, les dépendances verrouillées, `scripts/demo_versioning.py`, les données/modèle et le modèle `docs/versioning_strategy.md`. Tu dois encore activer les hooks, ajouter le job `build`, réaliser le cycle rouge → vert, produire les preuves DVC/MLflow et créer `versioning_strategy.md` à la racine. Le jalon 03 conservera tes commits par fusion ; il ne livre pas le corrigé M24.

Début de demi journée J1 après midi M24

Terminal du dépôt Parcours. Même si tu y travaillais hier, redonne le chemin complet ; cela évite un `Set-Location -LiteralPath $projet` depuis un dossier déjà ouvert. Arrête d'abord le serveur qui occupe ce terminal avec Ctrl+C si nécessaire. Ne lance pas de Git dans le terminal occupé par un serveur.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
pwd
$racineGit = git rev-parse --show-toplevel
```

```
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
git status --short
git remote -v
git branch --show-current
if (git status --porcelain) { throw 'Travail non commité : fais un checkpoint avant la
récupération.' }
$branche = Read-Host 'Nom exact de ta branche personnelle du Sprint 3'
if ([string]::IsNullOrWhiteSpace($branche) -or $branche -eq 'main' -or $branche -like
'jalon/*') { throw 'Utilise ta branche personnelle.' }
git switch $branche
if ($LASTEXITCODE -ne 0) { throw 'Branche non ouverte : lire le message Git.' }
code .
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Remote perso absent : configure ta destination de
sauvegarde avant la suite.' }
$refPerso = git ls-remote --heads perso $branche
if ($LASTEXITCODE -ne 0) { throw 'Accès distant échoué : ne pas continuer comme si le pull
avait réussi.' }
if ($refPerso) {
    git pull --ff-only perso $branche
    if ($LASTEXITCODE -ne 0) { throw 'Pull refusé : conserve ton travail et appelle le
formateur.' }
} else { Write-Host 'Premier passage : branche encore absente de perso, premier push à la
clôture.' }
git status --short
git branch --show-current
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
pwd
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
git status --short
git remote -v
git branch --show-current
[ -z "$(git status --porcelain)" ] || { printf 'Fais un checkpoint avant la récupération.\n';
exit 1; }
printf 'Nom exact de ta branche personnelle du Sprint 3 : '
IFS= read -r branche
```

```
case "$branche" in main|jalon/*|'') printf 'Utilise ta branche personnelle.\n'; exit 1;; esac
git switch "$branche" || exit 1
code .
git remote get-url perso || exit 1
ref_perso="$(git ls-remote --heads perso "$branche")" || exit 1
if [ -n "$ref_perso" ]; then
  git pull --ff-only perso "$branche" || exit 1
else
  printf 'Branche absente de perso : premier push à la clôture.\n'
fi
git status --short
git branch --show-current
```

Attendu : racine Parcours, branche personnelle et remote de sauvegarde corrects. Si `status` n'est pas vide, enregistre tes fichiers, lis le diff et sélectionne ceux à commiter comme dans la clôture ci-dessous. Ne récupère pas un jalon par-dessus du travail non commité. Si le pull échoue pour authentification, vérifie le compte et les droits dans le mécanisme GitHub proposé ; ne colle aucun token dans une commande. Un refus `--ff-only` indique des historiques divergents : fais les contrôles avec le formateur, sans reset ni push forcé.

Au signal du formateur, intègre **jalon/02** sur cette même branche avec les deux commandes suivantes. Le script effectue la fusion nécessaire des commits du cours ; `--ff-only` ci-dessus concernait uniquement la sauvegarde personnelle.

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 02
if ($LASTEXITCODE -ne 0) { throw 'Jalon non intégré : lire le diagnostic, ne pas lancer le module.' }
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 02
if ($LASTEXITCODE -ne 0) { throw 'Vérification du jalon échouée.' }
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
git branch --show-current
```

```
bash scripts/formation/mettre_a_niveau.sh 02 || exit 1
bash scripts/formation/verifier_jalon.sh 02 || exit 1
head -n 1 FORMATION/JALON_ACTUEL.md
git branch --show-current
```

Attendu : `Jalon integre sur … : jalon/02`, puis `Jalon verifie : jalon/02`, le marqueur correspondant au tableau de progression, et ta branche personnelle. En cas de conflit, le script annule sa fusion ; lis le diagnostic avec le formateur avant un éventuel rattrapage.

Reprise

- À relancer :

```
Set-Location -LiteralPath (Read-Host 'Chemin complet du dépôt Parcours') -ErrorAction Stop
uv sync --frozen --extra dev --extra mlops
uv run --frozen pytest -q
uv run --frozen ruff check .
git status
```

La CI le robot qualité

- 👀 `.github/workflows/ci.yml` — insiste sur `python-version: "3.13"`, `uv sync --frozen --extra dev`, `uv run --frozen ruff check .`, `uv run --frozen black --check .`, `uv run --frozen pytest -q`.

Démo PR rouge verte

- Si GitHub est prêt : ouvre l'onglet **Actions** (un job qui passe ou qui échoue). Sinon, en local : casse volontairement un test et observe le **log rouge**.
- Corrige, puis `uv run --frozen pytest -q` (vert).

pre commit secrets

- 👀 `.pre-commit-config.yaml` · `.gitignore`.

Pause obligatoire en visio

- Coupe caméra et partage d'écran ; reviens à l'heure annoncée. Ne lance pas le hook ni la démonstration de secret pendant la pause.

TP 1 pre commit déjà fourni dans le repo

```
# pre-commit est DÉJÀ une dépendance dev et .pre-commit-config.yaml est déjà fourni
uv sync --frozen --extra dev
uv run --frozen pre-commit install --hook-type pre-commit --hook-type pre-push
uv run --frozen pre-commit run --all-files
```

☑ **Résultat attendu :** ruff corrige tout seul · black reformate · gitleaks **bloque** un secret.

- Démo « secret bloqué » **réversible** : crée une sentinelle **explicitement fausse**, lance **seulement** le hook gitleaks (on ne tente pas de vrai commit), puis nettoie. ⚠ **Jamais une vraie clé, même révoquée.**

```
@'
# Valeurs d'exemple publiques AWS, invalides, réservées à la documentation.
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
'@ | Set-Content -LiteralPath .\fuite_demo.txt -Encoding ascii
git add -f -- .\fuite_demo.txt
uv run --frozen pre-commit run gitleaks --files .\fuite_demo.txt
git restore --staged -- .\fuite_demo.txt
Remove-Item -LiteralPath .\fuite_demo.txt
```

```
# macOS/Linux — même sentinelle publique et invalide
cat > fuite_demo.txt <<'EOF'
# Valeurs d'exemple publiques AWS, invalides, réservées à la documentation.
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
EOF
git add -f -- fuite_demo.txt
uv run --frozen pre-commit run gitleaks --files fuite_demo.txt
git restore --staged -- fuite_demo.txt
rm -- fuite_demo.txt
```

☑ **Résultat attendu :** le hook `gitleaks` **échoue**, avec `RuleID: aws-access-token` et `leaks found: 1`. Puis `git status --short` **ne montre plus** `fuite_demo.txt` : rien n'a été commité, le dépôt est propre.

TP 2 Workflow GitHub Actions job build

- Ouvre `.github/workflows/ci.yml`, identifie les étapes, vérifie Python en **3.13**, et **ajoute le job `build`** (le repo a `quality` mais pas `build`) :

```
build:
  needs: quality
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.13"
    - uses: astral-sh/setup-uv@v3
    - name: Build wheel
      run: uv build
    - name: Upload artifact
      uses: actions/upload-artifact@v4
```

```
      with:
        name: indusense-wheel
        path: dist/*.whl
```

Versioning données DVC

- En concept ou réel selon le setup (on versionne le **gold** et le **modèle**) :

```
# DVC + MLflow sont verrouillés : ne lance jamais `uv add` ni `uv lock`.
if (-not (Test-Path -LiteralPath .\.dvc)) { uv run --frozen dvc init }
New-Item -ItemType Directory -Force -Path ..\dvc-store | Out-Null
uv run --frozen dvc remote add -d -f localstore ..\dvc-store
foreach ($artefact in @('data/gold/gold_dataset.csv', 'artifacts/models/rf.joblib')) {
  git ls-files --error-unmatch -- $artefact 2>$null | Out-Null
  if ($LASTEXITCODE -eq 0) { git rm --cached -- $artefact }
}
uv run --frozen dvc add data/gold/gold_dataset.csv artifacts/models/rf.joblib
uv run --frozen dvc push
git status --short
uv run --frozen dvc status
```

- **macOS/Linux (Terminal bash ou zsh)** :

```
uv sync --frozen --extra dev --extra mlops
test -d .dvc || uv run --frozen dvc init
mkdir -p ../dvc-store
uv run --frozen dvc remote add -d -f localstore ../dvc-store
for artefact in data/gold/gold_dataset.csv artifacts/models/rf.joblib; do
  git ls-files --error-unmatch -- "$artefact" >/dev/null 2>&1 && git rm --cached --
"$artefact" || true
done
uv run --frozen dvc add data/gold/gold_dataset.csv artifacts/models/rf.joblib
uv run --frozen dvc push
git status --short
uv run --frozen dvc status
```

Versioning modèle metadata MLflow

- 👀 `artifacts/models/model_metadata.json`.
- Démo MLflow réelle (le **registre** exige un backend **SQLite**) :

```
# Terminal 1 : serveur local et registre SQLite (garde ce terminal ouvert).
uv run --frozen mlflow server --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port
5000
```

Dans un **second terminal** :

```
# Windows PowerShell
(Invoke-WebRequest -UseBasicParsing http://127.0.0.1:5000/health).StatusCode
uv run --frozen python scripts/demo_versioning.py --no-dvc --tracking-uri
http://127.0.0.1:5000 --split stratified
uv run --frozen python scripts/demo_versioning.py --no-dvc --tracking-uri
http://127.0.0.1:5000 --split temporal
if (-not (Test-Path .\versioning_strategy.md)) { Copy-Item .\docs\versioning_strategy.md
.\versioning_strategy.md }
```

```
# macOS/Linux
curl -fsS http://127.0.0.1:5000/health >/dev/null
uv run --frozen python scripts/demo_versioning.py --no-dvc --tracking-uri
http://127.0.0.1:5000 --split stratified
uv run --frozen python scripts/demo_versioning.py --no-dvc --tracking-uri
http://127.0.0.1:5000 --split temporal
test -f versioning_strategy.md || cp docs/versioning_strategy.md versioning_strategy.md
```

Le dernier run a réécrit le modèle. Dans le même second terminal, resynchronise donc son pointeur DVC avant de déposer tes preuves (commandes identiques sur les trois systèmes) :

```
uv run --frozen dvc add artifacts/models/rf.joblib
uv run --frozen dvc push
uv run --frozen dvc status
git status --short
```

Tu dois retrouver le pointeur modèle mis à jour, `model_metadata.json`, `metrics.json` et `params.yaml`. Chaque run fournit son `run_id` et enregistre une version dans `indusense-rf` : relève donc **deux `run_id` et deux versions**. Le run temporel est l'artefact final resynchronisé ; aucune promotion de stage n'est exigée ici.

- Complète `versioning_strategy.md` à la racine (sections **Code / Données / Modèle / Secrets** — structure fournie dans la **fiche TD 24**) avec les deux `run_id`, les deux versions et les preuves réellement obtenues.

Correction debug et dépôt des preuves

- Rejoue les quatre contrôles de la Definition of Done ; corrige d'abord l'outil qui échoue, puis dépose `versioning_strategy.md` et la sortie de `git status` sans secret.
- ☑ **Résultat attendu :** tests/lint/format/pre-commit à 0 échec, deux pointeurs DVC visibles, pointeur et remote du modèle resynchronisés, `metrics.json`, `params.yaml`, deux `run_id` + deux versions MLflow, stratégie racine lisible, `uv.lock` inchangé et aucun secret dans les fichiers suivis.

QCM de fin de journée

- **Passe le QCM J1** (`QCM_fin_de_journee_sprint3` / Kahoot `kahoot_J1.xlsx`), **avant** la mini-rétro.

À ne pas oublier FAQ définitions

🕮 **Définition. CI** : serveur qui rejoue lint+tests+build à chaque push. **DVC** : versionne les gros fichiers (pointeur dans Git + contenu sur un remote). **Stage MLflow** : candidate → Staging → Production → Archived. ❓ « DVC remplace Git ? » → non : Git pour le **code**, DVC pour les **gros fichiers** ; ils travaillent ensemble. ❓ « `dvc add` dit *already tracked by SCM* ? » → le fichier est déjà dans git : `git rm --cached -- <fichier>` puis `uv run --frozen dvc add <fichier>`. Et le **registre** MLflow exige `uv run --frozen mlflow server --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port 5000`. ❓ « Pourquoi révoquer un secret au lieu de le supprimer ? » → il reste dans **l'historique Git** ; exploitable tant qu'il n'est pas révoqué côté fournisseur. 🔑 À ne pas oublier de dire : reproduire un résultat = **code + données + modèle**, versionnés **ensemble**.

Transition module 25

☑ **Résultat attendu :** Preuve finale J1 : `uv run --frozen pytest -q` · `uv run --frozen ruff check .` · `uv run --frozen black --check .` · `uv run --frozen pre-commit run --all-files` · `uv run --frozen dvc status` · `git diff --exit-code -- uv.lock`. Livrables : `ci.yml` (+ job build), hooks activés depuis `.pre-commit-config.yaml`, deux pointeurs `.dvc` dont le modèle et le remote resynchronisés après le dernier run, `metrics.json`, `params.yaml`, deux `run_id` et deux versions enregistrées dans MLflow — une par run, le modèle temporel étant l'artefact final —, `versioning_strategy.md` racine et **0 secret** en clair.

Fin de demi journée J1 après midi M24

Enregistre les fichiers dans VS Code. Retourne à la **racine Parcours** avant de sauvegarder. Les chemins ci-dessous correspondent aux fichiers travaillés dans cette demi-journée : retire ceux que tu n'as pas modifiés et ajoute uniquement les nouveaux fichiers que tu as réellement créés et relus. Les dossiers temporaires, `.env`, clés, bases SQLite, captures contenant des données privées et environnements `.venv` restent hors Git. Si `git status` signale une suppression volontaire, inspecte-la avec le formateur avant son ajout.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
$brancheAttendue = Read-Host 'Nom exact de ta branche personnelle choisie en début de demi-
journée'
$branche = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheAttendue) -or $branche -cne
$brancheAttendue -or $branche -eq 'main' -or $branche -like 'jalon/*') { throw 'Branche
courante différente de ta branche personnelle attendue : arrêt avant ajout, commit et push.'
}
git status --short
git diff --stat
git diff
$fichiers = @('.github/workflows/ci.yml', '.pre-commit-config.yaml', '.dvc/config',
'.dvc/.gitignore', 'data/gold/gold_dataset.csv.dvc', 'artifacts/models/rf.joblib.dvc',
'.gitignore', 'artifacts/models/.gitignore', 'data/gold/.gitignore',
'artifacts/models/model_metadata.json', 'metrics.json', 'params.yaml',
'versioning_strategy.md')
# Relis et adapte cette liste avant de poursuivre.
$existants = @($fichiers | Where-Object { Test-Path -LiteralPath $_ })
if ($existants.Count -gt 0) {
    git add -- $existants
    if ($LASTEXITCODE -ne 0) { throw 'Ajout Git échoué : ne pas commiter.' }
}
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Corrige les erreurs signalées avant le commit.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J1 après midi M24 travail et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire identité Git, hook ou conflit.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Impossible de contrôler les changements indexés.' }
git status --short
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Sauvegarde distante non configurée : commits locaux
conservés.' }
$suivi = git for-each-ref --format="%(upstream:short)" "refs/heads/$branche"
if ($suivi -cne "perso/$branche") {
    git push -u perso $branche
} else { git push perso $branche }
if ($LASTEXITCODE -ne 0) { throw 'Push échoué : ta sauvegarde distante reste à faire.' }
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso $branche
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
```

```
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
printf 'Nom exact de ta branche personnelle choisie en début de demi-journée : '
IFS= read -r branche_attendue
branche="$(git branch --show-current)" || exit 1
[ -n "$branche_attendue" ] && [ "$branche" = "$branche_attendue" ] || { printf 'Branche
différente de la branche personnelle attendue.\n'; exit 1; }
case "$branche" in main|jalon/*|'') printf 'Branche personnelle requise avant ajout, commit
et push.\n'; exit 1;; esac
git status --short
git diff --stat
git diff
fichiers=(".github/workflows/ci.yml" ".pre-commit-config.yaml" ".dvc/config"
".dvc/.gitignore" "data/gold/gold_dataset.csv.dvc" "artifacts/models/rf.joblib.dvc"
".gitignore" "artifacts/models/.gitignore" "data/gold/.gitignore"
"artifacts/models/model_metadata.json" "metrics.json" "params.yaml" "versioning_strategy.md")
# Relis et adapte cette liste avant de poursuivre.
for fichier in "${fichiers[@]}"; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then
  git commit -m "Sprint 3 J1 après midi M24 travail et preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then
  printf 'Contrôle du diff impossible.\n'; exit 1
fi
git status --short
git remote get-url perso || exit 1
suivi="$(git for-each-ref --format='%(upstream:short)' "refs/heads/$branche")"
if [ "$suivi" != "perso/$branche" ]; then
  git push -u perso "$branche" || exit 1
else
  git push perso "$branche" || exit 1
fi
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso "$branche"
```

Attendu : un commit seulement si tu as des changements, un push terminé sans erreur et **le même hash** pour `HEAD` et la branche distante listée par `ls-remote`. Vérifie aussi ton dernier commit dans le dépôt personnel GitHub. `Everything up-to-date` est normal si tout était déjà envoyé. `nothing to commit` ne justifie pas un commit vide : vérifie et pousse les commits locaux éventuels. Si des fichiers restent dans `status`, conserve-les et identifie-les ; tu n'es pas encore prêt pour le prochain jalon.

Si le commit échoue à cause d'un hook qui reformate un fichier, relis le nouveau diff et indexe à nouveau le fichier avant un nouveau commit. Si le push répond `rejected` ou `non-fast-forward`, ne

force pas : un autre historique distant doit être réconcilié avec le formateur. Si un conflit est déjà en cours, `git status` indique les fichiers concernés ; travaille avec le formateur avant d'ajouter et de commiter la résolution.

25 API Design REST FastAPI J2 matin US3 2 C7

✓ **Preuve finale visée :** `/docs` lisible · `/health` 200 · `/predict-tabular` 200 (avec clé) · **401 sans clé** · **422** si < 7 relevés · **503** sans modèle · `docs/model_card.md` à trois niveaux, sans preuve inventée.

Début de demi journée J2 matin M25

Terminal du dépôt Parcours. Même si tu y travaillais hier, redonne le chemin complet ; cela évite un `Set-Location -LiteralPath $projet` depuis un dossier déjà ouvert. Arrête d'abord le serveur qui occupe ce terminal avec Ctrl+C si nécessaire. Ne lance pas de Git dans le terminal occupé par un serveur.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
pwd
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
git status --short
git remote -v
git branch --show-current
if (git status --porcelain) { throw 'Travail non commité : fais un checkpoint avant la
récupération.' }
$branche = Read-Host 'Nom exact de ta branche personnelle du Sprint 3'
if ([string]::IsNullOrWhiteSpace($branche) -or $branche -eq 'main' -or $branche -like
'jalon/*') { throw 'Utilise ta branche personnelle.' }
git switch $branche
if ($LASTEXITCODE -ne 0) { throw 'Branche non ouverte : lire le message Git.' }
code .
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Remote perso absent : configure ta destination de
sauvegarde avant la suite.' }
$refPerso = git ls-remote --heads perso $branche
if ($LASTEXITCODE -ne 0) { throw 'Accès distant échoué : ne pas continuer comme si le pull
avait réussi.' }
if ($refPerso) {
    git pull --ff-only perso $branche
    if ($LASTEXITCODE -ne 0) { throw 'Pull refusé : conserve ton travail et appelle le
formateur.' }
} else { Write-Host 'Premier passage : branche encore absente de perso, premier push à la
clôture.' }
git status --short
git branch --show-current
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
pwd
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
git status --short
git remote -v
git branch --show-current
[ -z "$(git status --porcelain)" ] || { printf 'Fais un checkpoint avant la récupération.\n';
exit 1; }
printf 'Nom exact de ta branche personnelle du Sprint 3 : '
IFS= read -r branche
case "$branche" in main|jalon/*|'') printf 'Utilise ta branche personnelle.\n'; exit 1;; esac
git switch "$branche" || exit 1
code .
git remote get-url perso || exit 1
ref_perso="$(git ls-remote --heads perso "$branche")" || exit 1
if [ -n "$ref_perso" ]; then
  git pull --ff-only perso "$branche" || exit 1
else
  printf 'Branche absente de perso : premier push à la clôture.\n'
fi
git status --short
git branch --show-current
```

Attendu : racine Parcours, branche personnelle et remote de sauvegarde corrects. Si `status` n'est pas vide, enregistre tes fichiers, lis le diff et sélectionne ceux à commiter comme dans la clôture ci-dessous. Ne récupère pas un jalon par-dessus du travail non commité. Si le pull échoue pour authentification, vérifie le compte et les droits dans le mécanisme GitHub proposé ; ne colle aucun token dans une commande. Un refus `--ff-only` indique des historiques divergents : fais les contrôles avec le formateur, sans reset ni push forcé.

Au signal du formateur, intègre **jalon/03** sur cette même branche avec les deux commandes suivantes. Le script effectue la fusion nécessaire des commits du cours ; `--ff-only` ci-dessus concernait uniquement la sauvegarde personnelle.

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 03
if ($LASTEXITCODE -ne 0) { throw 'Jalon non intégré : lire le diagnostic, ne pas lancer le
module.' }
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 03
if ($LASTEXITCODE -ne 0) { throw 'Vérification du jalon échouée.' }
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
git branch --show-current
```

```
bash scripts/formation/mettre_a_niveau.sh 03 || exit 1
bash scripts/formation/verifier_jalon.sh 03 || exit 1
head -n 1 FORMATION/JALON_ACTUEL.md
git branch --show-current
```

Attendu : `Jalon integre sur … : jalon/03`, puis `Jalon verifie : jalon/03`, le marqueur correspondant au tableau de progression, et ta branche personnelle. En cas de conflit, le script annule sa fusion ; lis le diagnostic avec le formateur avant un éventuel rattrapage.

Reprise ouverture du poste et préflight

- Ouvre **VS Code**, puis **Fichier > Ouvrir le dossier… > `CISIA_24082026_Parcours`**. Ouvre **Terminal > Nouveau terminal** et vérifie `uv run python --version`, `Test-Path .\uv.lock` et `git status --short` avant de commencer.

Théorie I API comme contrat

- 👀 Slide « Une API, c'est un contrat ».
- 👀 Slide « /health vs /ready » : `/health` = le **process tourne** (liveness, 200 dès le lancement) ; `/ready` = le **modèle est chargé** (readiness, **503** sinon).

Tour du code I API du squelette

- 👀 Précisément, dans `src/indusense/api/` : `schemas.py`, `main.py`, `model_store.py`.
- Ouvre **deux terminaux PowerShell** dans VS Code avec **Terminal > Nouveau terminal**. Le terminal 1 restera occupé par Uvicorn ; les tests se lancent dans le terminal 2. Une variable définie dans l'un n'existe pas dans l'autre.
- **Terminal 1 — serveur :**

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
if (-not (Test-Path -LiteralPath .\.env)) {
    Copy-Item -LiteralPath .\.env.example -Destination .\.env
}
git check-ignore .env      # attendu : .env ; ne jamais afficher son contenu
uv run uvicorn indusense.api.main:app --reload --port 8000
# ouvrir http://127.0.0.1:8000/docs
```

☑ **Résultat attendu :** `/docs` s'ouvre, les schémas sont visibles, `/health` répond 200.

Pause obligatoire en visio

- Laisse Uvicorn tourner dans le terminal 1, coupe le partage d'écran et reviens à l'heure annoncée. À la reprise, utilise le **terminal 2** pour toutes les preuves.

Lire le contrat API et contrôler le code HTTP

Dans le navigateur, ouvre http://127.0.0.1:8000/docs pour Swagger UI et http://127.0.0.1:8000/openapi.json pour le contrat JSON. Ce sont des URL locales sur **ton poste**, accessibles pendant l'exécution du serveur de ce module. Si Swagger reste vide hors ligne, ouvre le JSON pour examiner les chemins, paramètres et schémas.

Dans Swagger, ouvre **POST /predict-tabular**, clique **Try it out**, remplis le paramètre d'en-tête **X-API-Key**, copie le corps du `payload.json` fourni à la racine du dépôt, puis clique **Execute**. Lis **Server response / Code** et **Response body**. La clé est un paramètre de cette opération ; ne cherche pas

un bouton global Authorize qui n'est pas déclaré par ce code. Coupe le partage d'écran pendant sa saisie et masque l'en-tête dans tes captures.

Requête	Code HTTP attendu	Corps à lire
GET /health	200	`{"status":"ok"}`, le processus répond
GET /ready avec modèle chargé	200	`status` vaut `ready`, avec `model_version`
GET /ready sans modèle	503	`detail` indique que le modèle n'est pas chargé
POST /predict-tabular sans clé	401	refus d'authentification
POST /predict-tabular avec clé et corps valides	200	`machine_id`, `proba_panne`, `decision`, `model_version`, `threshold`

200 est un code HTTP ; **ok** ou **ready** est une valeur JSON. La prédiction ne contient pas de champ `status`. Pour le payload livré, `machine_id` doit être `MACH-07`. `proba_panne` est entre 0 et 1 ; `decision` vaut `alerte` si la probabilité atteint `threshold`, sinon `ok`. La valeur numérique et la version dépendent de l'artefact chargé : relève celles que ton poste renvoie.

Le contrat exige au moins **sept relevés** ; le `payload.json` livré en contient **huit**, de 00 h à 07 h le 1er février 2025, avec température 50 à 57 et pression 195 à 198,5. Chaque relevé contient `timestamp`, `temperature` et `pressure_bar`. Utilise le fichier fourni ; les corps d'erreur 422 varient selon ce qui est invalide.

Dans un **terminal de contrôle distinct du terminal serveur**, utilise le bloc adapté ci-dessous. Il redemande le chemin du dépôt et refuse un sous-dossier ou un autre dépôt avant les commandes API.

Windows PowerShell utilise **curl.exe**, car `curl` peut désigner un alias :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
if (-not (Test-Path -LiteralPath .\payload.json)) { throw 'payload.json absent : vérifier
dossier et jalon.' }
curl.exe -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/health
if ($LASTEXITCODE -ne 0) { throw 'Échec de connexion /health.' }
curl.exe -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/ready
if ($LASTEXITCODE -ne 0) { throw 'Échec de connexion /ready.' }
curl.exe -sS -X POST -H 'Content-Type: application/json' --data-binary '@payload.json' -w
"\nHTTP %{http_code}\n" http://127.0.0.1:8000/predict-tabular
if ($LASTEXITCODE -ne 0) { throw 'Échec de connexion pour le POST sans clé.' }

# Saisir la même clé locale que celle de .env, sans la mettre dans la commande.
```

```
$apiKeySecure = Read-Host 'Clé locale INDUSENSE_API_KEY (saisie masquée)' -AsSecureString
$apiKey = [System.Net.NetworkCredential]::new('', $apiKeySecure).Password
if ([string]::IsNullOrWhiteSpace($apiKey) -or $apiKey.Contains("`r") -or
$apiKey.Contains("`n")) { throw 'Clé vide ou retour à la ligne : reprendre la saisie.' }
$escapedApiKey = $apiKey.Replace('\', '\\').Replace('"', '\"')
@"
header = "X-API-Key: $escapedApiKey"
"@ | curl.exe -sS --config - -X POST -H 'Content-Type: application/json' --data-binary
'@payload.json' -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/predict-tabular
if ($LASTEXITCODE -ne 0) { throw 'Échec de connexion pour le POST avec clé.' }
Remove-Variable apiKey, escapedApiKey, apiKeySecure
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
test -f payload.json || { printf '%s\n' 'payload.json absent : vérifier dossier et jalon.';
exit 1; }
curl -sS -w '\nHTTP %{http_code}\n' http://127.0.0.1:8000/health || { printf '%s\n'
'Connexion locale échouée'; exit 1; }
curl -sS -w '\nHTTP %{http_code}\n' http://127.0.0.1:8000/ready || { printf '%s\n' 'Connexion
locale échouée'; exit 1; }
curl -sS -X POST -H 'Content-Type: application/json' --data-binary '@payload.json' -w '\nHTTP
%{http_code}\n' http://127.0.0.1:8000/predict-tabular || { printf '%s\n' 'Connexion locale
échouée'; exit 1; }
read -r -s -p 'Clé locale INDUSENSE_API_KEY (saisie masquée) : ' api_key
printf '\n'
case "$api_key" in ''|*$'\r'*|*$'\n'*) printf '%s\n' 'Clé vide ou retour à la ligne :
reprendre la saisie.'; exit 1;; esac
escaped_api_key=${api_key//\\/\\\\}
escaped_api_key=${escaped_api_key//\"/\\\"}
printf 'header = "X-API-Key: %s"\n' "$escaped_api_key" | curl -sS --config - -X POST -H
'Content-Type: application/json' --data-binary '@payload.json' -w '\nHTTP %{http_code}\n'
http://127.0.0.1:8000/predict-tabular || { printf '%s\n' 'Connexion locale échouée'; exit 1;
}
unset api_key escaped_api_key
```

Chaque requête affiche le **corps**, puis `HTTP 200`, `HTTP 401` ou le code réellement obtenu. Un code processus curl nul ne prouve pas HTTP 200 : lis le suffixe HTTP. `HTTP 000` avec une erreur de connexion signifie qu'aucune réponse API n'a été obtenue. La clé saisie doit être celle configurée localement dans `.env` ; elle passe par l'entrée standard de curl, pas par une valeur collée dans la commande.

TP 2 TestClient les codes d erreur

- Dans l'Explorateur Windows, copie le dossier

`05_DONNEES_ET_EXERCICES\07_M25_API_PROOFS\tp_api_m25_v1_20260823` du pack apprenant **à côté** de `CISIA_24082026_Parcours`. Puis, dans le **terminal 2**, à la racine du dépôt :

```
$proofRoot = (Resolve-Path -LiteralPath '..\tp_api_m25_v1_20260823').Path
```

```
if (-not (Test-Path -LiteralPath (Join-Path $proofRoot 'APPLIQUER_PREUVES_M25.ps1'))) {
    throw 'Ressource M25 absente, incomplète ou placée au mauvais endroit.'
}
& (Join-Path $proofRoot 'APPLIQUER_PREUVES_M25.ps1') -ProjectPath .
if (-not $?) { throw 'Application de la surcouche M25 impossible.' }
uv run pytest -q tests/test_api.py tests/test_readiness_probe.py
tests/test_model_card_gate.py
```

☑ **Résultat attendu :** `M25_OVERLAY=READY`, puis **12 passed, 0 échec** ; `/health` 200 · **sans clé** → **401** · historique court → **422** · sans modèle → **503** · requête correcte → **200**. Si un fichier différent existait, son chemin de sauvegarde temporaire est affiché ; si tu ouvres un nouveau terminal, redéfinis `$proofRoot`.

Au jalon 03 du module 25, la clé est contrôlée, puis le corps est validé et le modèle est utilisé. Clé absente : HTTP 401 ; corps invalide ou moins de sept relevés : HTTP 422 ; modèle absent : HTTP 503 ; requête correcte : HTTP 200. Au jalon 04 du module 26, le rate limit intervient après la clé et peut produire HTTP 429.

TP 3 request id normalisation au bord

- Activité : ajouter/observer le **middleware request-id** (déjà présent) ; vérifier que `X-Request-ID` revient dans la réponse.
- L'endpoint image est une extension discutée à partir de M26. Son placeholder ne réalise pas de prédiction image.

Model Card à trois niveaux C4 C5

- Crée `docs/model_card.md` pour trois lecteurs :
  1. **Métier** — finalité, utilisateurs, horizon et seuil réellement mesurés, coût des erreurs, limites et décision humaine.
  2. **Technique / maintenance** — données/version, features, métriques, dépendances, coût, version du modèle, réévaluation et `run_id` MLflow réel ; sans run, écrire exactement `à produire`.
  3. **Conformité AI Act** — usage prévu/interdit, supervision, risques et traçabilité ; écrire **« à confirmer avec le référent conformité »** tant que la qualification n'est pas établie.
- Le benchmark Marine est un encadré **distinct**, jamais ton résultat : XGBoost 24 h, seuil ≈ 0,41, PR-AUC ≈ 0,62, prévalence 16,6 % ; frugal 202,6 s / 0,158 gCO2e contre lourd 612,8 s / 0,352 gCO2e.
- Plan B hors ligne : produire la carte, marquer `run_id : à produire` et noter l'action de rattachement future ; ne fabrique jamais un UUID ni une classe AI Act.

Preuve finale

- Tu produis :

```
$proofRoot = (Resolve-Path -LiteralPath '..\tp_api_m25_v1_20260823').Path
uv run pytest -q tests/test_api.py tests/test_readiness_probe.py
tests/test_model_card_gate.py
# attendu : 12 passed, 0 échec
$mc = '.\docs\model_card.md'
if (-not (Test-Path -LiteralPath $mc)) { throw 'model_card.md absent' }
$rubriques = @(
  '^## Métier',
  '^## Technique / maintenance',
  '^## Conformité AI Act',
  'run_id'
)
foreach ($rubrique in $rubriques) {
```

```
  if (-not (Select-String -LiteralPath $mc -Pattern $rubrique -Quiet)) {
    throw "Rubrique manquante : $rubrique"
  }
}
uv run python .\scripts\validate_model_card.py .\docs\model_card.md --project-root .
if ($LASTEXITCODE -ne 0) { throw 'Model Card non recevable.' }
$lockState = git status --short -- uv.lock
if ($lockState) { throw "uv.lock a changé pendant M25 : $lockState" }
# /docs ouvert + capture : /health 200, 401 sans clé, 422 (<7 relevés)
# le 503 « sans modèle » est prouvé par test_readiness_probe.py
```

☑ **Résultat attendu :** `/docs` lisible · `/health` 200 · 401 sans clé · 422 · 503 sans modèle · **12 tests verts** · validateur Model Card sans erreur · `uv.lock` non modifié · trois niveaux présents · `run_id` réel ou `à produire` · statut AI Act prudent. **Partage tes captures et le chemin de la carte**.

À ne pas oublier FAQ définitions

📖 **Définition. REST** : ressources + verbes HTTP. **Liveness** (`/health`) : le process tourne. **Readiness** (`/ready`) : le modèle est chargé. **Pydantic** : valide et documente le contrat I/O. ❓ « Charger le modèle à chaque requête ? » → non, la latence exploserait : **une fois au démarrage** (`lifespan`). ❓ « 401 ou 422 quand la clé manque ? » → **401** (authentification) ; 422 = corps invalide. 🔑 À ne pas oublier de dire : **`/docs` décrit les schémas et paramètres déclarés dans le code.** Les requêtes et les tests valident le comportement et les erreurs.

- 🔗 « Le **contrat I/O** du module 22 (Sprint 2) devient les **schémas Pydantic** ici. »

Transition module 26

Fin de demi journée J2 matin M25

Enregistre les fichiers dans VS Code. Retourne à la **racine Parcours** avant de sauvegarder. Les chemins ci-dessous correspondent aux fichiers travaillés dans cette demi-journée : retire ceux que tu n'as pas modifiés et ajoute uniquement les nouveaux fichiers que tu as réellement créés et relus. Les dossiers temporaires, `.env`, clés, bases SQLite, captures contenant des données privées et environnements `.venv` restent hors Git. Si `git status` signale une suppression volontaire, inspecte-la avec le formateur avant son ajout.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
$brancheAttendue = Read-Host 'Nom exact de ta branche personnelle choisie en début de demi-
journée'
$branche = git branch --show-current
```

```
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheAttendue) -or $branche -cne
$brancheAttendue -or $branche -eq 'main' -or $branche -like 'jalon/*') { throw 'Branche
courante différente de ta branche personnelle attendue : arrêt avant ajout, commit et push.'
}
git status --short
git diff --stat
git diff
$fichiers = @('src/indusense/api/main.py', 'src/indusense/api/schemas.py',
'tests/test_api.py', 'tests/test_readiness_probe.py', 'tests/test_model_card_gate.py',
'scripts/validate_model_card.py', 'docs/model_card.md')
# Relis et adapte cette liste avant de poursuivre.
$existants = @($fichiers | Where-Object { Test-Path -LiteralPath $_ })
if ($existants.Count -gt 0) {
    git add -- $existants
    if ($LASTEXITCODE -ne 0) { throw 'Ajout Git échoué : ne pas commiter.' }
}
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Corrige les erreurs signalées avant le commit.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J2 matin M25 travail et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire identité Git, hook ou conflit.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Impossible de contrôler les changements indexés.' }
git status --short
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Sauvegarde distante non configurée : commits locaux
conservés.' }
$suivi = git for-each-ref --format="%(upstream:short)" "refs/heads/$branche"
if ($suivi -cne "perso/$branche") {
    git push -u perso $branche
} else { git push perso $branche }
if ($LASTEXITCODE -ne 0) { throw 'Push échoué : ta sauvegarde distante reste à faire.' }
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso $branche
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
printf 'Nom exact de ta branche personnelle choisie en début de demi-journée : '
IFS= read -r branche_attendue
branche="$(git branch --show-current)" || exit 1
[ -n "$branche_attendue" ] && [ "$branche" = "$branche_attendue" ] || { printf 'Branche
différente de la branche personnelle attendue.\n'; exit 1; }
case "$branche" in main|jalon/*|'') printf 'Branche personnelle requise avant ajout, commit
et push.\n'; exit 1;; esac
git status --short
git diff --stat
```

```
git diff
fichiers=("src/indusense/api/main.py" "src/indusense/api/schemas.py" "tests/test_api.py"
"tests/test_readiness_probe.py" "tests/test_model_card_gate.py"
"scripts/validate_model_card.py" "docs/model_card.md")
# Relis et adapte cette liste avant de poursuivre.
for fichier in "${fichiers[@]}"; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then
  git commit -m "Sprint 3 J2 matin M25 travail et preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then
  printf 'Contrôle du diff impossible.\n'; exit 1
fi
git status --short
git remote get-url perso || exit 1
suivi="$(git for-each-ref --format='%(upstream:short)' "refs/heads/$branche")"
if [ "$suivi" != "perso/$branche" ]; then
  git push -u perso "$branche" || exit 1
else
  git push perso "$branche" || exit 1
fi
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso "$branche"
```

Attendu : un commit seulement si tu as des changements, un push terminé sans erreur et **le même hash** pour `HEAD` et la branche distante listée par `ls-remote`. Vérifie aussi ton dernier commit dans le dépôt personnel GitHub. `Everything up-to-date` est normal si tout était déjà envoyé. `nothing to commit` ne justifie pas un commit vide : vérifie et pousse les commits locaux éventuels. Si des fichiers restent dans `status`, conserve-les et identifie-les ; tu n'es pas encore prêt pour le prochain jalon.

Si le commit échoue à cause d'un hook qui reformate un fichier, relis le nouveau diff et indexe à nouveau le fichier avant un nouveau commit. Si le push répond `rejected` ou `non-fast-forward`, ne force pas : un autre historique distant doit être réconcilié avec le formateur. Si un conflit est déjà en cours, `git status` indique les fichiers concernés ; travaille avec le formateur avant d'ajouter et de commiter la résolution.

26 Sécurité menaces sur l IA J2 après midi Sécurité C2

✓ **Preuve finale visée :** `threat_model.md` **+** `security_controls.md` · **5 contrôles priorisés : 4 implémentés et prouvés** (`401`, `422`, `429`, `413`) **+ audit logging Planifié v0** · suites API/sécurité à **0 échec**.

Début de demi journée J2 après midi M26

Terminal du dépôt Parcours. Même si tu y travaillais hier, redonne le chemin complet ; cela évite un `Set-Location -LiteralPath $projet` depuis un dossier déjà ouvert. Arrête d'abord le serveur qui occupe ce terminal avec Ctrl+C si nécessaire. Ne lance pas de Git dans le terminal occupé par un serveur.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
pwd
$racineGit = git rev-parse --show-toplevel
```

```
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
git status --short
git remote -v
git branch --show-current
if (git status --porcelain) { throw 'Travail non commité : fais un checkpoint avant la
récupération.' }
$branche = Read-Host 'Nom exact de ta branche personnelle du Sprint 3'
if ([string]::IsNullOrWhiteSpace($branche) -or $branche -eq 'main' -or $branche -like
'jalon/*') { throw 'Utilise ta branche personnelle.' }
git switch $branche
if ($LASTEXITCODE -ne 0) { throw 'Branche non ouverte : lire le message Git.' }
code .
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Remote perso absent : configure ta destination de
sauvegarde avant la suite.' }
$refPerso = git ls-remote --heads perso $branche
if ($LASTEXITCODE -ne 0) { throw 'Accès distant échoué : ne pas continuer comme si le pull
avait réussi.' }
if ($refPerso) {
    git pull --ff-only perso $branche
    if ($LASTEXITCODE -ne 0) { throw 'Pull refusé : conserve ton travail et appelle le
formateur.' }
} else { Write-Host 'Premier passage : branche encore absente de perso, premier push à la
clôture.' }
git status --short
git branch --show-current
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
pwd
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
git status --short
git remote -v
git branch --show-current
[ -z "$(git status --porcelain)" ] || { printf 'Fais un checkpoint avant la récupération.\n';
exit 1; }
printf 'Nom exact de ta branche personnelle du Sprint 3 : '
IFS= read -r branche
```

```
case "$branche" in main|jalon/*|'') printf 'Utilise ta branche personnelle.\n'; exit 1;; esac
git switch "$branche" || exit 1
code .
git remote get-url perso || exit 1
ref_perso="$(git ls-remote --heads perso "$branche")" || exit 1
if [ -n "$ref_perso" ]; then
  git pull --ff-only perso "$branche" || exit 1
else
  printf 'Branche absente de perso : premier push à la clôture.\n'
fi
git status --short
git branch --show-current
```

Attendu : racine Parcours, branche personnelle et remote de sauvegarde corrects. Si `status` n'est pas vide, enregistre tes fichiers, lis le diff et sélectionne ceux à commiter comme dans la clôture ci-dessous. Ne récupère pas un jalon par-dessus du travail non commité. Si le pull échoue pour authentification, vérifie le compte et les droits dans le mécanisme GitHub proposé ; ne colle aucun token dans une commande. Un refus `--ff-only` indique des historiques divergents : fais les contrôles avec le formateur, sans reset ni push forcé.

Au signal du formateur, intègre **jalon/04** sur cette même branche avec les deux commandes suivantes. Le script effectue la fusion nécessaire des commits du cours ; `--ff-only` ci-dessus concernait uniquement la sauvegarde personnelle.

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 04
if ($LASTEXITCODE -ne 0) { throw 'Jalon non intégré : lire le diagnostic, ne pas lancer le
module.' }
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 04
if ($LASTEXITCODE -ne 0) { throw 'Vérification du jalon échouée.' }
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
git branch --show-current
```

```
bash scripts/formation/mettre_a_niveau.sh 04 || exit 1
bash scripts/formation/verifier_jalon.sh 04 || exit 1
head -n 1 FORMATION/JALON_ACTUEL.md
git branch --show-current
```

Attendu : `Jalon integre sur … : jalon/04`, puis `Jalon verifie : jalon/04`, le marqueur correspondant au tableau de progression, et ta branche personnelle. En cas de conflit, le script annule sa fusion ; lis le diagnostic avec le formateur avant un éventuel rattrapage.

Ouverture

- Classe les cinq contrôles : auth, validation, rate limit, taille de payload, audit logging.

☑ **À trouver :** les quatre premiers sont implémentés et prouvables ; l'audit logging est seulement **Planifié v0**.

Théorie penser comme un attaquant

- 👀 Slides 6-9 : STRIDE, menaces propres au ML, puis matrice **4 prouvés + 1 planifié**.
- Auth clé → **401** · validation Pydantic v2 → **422** · rate limit 60/min/IP → **429** · payload > 64 Ko → **413**.
- Audit logging → **Planifié v0** : preuve future = événement structuré sans clé, payload ni PII + test dédié.

Tour du code la sécurité du squelette

- 👀 `src/indusense/api/security.py`, `src/indusense/api/main.py`, les schémas et les deux suites de tests.

```
Select-String -Path .\src\indusense\api\security.py -Pattern 'MAX_BODY_BYTES|def
rate_limit|def rate_limit_dependency'
```

```
Select-String -Path .\src\indusense\api\main.py -Pattern
'require_api_key|rate_limit_dependency|limit_body_size'
```

☑ **À trouver :** politique 60/60 fermée par `rate_limit_dependency`, limite 64 Ko, 401/422/429/413 ; aucun événement ni test d'audit dédié. Un `Content-Length` illisible est refusé par **400**.

Pause

- À la reprise, donne un actif et une menace associée avant de passer au TP.

TP 1 Attack tree contrôles testables

- À la racine de `CISIA_24082026_Parcours`, synchronise exactement l'environnement puis vérifie Python :

```
uv sync --frozen --extra dev
uv run python --version
```

- Rédige `threat_model.md` (STRIDE sur `/predict-tabular` + pipeline) et `security_controls.md` avec cinq lignes : statut, preuve actuelle, risque résiduel, action suivante.
- Dans le registre, écris exactement **4 × Implémenté** et **1 × Planifié v0** (audit logging).

☑ **Résultat attendu :** Python **3.13.x** ; fichiers créés sans écraser un travail existant ; aucune clé, aucun payload et aucune PII dans la preuve future d'audit.

TP 2 Durcir sans casser

```
uv run pytest tests/test_api.py tests/test_security.py -q
```

☑ **Résultat attendu :** 0 échec ; `/health` reste **200 sans clé** ; `/predict-tabular` prouve **401**, **422**, **429** et **413**. Le test direct accepte 60 appels puis bloque le 61e ; une rafale API de 70 appels doit seulement contenir au moins un 429.

TP 3 preuve complémentaire

- Au choix : test de l'OpenAPI prouvant que `limit` et `window` ne sont pas exposés, ou test de non-divulgation dans les logs.

☑ **Résultat attendu :** preuve verte et intitulé exact. Le test de non-divulgation **ne prouve pas** que l'audit logging existe : il reste Planifié v0.

Preuve finale

```
uv run pytest tests/test_api.py tests/test_security.py -q
$rows = @(Get-Content -LiteralPath .\security_controls.md |
  Where-Object {
    $_ -match '^\| (Auth|Validation|Rate limit|Taille payload|Audit logging) \|'
  })
$implemented = @($rows | Where-Object {
  $_ -match '\| Implémenté \|'
}).Count
$planned = @($rows | Where-Object {
  $_ -match '\| Planifié v0 \|'
}).Count
if ($rows.Count -ne 5 -or $implemented -ne 4 -or $planned -ne 1) {
  throw 'Registre attendu : 4 Implémenté + 1 Planifié v0.'
}
```

☑ **Résultat attendu :** tests à 0 échec ; codes `401/422/429/413` ; cinq lignes de registre correctement classées. **Preuve C2** : risques et contrôles ; **preuve C8** : tests et risques résiduels.

QCM de fin de JOURNÉE 2

- Passe **une seule** banque : QCM papier J2 ou Kahoot. Question de contrôle : « lequel reste Planifié v0 ? » → **audit logging**.

À ne pas oublier FAQ définitions

📖 **Définition. STRIDE** : 6 familles de menaces (usurpation, altération, déni d'action, fuite, déni de service, élévation de privilège). **Adversarial** : entrée truquée pour tromper le modèle. **Moindre privilège** : n'accorder que le nécessaire. ❓ « Un contrôle documenté suffit ? » → non : **priorisé ≠ implémenté**. Les preuves actuelles sont 401/422/429/413. ❓ « Pourquoi laisser `/health` ouvert ? » → c'est la sonde de **liveness** de l'orchestrateur ; la protéger casserait le monitoring. ❓ « Le request-id est-il un audit log ? » → non : il corrèle les requêtes, mais ne constitue ni un événement d'audit structuré ni sa preuve. 🛠 **Plan B hors ligne / sans GPU / sans Docker :** tout M26 utilise le code et les tests FastAPI locaux. Si Python ne démarre pas, inspecte code/tests, marque « preuve à rejouer » et ne promeus aucun contrôle. 🔑 À ne pas oublier : **ne jamais logguer** clé, payload ou PII (log bavard = fuite).

Transition module 27

- Quatre défenses applicatives sont prouvées, la traçabilité reste planifiée ; le module 27 durcit l'image Docker (multi-stage, non-root, secrets hors image).

Fin de demi journée J2 après midi M26

Enregistre les fichiers dans VS Code. Retourne à la **racine Parcours** avant de sauvegarder. Les chemins ci-dessous correspondent aux fichiers travaillés dans cette demi-journée : retire ceux que tu n'as pas modifiés et ajoute uniquement les nouveaux fichiers que tu as réellement créés et relus. Les dossiers temporaires, `.env`, clés, bases SQLite, captures contenant des données privées et environnements `.venv` restent hors Git. Si `git status` signale une suppression volontaire, inspecte-la avec le formateur avant son ajout.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
$brancheAttendue = Read-Host 'Nom exact de ta branche personnelle choisie en début de demi-
journée'
$branche = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheAttendue) -or $branche -cne
$brancheAttendue -or $branche -eq 'main' -or $branche -like 'jalon/*') { throw 'Branche
courante différente de ta branche personnelle attendue : arrêt avant ajout, commit et push.'
}
git status --short
git diff --stat
git diff
```

```
$fichiers = @('src/indusense/api/main.py', 'src/indusense/api/security.py',
'tests/test_api.py', 'tests/test_security.py', 'threat_model.md', 'security_controls.md')
# Relis et adapte cette liste avant de poursuivre.
$existants = @($fichiers | Where-Object { Test-Path -LiteralPath $_ })
if ($existants.Count -gt 0) {
    git add -- $existants
    if ($LASTEXITCODE -ne 0) { throw 'Ajout Git échoué : ne pas commiter.' }
}
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Corrige les erreurs signalées avant le commit.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J2 après midi M26 travail et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire identité Git, hook ou conflit.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Impossible de contrôler les changements indexés.' }
git status --short
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Sauvegarde distante non configurée : commits locaux
conservés.' }
$suivi = git for-each-ref --format="%(upstream:short)" "refs/heads/$branche"
if ($suivi -cne "perso/$branche") {
    git push -u perso $branche
} else { git push perso $branche }
if ($LASTEXITCODE -ne 0) { throw 'Push échoué : ta sauvegarde distante reste à faire.' }
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso $branche
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
printf 'Nom exact de ta branche personnelle choisie en début de demi-journée : '
IFS= read -r branche_attendue
branche="$(git branch --show-current)" || exit 1
[ -n "$branche_attendue" ] && [ "$branche" = "$branche_attendue" ] || { printf 'Branche
différente de la branche personnelle attendue.\n'; exit 1; }
case "$branche" in main|jalon/*|'') printf 'Branche personnelle requise avant ajout, commit
et push.\n'; exit 1;; esac
git status --short
git diff --stat
git diff
fichiers=("src/indusense/api/main.py" "src/indusense/api/security.py" "tests/test_api.py"
"tests/test_security.py" "threat_model.md" "security_controls.md")
# Relis et adapte cette liste avant de poursuivre.
for fichier in "${fichiers[@]}"; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
```

```
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then
  git commit -m "Sprint 3 J2 après midi M26 travail et preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then
  printf 'Contrôle du diff impossible.\n'; exit 1
fi
git status --short
git remote get-url perso || exit 1
suivi="$(git for-each-ref --format='%(upstream:short)' "refs/heads/$branche")"
if [ "$suivi" != "perso/$branche" ]; then
  git push -u perso "$branche" || exit 1
else
  git push perso "$branche" || exit 1
fi
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso "$branche"
```

Attendu : un commit seulement si tu as des changements, un push terminé sans erreur et **le même hash** pour `HEAD` et la branche distante listée par `ls-remote`. Vérifie aussi ton dernier commit dans le dépôt personnel GitHub. `Everything up-to-date` est normal si tout était déjà envoyé. `nothing to commit` ne justifie pas un commit vide : vérifie et pousse les commits locaux éventuels. Si des fichiers restent dans `status`, conserve-les et identifie-les ; tu n'es pas encore prêt pour le prochain jalon.

Si le commit échoue à cause d'un hook qui reformate un fichier, relis le nouveau diff et indexe à nouveau le fichier avant un nouveau commit. Si le push répond `rejected` ou `non-fast-forward`, ne force pas : un autre historique distant doit être réconcilié avec le formateur. Si un conflit est déjà en cours, `git status` indique les fichiers concernés ; travaille avec le formateur avant d'ajouter et de commiter la résolution.

27 Conteneurisation Dockerfile J3 matin US3 3 C6

✓ **Preuve finale visée :** `docker build` OK · `docker run` → `/health` 200 → `/ready` 200 → `/predict-tabular` 200 · `whoami` = **appuser**.

Début de demi journée J3 matin M27

Terminal du dépôt Parcours. Même si tu y travaillais hier, redonne le chemin complet ; cela évite un `Set-Location -LiteralPath $projet` depuis un dossier déjà ouvert. Arrête d'abord le serveur qui occupe ce terminal avec Ctrl+C si nécessaire. Ne lance pas de Git dans le terminal occupé par un serveur.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
pwd
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
```

```
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
git status --short
git remote -v
git branch --show-current
if (git status --porcelain) { throw 'Travail non commité : fais un checkpoint avant la
récupération.' }
$branche = Read-Host 'Nom exact de ta branche personnelle du Sprint 3'
if ([string]::IsNullOrWhiteSpace($branche) -or $branche -eq 'main' -or $branche -like
'jalon/*') { throw 'Utilise ta branche personnelle.' }
git switch $branche
if ($LASTEXITCODE -ne 0) { throw 'Branche non ouverte : lire le message Git.' }
code .
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Remote perso absent : configure ta destination de
sauvegarde avant la suite.' }
$refPerso = git ls-remote --heads perso $branche
if ($LASTEXITCODE -ne 0) { throw 'Accès distant échoué : ne pas continuer comme si le pull
avait réussi.' }
if ($refPerso) {
    git pull --ff-only perso $branche
    if ($LASTEXITCODE -ne 0) { throw 'Pull refusé : conserve ton travail et appelle le
formateur.' }
} else { Write-Host 'Premier passage : branche encore absente de perso, premier push à la
clôture.' }
git status --short
git branch --show-current
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
pwd
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
git status --short
git remote -v
git branch --show-current
[ -z "$(git status --porcelain)" ] || { printf 'Fais un checkpoint avant la récupération.\n';
exit 1; }
printf 'Nom exact de ta branche personnelle du Sprint 3 : '
IFS= read -r branche
case "$branche" in main|jalon/*|'') printf 'Utilise ta branche personnelle.\n'; exit 1;; esac
git switch "$branche" || exit 1
code .
git remote get-url perso || exit 1
ref_perso="$(git ls-remote --heads perso "$branche")" || exit 1
if [ -n "$ref_perso" ]; then
  git pull --ff-only perso "$branche" || exit 1
else
  printf 'Branche absente de perso : premier push à la clôture.\n'
fi
git status --short
```

```
git branch --show-current
```

Attendu : racine Parcours, branche personnelle et remote de sauvegarde corrects. Si `status` n'est pas vide, enregistre tes fichiers, lis le diff et sélectionne ceux à commiter comme dans la clôture ci-dessous. Ne récupère pas un jalon par-dessus du travail non commité. Si le pull échoue pour authentification, vérifie le compte et les droits dans le mécanisme GitHub proposé ; ne colle aucun token dans une commande. Un refus `--ff-only` indique des historiques divergents : fais les contrôles avec le formateur, sans reset ni push forcé.

Au signal du formateur, intègre **jalon/05** sur cette même branche avec les deux commandes suivantes. Le script effectue la fusion nécessaire des commits du cours ; `--ff-only` ci-dessus concernait uniquement la sauvegarde personnelle.

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 05
if ($LASTEXITCODE -ne 0) { throw 'Jalon non intégré : lire le diagnostic, ne pas lancer le
module.' }
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 05
if ($LASTEXITCODE -ne 0) { throw 'Vérification du jalon échouée.' }
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
git branch --show-current
```

```
bash scripts/formation/mettre_a_niveau.sh 05 || exit 1
bash scripts/formation/verifier_jalon.sh 05 || exit 1
head -n 1 FORMATION/JALON_ACTUEL.md
git branch --show-current
```

Attendu : `Jalon integre sur … : jalon/05`, puis `Jalon verifie : jalon/05`, le marqueur correspondant au tableau de progression, et ta branche personnelle. En cas de conflit, le script annule sa fusion ; lis le diagnostic avec le formateur avant un éventuel rattrapage.

Ouverture préflight Docker

- Préflight obligatoire : `docker run --rm hello-world` chez chacun (Windows/WSL2).

Théorie image couches multi stage non root

- 👀 Slide « Une image = des couches » puis « Multi-stage : builder gros, livrer mince ».

Tour du code Dockerfile réel

- 👀 `Dockerfile` du squelette : 2 stages (`build` avec uv `--frozen --no-dev --no-editable`, `runtime` `python:3.13-slim`), `useradd appuser` (uid 10001), `COPY artifacts/models` (Variante A), `HEALTHCHECK` sur `/health`, `CMD uvicorn … --host 0.0.0.0 --port 8000`.
- 👀 `.dockerignore` (Variante A : exclut tout le lourd mais **garde** `!artifacts/models`) et le `Makefile` (cibles `sync/test/lint/serve`).

Pause obligatoire en visio

- Coupe le partage et reviens à l'heure annoncée. Docker Desktop reste ouvert, mais aucun build n'est lancé pendant la pause.

TP construire puis contrôler le conteneur

Reste à la racine Parcours. Si Uvicorn M25 utilise encore le port 8000, arrête **ce serveur du cours** avec Ctrl+C dans son terminal. Observe les conteneurs qui publient 8000 ; si un autre service utilise le port, identifie-le avec le formateur au lieu de le supprimer.

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
```

```
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
docker ps --filter publish=8000 --format "table {{.Names}}\t{{.Ports}}"
if (-not (Test-Path -LiteralPath .\.env)) { Copy-Item -LiteralPath .\.env.example -
Destination .\.env }
git check-ignore .env
docker build -t indusense-api:m27 .
if ($LASTEXITCODE -ne 0) { throw 'Build échoué : ne pas lancer le conteneur.' }
docker run --rm -d --name indusense-m27 -p 8000:8000 --env-file .env indusense-api:m27
if ($LASTEXITCODE -ne 0) { throw 'Conteneur non démarré : lire le diagnostic et vérifier le
port.' }
docker exec indusense-m27 whoami
```

Les commandes Docker sont identiques dans Bash. Pour créer `.env` s'il manque, utilise `test -f .env || cp .env.example .env`. N'affiche pas ce fichier ; seule la copie `.env.example` est versionnée.

Attendu de `whoami` : **appuser**. Attends le démarrage, puis réalise maintenant les contrôles suivants **avant d'arrêter le conteneur**.

Lire le contrat API et contrôler le code HTTP

Dans le navigateur, ouvre http://127.0.0.1:8000/docs pour Swagger UI et http://127.0.0.1:8000/openapi.json pour le contrat JSON. Ce sont des URL locales sur **ton poste**, accessibles pendant l'exécution du serveur de ce module. Si Swagger reste vide hors ligne, ouvre le JSON pour examiner les chemins, paramètres et schémas.

Dans Swagger, ouvre **POST /predict-tabular**, clique **Try it out**, remplis le paramètre d'en-tête **X-API-Key**, copie le corps du `payload.json` fourni à la racine du dépôt, puis clique **Execute**. Lis **Server response / Code** et **Response body**. La clé est un paramètre de cette opération ; ne cherche pas un bouton global Authorize qui n'est pas déclaré par ce code. Coupe le partage d'écran pendant sa saisie et masque l'en-tête dans tes captures.

Requête	Code HTTP attendu	Corps à lire
GET /health	200	`{"status":"ok"}`, le processus répond
GET /ready avec modèle chargé	200	`status` vaut `ready`, avec `model_version`
GET /ready sans modèle	503	`detail` indique que le modèle n'est pas chargé
POST /predict-tabular sans clé	401	refus d'authentification
POST /predict-tabular avec clé et corps valides	200	`machine_id`, `proba_panne`, `decision`, `model_version`, `threshold`

200 est un code HTTP ; **ok** ou **ready** est une valeur JSON. La prédiction ne contient pas de champ `status`. Pour le payload livré, `machine_id` doit être `MACH-07`. `proba_panne` est entre 0 et 1 ; `decision` vaut `alerte` si la probabilité atteint `threshold`, sinon `ok`. La valeur numérique et la version dépendent de l'artefact chargé : relève celles que ton poste renvoie.

Le contrat exige au moins **sept relevés** ; le `payload.json` livré en contient **huit**, de 00 h à 07 h le 1er février 2025, avec température 50 à 57 et pression 195 à 198,5. Chaque relevé contient `timestamp`, `temperature` et `pressure_bar`. Utilise le fichier fourni ; les corps d'erreur 422 varient selon ce qui est invalide.

Dans un **terminal de contrôle distinct du terminal serveur**, utilise le bloc adapté ci-dessous. Il redemande le chemin du dépôt et refuse un sous-dossier ou un autre dépôt avant les commandes API.

Windows PowerShell utilise **curl.exe**, car `curl` peut désigner un alias :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
if (-not (Test-Path -LiteralPath .\payload.json)) { throw 'payload.json absent : vérifier
dossier et jalon.' }
curl.exe -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/health
if ($LASTEXITCODE -ne 0) { throw 'Échec de connexion /health.' }
curl.exe -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/ready
if ($LASTEXITCODE -ne 0) { throw 'Échec de connexion /ready.' }
curl.exe -sS -X POST -H 'Content-Type: application/json' --data-binary '@payload.json' -w
"\nHTTP %{http_code}\n" http://127.0.0.1:8000/predict-tabular
if ($LASTEXITCODE -ne 0) { throw 'Échec de connexion pour le POST sans clé.' }

# Saisir la même clé locale que celle de .env, sans la mettre dans la commande.
$apiKeySecure = Read-Host 'Clé locale INDUSENSE_API_KEY (saisie masquée)' -AsSecureString
$apiKey = [System.Net.NetworkCredential]::new('', $apiKeySecure).Password
if ([string]::IsNullOrWhiteSpace($apiKey) -or $apiKey.Contains("`r") -or
$apiKey.Contains("`n")) { throw 'Clé vide ou retour à la ligne : reprendre la saisie.' }
$escapedApiKey = $apiKey.Replace('\', '\\').Replace('"', '\"')
@"
header = "X-API-Key: $escapedApiKey"
"@ | curl.exe -sS --config - -X POST -H 'Content-Type: application/json' --data-binary
'@payload.json' -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/predict-tabular
if ($LASTEXITCODE -ne 0) { throw 'Échec de connexion pour le POST avec clé.' }
Remove-Variable apiKey, escapedApiKey, apiKeySecure
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in
```

```
https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
test -f payload.json || { printf '%s\n' 'payload.json absent : vérifier dossier et jalon.';
exit 1; }
curl -sS -w '\nHTTP %{http_code}\n' http://127.0.0.1:8000/health || { printf '%s\n'
'Connexion locale échouée'; exit 1; }
curl -sS -w '\nHTTP %{http_code}\n' http://127.0.0.1:8000/ready || { printf '%s\n' 'Connexion
locale échouée'; exit 1; }
curl -sS -X POST -H 'Content-Type: application/json' --data-binary '@payload.json' -w '\nHTTP
%{http_code}\n' http://127.0.0.1:8000/predict-tabular || { printf '%s\n' 'Connexion locale
échouée'; exit 1; }
read -r -s -p 'Clé locale INDUSENSE_API_KEY (saisie masquée) : ' api_key
printf '\n'
case "$api_key" in ''|*$'\r'*|*$'\n'*) printf '%s\n' 'Clé vide ou retour à la ligne :
reprendre la saisie.'; exit 1;; esac
escaped_api_key=${api_key//\\/\\\\}
escaped_api_key=${escaped_api_key//\"/\\\"}
printf 'header = "X-API-Key: %s"\n' "$escaped_api_key" | curl -sS --config - -X POST -H
'Content-Type: application/json' --data-binary '@payload.json' -w '\nHTTP %{http_code}\n'
http://127.0.0.1:8000/predict-tabular || { printf '%s\n' 'Connexion locale échouée'; exit 1;
}
unset api_key escaped_api_key
```

Chaque requête affiche le **corps**, puis `HTTP 200`, `HTTP 401` ou le code réellement obtenu. Un code processus curl nul ne prouve pas HTTP 200 : lis le suffixe HTTP. `HTTP 000` avec une erreur de connexion signifie qu'aucune réponse API n'a été obtenue. La clé saisie doit être celle configurée localement dans `.env` ; elle passe par l'entrée standard de curl, pas par une valeur collée dans la commande.

Arrêter le conteneur M27 après les preuves

Conserve les corps JSON et les codes HTTP sans clé visible, puis :

```
docker stop indusense-m27
docker ps --filter publish=8000 --format "table {{.Names}}\t{{.Ports}}"
```

L'option --rm supprime automatiquement ce conteneur après son arrêt. L'image indusense-api:m27 et les fichiers sont conservés. Si le conteneur est déjà absent, ne le retire pas de nouveau. Un autre nom publiant toujours 8000 est à faire identifier. Un conflit de port est possible, pas garanti.

TP durcir alléger Variante A

- Vérifier la Variante A : modèle présent dans l'image (sinon `/ready` 503).

Correction guidée et preuve Docker

Vérifie les réponses /health, /ready et POST /predict-tabular et l'utilisateur appuser. Une réponse 503 à /ready exige de contrôler l'artefact dans l'image. Arrête indusense-m27 après les preuves.

À ne pas oublier FAQ définitions

📖 **Définition. Image Docker** : paquet **immuable** de l'application + son runtime (empilés en **couches** mises en cache) ; en **Variante A**, l'**artefact modèle** y est *inclus* — l'image **n'est PAS** « le modèle figé », elle l'**emballe**. **Conteneur** : instance en exécution de l'image. **Multi-stage** : build lourd → runtime mince. **Variante A** : modèle livré dans l'image → `/ready` 200 d'emblée. **?** « gitignoré = absent de l'image ? » → **non** : Git et Docker sont 2 périmètres distincts (d'où `!artifacts/models`). **?**

« Pourquoi non-root ? » → si l'app est compromise, l'attaquant n'est **pas root** dans le conteneur. 🔑
À ne pas oublier de dire : **dépendances avant le code** (cache) et **`--host 0.0.0.0`** (sinon injoignable).

Transition module 28

Fin de demi journée J3 matin M27

Enregistre les fichiers dans VS Code. Retourne à la **racine Parcours** avant de sauvegarder. Les chemins ci-dessous correspondent aux fichiers travaillés dans cette demi-journée : retire ceux que tu n'as pas modifiés et ajoute uniquement les nouveaux fichiers que tu as réellement créés et relus. Les dossiers temporaires, `.env`, clés, bases SQLite, captures contenant des données privées et environnements `.venv` restent hors Git. Si `git status` signale une suppression volontaire, inspecte-la avec le formateur avant son ajout.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
$brancheAttendue = Read-Host 'Nom exact de ta branche personnelle choisie en début de demi-
journée'
$branche = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheAttendue) -or $branche -cne
$brancheAttendue -or $branche -eq 'main' -or $branche -like 'jalon/*') { throw 'Branche
courante différente de ta branche personnelle attendue : arrêt avant ajout, commit et push.'
}
git status --short
git diff --stat
git diff
$fichiers = @('Dockerfile', '.dockerignore')
# Relis et adapte cette liste avant de poursuivre.
$existants = @($fichiers | Where-Object { Test-Path -LiteralPath $_ })
if ($existants.Count -gt 0) {
    git add -- $existants
    if ($LASTEXITCODE -ne 0) { throw 'Ajout Git échoué : ne pas commiter.' }
}
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Corrige les erreurs signalées avant le commit.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J3 matin M27 travail et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire identité Git, hook ou conflit.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Impossible de contrôler les changements indexés.' }
git status --short
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Sauvegarde distante non configurée : commits locaux
conservés.' }
$suivi = git for-each-ref --format="%(upstream:short)" "refs/heads/$branche"
```

```
if ($suivi -cne "perso/$branche") {
    git push -u perso $branche
} else { git push perso $branche }
if ($LASTEXITCODE -ne 0) { throw 'Push échoué : ta sauvegarde distante reste à faire.' }
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso $branche
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
printf 'Nom exact de ta branche personnelle choisie en début de demi-journée : '
IFS= read -r branche_attendue
branche="$(git branch --show-current)" || exit 1
[ -n "$branche_attendue" ] && [ "$branche" = "$branche_attendue" ] || { printf 'Branche
différente de la branche personnelle attendue.\n'; exit 1; }
case "$branche" in main|jalon/*|'') printf 'Branche personnelle requise avant ajout, commit
et push.\n'; exit 1;; esac
git status --short
git diff --stat
git diff
fichiers=("Dockerfile" ".dockerignore")
# Relis et adapte cette liste avant de poursuivre.
for fichier in "${fichiers[@]}"; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then
  git commit -m "Sprint 3 J3 matin M27 travail et preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then
  printf 'Contrôle du diff impossible.\n'; exit 1
fi
git status --short
git remote get-url perso || exit 1
suivi="$(git for-each-ref --format='%(upstream:short)' "refs/heads/$branche")"
if [ "$suivi" != "perso/$branche" ]; then
  git push -u perso "$branche" || exit 1
else
  git push perso "$branche" || exit 1
fi
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso "$branche"
```

Attendu : un commit seulement si tu as des changements, un push terminé sans erreur et **le même hash** pour `HEAD` et la branche distante listée par `ls-remote`. Vérifie aussi ton dernier commit dans le dépôt personnel GitHub. `Everything up-to-date` est normal si tout était déjà envoyé. `nothing to commit` ne justifie pas un commit vide : vérifie et pousse les commits locaux éventuels. Si des fichiers restent dans `status`, conserve-les et identifie-les ; tu n'es pas encore prêt pour le prochain jalon.

Si le commit échoue à cause d'un hook qui reformate un fichier, relis le nouveau diff et indexe à nouveau le fichier avant un nouveau commit. Si le push répond `rejected` ou `non-fast-forward`, ne force pas : un autre historique distant doit être réconcilié avec le formateur. Si un conflit est déjà en cours, `git status` indique les fichiers concernés ; travaille avec le formateur avant d'ajouter et de commiter la résolution.

28 Déploiement local compose J3 après midi US3 3 C6

✓ **Preuve finale visée :** `docker compose up -d --wait` → services **healthy** · **smoke tests verts** · `POST /predict-tabular` HTTP 401 sans clé / HTTP 200 avec clé.

Début de demi journée J3 après midi M28

Terminal du dépôt Parcours. Même si tu y travaillais hier, redonne le chemin complet ; cela évite un `Set-Location -LiteralPath $projet` depuis un dossier déjà ouvert. Arrête d'abord le serveur qui occupe ce terminal avec Ctrl+C si nécessaire. Ne lance pas de Git dans le terminal occupé par un serveur.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
pwd
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
git status --short
git remote -v
git branch --show-current
if (git status --porcelain) { throw 'Travail non commité : fais un checkpoint avant la
récupération.' }
$branche = Read-Host 'Nom exact de ta branche personnelle du Sprint 3'
if ([string]::IsNullOrWhiteSpace($branche) -or $branche -eq 'main' -or $branche -like
'jalon/*') { throw 'Utilise ta branche personnelle.' }
git switch $branche
if ($LASTEXITCODE -ne 0) { throw 'Branche non ouverte : lire le message Git.' }
code .
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Remote perso absent : configure ta destination de
sauvegarde avant la suite.' }
$refPerso = git ls-remote --heads perso $branche
if ($LASTEXITCODE -ne 0) { throw 'Accès distant échoué : ne pas continuer comme si le pull
avait réussi.' }
if ($refPerso) {
```

```
    git pull --ff-only perso $branche
    if ($LASTEXITCODE -ne 0) { throw 'Pull refusé : conserve ton travail et appelle le
formateur.' }
} else { Write-Host 'Premier passage : branche encore absente de perso, premier push à la
clôture.' }
git status --short
git branch --show-current
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
pwd
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
git status --short
git remote -v
git branch --show-current
[ -z "$(git status --porcelain)" ] || { printf 'Fais un checkpoint avant la récupération.\n';
exit 1; }
printf 'Nom exact de ta branche personnelle du Sprint 3 : '
IFS= read -r branche
case "$branche" in main|jalon/*|'') printf 'Utilise ta branche personnelle.\n'; exit 1;; esac
git switch "$branche" || exit 1
code .
git remote get-url perso || exit 1
ref_perso="$(git ls-remote --heads perso "$branche")" || exit 1
if [ -n "$ref_perso" ]; then
  git pull --ff-only perso "$branche" || exit 1
else
  printf 'Branche absente de perso : premier push à la clôture.\n'
fi
git status --short
git branch --show-current
```

Attendu : racine Parcours, branche personnelle et remote de sauvegarde corrects. Si `status` n'est pas vide, enregistre tes fichiers, lis le diff et sélectionne ceux à commiter comme dans la clôture ci-dessous. Ne récupère pas un jalon par-dessus du travail non commité. Si le pull échoue pour authentification, vérifie le compte et les droits dans le mécanisme GitHub proposé ; ne colle aucun token dans une commande. Un refus `--ff-only` indique des historiques divergents : fais les contrôles avec le formateur, sans reset ni push forcé.

Au signal du formateur, intègre **jalon/06** sur cette même branche avec les deux commandes suivantes. Le script effectue la fusion nécessaire des commits du cours ; `--ff-only` ci-dessus concernait uniquement la sauvegarde personnelle.

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 06
if ($LASTEXITCODE -ne 0) { throw 'Jalon non intégré : lire le diagnostic, ne pas lancer le
module.' }
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 06
if ($LASTEXITCODE -ne 0) { throw 'Vérification du jalon échouée.' }
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
```

```
git branch --show-current
```

```
bash scripts/formation/mettre_a_niveau.sh 06 || exit 1
bash scripts/formation/verifier_jalon.sh 06 || exit 1
head -n 1 FORMATION/JALON_ACTUEL.md
git branch --show-current
```

Attendu : `Jalon integre sur … : jalon/06`, puis `Jalon verifie : jalon/06`, le marqueur correspondant au tableau de progression, et ta branche personnelle. En cas de conflit, le script annule sa fusion ; lis le diagnostic avec le formateur avant un éventuel rattrapage.

Reprise et préflight Compose

- Ouvre **Docker Desktop**, puis dans VS Code **Terminal > Nouveau terminal**. Reviens à la racine `CISIA_24082026_Parcours`, vérifie `docker version`, `docker compose version` et que indusense-m27 a été arrêté après les preuves.

Théorie réseau privé démarré prêt

- 👀 Slide « Plusieurs services qui se parlent » + « Démarré n'est pas prêt ».

Tour du code docker compose yml réel

- 👀 `docker-compose.yml` : service `api` (`depends_on: db: condition: service_healthy`, env `INDUSENSE_API_KEY`, `INDUSENSE_DB_URL=postgresql+psycopg://…@db:5432/indusense`), service `db` (`postgres:16`, healthcheck `pg_isready -U indusense`, volume `pgdata`), **prometheus** + **grafana** déjà déclarés.
- 👀 `.env` (gitignoré) : `INDUSENSE_API_KEY`, `POSTGRES_PASSWORD`. On ne versionne que `.env.example`.

Pause obligatoire en visio

- Coupe le partage d'écran. À la reprise, reste dans le même terminal et valide d'abord la configuration avant tout `up`.

TP compose up smoke tests

```
if (-not (Test-Path .\.env)) {
    Copy-Item -LiteralPath .\.env.example -Destination .\.env
}
docker compose config --quiet
if ($LASTEXITCODE -ne 0) { throw 'Configuration Compose invalide.' }
docker compose up -d --wait
docker compose ps          # api + db : healthy ; prometheus + grafana : running
# Compléter puis activer le squelette selon les trois tests ci-dessous.
```

☑ **Résultat attendu :** `api` + `db` **healthy**, Prometheus + Grafana **running** · `/predict-tabular` 401 sans clé, 200 avec · 3 smoke tests verts. Un service sans healthcheck ne peut pas afficher `healthy` : `running` est alors normal.

Compléter et activer les trois smoke tests M28

Le jalon 06 livre un squelette `tests/test_smoke_compose.py` avec `@pytest.mark.skip` et `NotImplementedError`. Complète cet exercice avant sa validation : le simple lancement peut afficher **1 skipped**, ce qui signifie que le test n'a pas été exécuté.

Écris trois contrôles réseau avec `httpx`, déjà disponible dans l'environnement :

1. GET /health et GET /ready renvoient HTTP 200.
2. POST /predict-tabular, avec le corps valide livré mais sans X-API-Key, renvoie HTTP 401.
3. Le même POST avec la clé locale renvoie HTTP 200.

Utilise `http://127.0.0.1:8000` comme URL de base et les huit relevés du `payload.json` du dépôt. Donne un délai de 10 secondes aux requêtes. Lis la clé depuis `os.getenv("INDUSENSE_API_KEY")`, sans l'écrire dans le test ou dans Git. Retire le `skip` inconditionnel une fois les fonctions complétées ; protège les tests réseau avec `pytest.mark.skipif(os.getenv("INDUSENSE_COMPOSE_SMOKE") != "1", reason="Activer après Compose")`. Si tu utilises la correction REX remise par le formateur, elle contient déjà ces trois contrôles et ce garde : la variable est tout de même nécessaire.

Dans PowerShell, à la racine Parcours, saisis la clé locale puis active explicitement ces tests :

```
$cleSmoke = Read-Host 'Clé locale INDUSENSE_API_KEY' -AsSecureString
$env:INDUSENSE_API_KEY = [System.Net.NetworkCredential]::new('', $cleSmoke).Password
$env:INDUSENSE_COMPOSE_SMOKE = '1'
$env:INDUSENSE_SMOKE_BASE_URL = 'http://127.0.0.1:8000'
try {
    uv run --frozen pytest tests/test_smoke_compose.py -q
    if ($LASTEXITCODE -ne 0) { throw 'Smoke échoué : lire les assertions.' }
} finally {
    Remove-Item Env:INDUSENSE_API_KEY, Env:INDUSENSE_COMPOSE_SMOKE,
Env:INDUSENSE_SMOKE_BASE_URL
    Remove-Variable cleSmoke
}
```

Dans Bash :

```
read -r -s -p 'Clé locale INDUSENSE_API_KEY : ' INDUSENSE_API_KEY
printf '\n'
export INDUSENSE_API_KEY
export INDUSENSE_COMPOSE_SMOKE=1
export INDUSENSE_SMOKE_BASE_URL=http://127.0.0.1:8000
uv run --frozen pytest tests/test_smoke_compose.py -q
code_smoke=$?
unset INDUSENSE_API_KEY INDUSENSE_COMPOSE_SMOKE INDUSENSE_SMOKE_BASE_URL
[ "$code_smoke" -eq 0 ] || { printf 'Smoke échoué.\n'; exit 1; }
```

Attendu après cette complétion : **3 passed et 0 skipped**. `api` et `db` sont `healthy` ; Prometheus et Grafana sont `running`. Le healthcheck de l'API appelle /health : cela prouve qu'elle répond, tandis que le premier test vérifie aussi /ready et donc le modèle chargé. Ces contrôles ne prouvent pas qu'une prédiction a été écrite dans PostgreSQL ; l'écriture et l'idempotence sont vérifiées en M30.

Lire le contrat API et contrôler le code HTTP

Dans le navigateur, ouvre http://127.0.0.1:8000/docs pour Swagger UI et http://127.0.0.1:8000/openapi.json pour le contrat JSON. Ce sont des URL locales sur **ton poste**, accessibles pendant l'exécution du serveur de ce module. Si Swagger reste vide hors ligne, ouvre le JSON pour examiner les chemins, paramètres et schémas.

Dans Swagger, ouvre **POST /predict-tabular**, clique **Try it out**, remplis le paramètre d'en-tête **X-API-Key**, copie le corps du `payload.json` fourni à la racine du dépôt, puis clique **Execute**. Lis **Server response / Code** et **Response body**. La clé est un paramètre de cette opération ; ne cherche pas un bouton global Authorize qui n'est pas déclaré par ce code. Coupe le partage d'écran pendant sa saisie et masque l'en-tête dans tes captures.

Requête	Code HTTP attendu	Corps à lire
GET /health	200	`{"status":"ok"}`, le processus répond
GET /ready avec modèle chargé	200	`status` vaut `ready`, avec `model_version`
GET /ready sans modèle	503	`detail` indique que le modèle n'est pas chargé

Requête	Code HTTP attendu	Corps à lire
POST /predict-tabular sans clé	401	refus d'authentification
POST /predict-tabular avec clé et corps valides	200	`machine_id`, `proba_panne`, `decision`, `model_version`, `threshold`

200 est un code HTTP ; **ok** ou **ready** est une valeur JSON. La prédiction ne contient pas de champ `status`. Pour le payload livré, `machine_id` doit être `MACH-07`. `proba_panne` est entre 0 et 1 ; `decision` vaut `alerte` si la probabilité atteint `threshold`, sinon `ok`. La valeur numérique et la version dépendent de l'artefact chargé : relève celles que ton poste renvoie.

Le contrat exige au moins **sept relevés** ; le `payload.json` livré en contient **huit**, de 00 h à 07 h le 1er février 2025, avec température 50 à 57 et pression 195 à 198,5. Chaque relevé contient `timestamp`, `temperature` et `pressure_bar`. Utilise le fichier fourni ; les corps d'erreur 422 varient selon ce qui est invalide.

Dans un **terminal de contrôle distinct du terminal serveur**, utilise le bloc adapté ci-dessous. Il redemande le chemin du dépôt et refuse un sous-dossier ou un autre dépôt avant les commandes API.

Windows PowerShell utilise **curl.exe**, car `curl` peut désigner un alias :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
if (-not (Test-Path -LiteralPath .\payload.json)) { throw 'payload.json absent : vérifier
dossier et jalon.' }
curl.exe -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/health
if ($LASTEXITCODE -ne 0) { throw 'Échec de connexion /health.' }
curl.exe -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/ready
if ($LASTEXITCODE -ne 0) { throw 'Échec de connexion /ready.' }
curl.exe -sS -X POST -H 'Content-Type: application/json' --data-binary '@payload.json' -w
"\nHTTP %{http_code}\n" http://127.0.0.1:8000/predict-tabular
if ($LASTEXITCODE -ne 0) { throw 'Échec de connexion pour le POST sans clé.' }

# Saisir la même clé locale que celle de .env, sans la mettre dans la commande.
$apiKeySecure = Read-Host 'Clé locale INDUSENSE_API_KEY (saisie masquée)' -AsSecureString
$apiKey = [System.Net.NetworkCredential]::new('', $apiKeySecure).Password
if ([string]::IsNullOrWhiteSpace($apiKey) -or $apiKey.Contains("`r") -or
$apiKey.Contains("`n")) { throw 'Clé vide ou retour à la ligne : reprendre la saisie.' }
$escapedApiKey = $apiKey.Replace('\', '\\').Replace('"', '\"')
@"
header = "X-API-Key: $escapedApiKey"
"@ | curl.exe -sS --config - -X POST -H 'Content-Type: application/json' --data-binary
'@payload.json' -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/predict-tabular
if ($LASTEXITCODE -ne 0) { throw 'Échec de connexion pour le POST avec clé.' }
```

```
Remove-Variable apiKey, escapedApiKey, apiKeySecure
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
test -f payload.json || { printf '%s\n' 'payload.json absent : vérifier dossier et jalon.';
exit 1; }
curl -sS -w '\nHTTP %{http_code}\n' http://127.0.0.1:8000/health || { printf '%s\n'
'Connexion locale échouée'; exit 1; }
curl -sS -w '\nHTTP %{http_code}\n' http://127.0.0.1:8000/ready || { printf '%s\n' 'Connexion
locale échouée'; exit 1; }
curl -sS -X POST -H 'Content-Type: application/json' --data-binary '@payload.json' -w '\nHTTP
%{http_code}\n' http://127.0.0.1:8000/predict-tabular || { printf '%s\n' 'Connexion locale
échouée'; exit 1; }
read -r -s -p 'Clé locale INDUSENSE_API_KEY (saisie masquée) : ' api_key
printf '\n'
case "$api_key" in ''|*$'\r'*|*$'\n'*) printf '%s\n' 'Clé vide ou retour à la ligne :
reprendre la saisie.'; exit 1;; esac
escaped_api_key=${api_key//\\/\\\\}
escaped_api_key=${escaped_api_key//\"/\\\"}
printf 'header = "X-API-Key: %s"\n' "$escaped_api_key" | curl -sS --config - -X POST -H
'Content-Type: application/json' --data-binary '@payload.json' -w '\nHTTP %{http_code}\n'
http://127.0.0.1:8000/predict-tabular || { printf '%s\n' 'Connexion locale échouée'; exit 1;
}
unset api_key escaped_api_key
```

Chaque requête affiche le **corps**, puis `HTTP 200`, `HTTP 401` ou le code réellement obtenu. Un code processus curl nul ne prouve pas HTTP 200 : lis le suffixe HTTP. `HTTP 000` avec une erreur de connexion signifie qu'aucune réponse API n'a été obtenue. La clé saisie doit être celle configurée localement dans `.env` ; elle passe par l'entrée standard de curl, pas par une valeur collée dans la commande.

Autonomie correction et preuve du module 28

- Rejoue le démarrage depuis une stack arrêtée, vérifie le healthcheck de `db`, puis refais les trois smoke tests. Corrige la course au démarrage avant de capturer la preuve finale.

☑ **Résultat attendu :** `docker compose ps` montre la stack attendue, `/health` répond 200 et le smoke est reproductible. Le module 29 commence au signal du formateur en J4 matin.

QCM de fin de JOURNÉE 3

- **Passe le QCM J3** (`kahoot_J3.xlsx`).

À ne pas oublier FAQ définitions

📖 **Définition. Service** : un conteneur déclaré dans compose. **Healthcheck** : sonde définie pour le service ; ici /health vérifie la liveness de l'API, /ready son modèle et pg_isready PostgreSQL. **Réseau compose** : on joint par le **nom** du service. **?** « `localhost` ou `db` ? » → **`db:5432`** ; dans un conteneur,

`localhost` = lui-même. ❓ « Pourquoi l'API plante 1 fois sur 2 ? » → **course au démarrage** : `depends_on` sans `condition: service_healthy`. 🔑 À ne pas oublier de dire : secrets dans **`.env` gitignoré** ; on ne versionne que `.env.example`.

- 🔗 « L'**image** du module 27 devient un **service** dans compose ; le `/health` du 25 sert de **healthcheck**. »

🗺️ **Topologie compose** (noms de service, sens du scrape, emplacement du modèle) :

Le client appelle l'API sur 8000. L'API joint PostgreSQL via db:5432 et charge le modèle de l'image. Prometheus collecte /metrics sur api:8000 ; Grafana utilise prometheus:9090.

*(Un 2e job Prometheus `indusense-drift` scrape l'exporteur du TP, **hors Docker**, via `host.docker.internal:9109` — down au préflight = normal.)*

Transition module 29

Fin de demi journée J3 après midi M28

Enregistre les fichiers dans VS Code. Retourne à la **racine Parcours** avant de sauvegarder. Les chemins ci-dessous correspondent aux fichiers travaillés dans cette demi-journée : retire ceux que tu n'as pas modifiés et ajoute uniquement les nouveaux fichiers que tu as réellement créés et relus. Les dossiers temporaires, `.env`, clés, bases SQLite, captures contenant des données privées et environnements `.venv` restent hors Git. Si `git status` signale une suppression volontaire, inspecte-la avec le formateur avant son ajout.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
$brancheAttendue = Read-Host 'Nom exact de ta branche personnelle choisie en début de demi-
journée'
$branche = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheAttendue) -or $branche -cne
$brancheAttendue -or $branche -eq 'main' -or $branche -like 'jalon/*') { throw 'Branche
courante différente de ta branche personnelle attendue : arrêt avant ajout, commit et push.'
}
git status --short
git diff --stat
git diff
$fichiers = @('docker-compose.yml', 'tests/test_smoke_compose.py')
# Relis et adapte cette liste avant de poursuivre.
$existants = @($fichiers | Where-Object { Test-Path -LiteralPath $_ })
if ($existants.Count -gt 0) {
    git add -- $existants
    if ($LASTEXITCODE -ne 0) { throw 'Ajout Git échoué : ne pas commiter.' }
}
git diff --cached --check
```

```
if ($LASTEXITCODE -ne 0) { throw 'Corrige les erreurs signalées avant le commit.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J3 après midi M28 travail et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire identité Git, hook ou conflit.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Impossible de contrôler les changements indexés.' }
git status --short
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Sauvegarde distante non configurée : commits locaux
conservés.' }
$suivi = git for-each-ref --format="%(upstream:short)" "refs/heads/$branche"
if ($suivi -cne "perso/$branche") {
    git push -u perso $branche
} else { git push perso $branche }
if ($LASTEXITCODE -ne 0) { throw 'Push échoué : ta sauvegarde distante reste à faire.' }
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso $branche
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
printf 'Nom exact de ta branche personnelle choisie en début de demi-journée : '
IFS= read -r branche_attendue
branche="$(git branch --show-current)" || exit 1
[ -n "$branche_attendue" ] && [ "$branche" = "$branche_attendue" ] || { printf 'Branche
différente de la branche personnelle attendue.\n'; exit 1; }
case "$branche" in main|jalon/*|'') printf 'Branche personnelle requise avant ajout, commit
et push.\n'; exit 1;; esac
git status --short
git diff --stat
git diff
fichiers=("docker-compose.yml" "tests/test_smoke_compose.py")
# Relis et adapte cette liste avant de poursuivre.
for fichier in "${fichiers[@]}"; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then
  git commit -m "Sprint 3 J3 après midi M28 travail et preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then
  printf 'Contrôle du diff impossible.\n'; exit 1
fi
git status --short
git remote get-url perso || exit 1
```

```
suivi="$(git for-each-ref --format='%(upstream:short)' "refs/heads/$branche")"
if [ "$suivi" != "perso/$branche" ]; then
  git push -u perso "$branche" || exit 1
else
  git push perso "$branche" || exit 1
fi
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso "$branche"
```

Attendu : un commit seulement si tu as des changements, un push terminé sans erreur et **le même hash** pour `HEAD` et la branche distante listée par `ls-remote`. Vérifie aussi ton dernier commit dans le dépôt personnel GitHub. `Everything up-to-date` est normal si tout était déjà envoyé. `nothing to commit` ne justifie pas un commit vide : vérifie et pousse les commits locaux éventuels. Si des fichiers restent dans `status`, conserve-les et identifie-les ; tu n'es pas encore prêt pour le prochain jalon.

Si le commit échoue à cause d'un hook qui reformate un fichier, relis le nouveau diff et indexe à nouveau le fichier avant un nouveau commit. Si le push répond `rejected` ou `non-fast-forward`, ne force pas : un autre historique distant doit être réconcilié avec le formateur. Si un conflit est déjà en cours, `git status` indique les fichiers concernés ; travaille avec le formateur avant d'ajouter et de commiter la résolution.

29 Orchestration Prefect design J4 matin 1 2 US3 4 C6 C7

✓ **Preuve finale visée :** flow « hello » exécutable (retries) · design `ingest→feature→predict→store` (schéma + table I/O).

Notice locale complémentaire : `Guide_Prefect_3_local_InduSense.pdf` détaille le serveur et le dashboard self-hosted, les déploiements déclaratifs (`prefect.yaml`), Git, les work pools/workers, le rôle éventuel de Terraform et la sécurité. La notice est incluse dans ce même dossier ; n'improvise ni connexion Cloud ni exposition réseau de l'UI.

Début de demi journée J4 matin M29 et M30

Terminal du dépôt Parcours. Même si tu y travaillais hier, redonne le chemin complet ; cela évite un `Set-Location -LiteralPath $projet` depuis un dossier déjà ouvert. Arrête d'abord le serveur qui occupe ce terminal avec Ctrl+C si nécessaire. Ne lance pas de Git dans le terminal occupé par un serveur.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
pwd
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
git status --short
git remote -v
```

```
git branch --show-current
if (git status --porcelain) { throw 'Travail non commité : fais un checkpoint avant la
récupération.' }
$branche = Read-Host 'Nom exact de ta branche personnelle du Sprint 3'
if ([string]::IsNullOrWhiteSpace($branche) -or $branche -eq 'main' -or $branche -like
'jalon/*') { throw 'Utilise ta branche personnelle.' }
git switch $branche
if ($LASTEXITCODE -ne 0) { throw 'Branche non ouverte : lire le message Git.' }
code .
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Remote perso absent : configure ta destination de
sauvegarde avant la suite.' }
$refPerso = git ls-remote --heads perso $branche
if ($LASTEXITCODE -ne 0) { throw 'Accès distant échoué : ne pas continuer comme si le pull
avait réussi.' }
if ($refPerso) {
    git pull --ff-only perso $branche
    if ($LASTEXITCODE -ne 0) { throw 'Pull refusé : conserve ton travail et appelle le
formateur.' }
} else { Write-Host 'Premier passage : branche encore absente de perso, premier push à la
clôture.' }
git status --short
git branch --show-current
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
pwd
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
git status --short
git remote -v
git branch --show-current
[ -z "$(git status --porcelain)" ] || { printf 'Fais un checkpoint avant la récupération.\n';
exit 1; }
printf 'Nom exact de ta branche personnelle du Sprint 3 : '
IFS= read -r branche
case "$branche" in main|jalon/*|'') printf 'Utilise ta branche personnelle.\n'; exit 1;; esac
git switch "$branche" || exit 1
code .
git remote get-url perso || exit 1
ref_perso="$(git ls-remote --heads perso "$branche")" || exit 1
if [ -n "$ref_perso" ]; then
  git pull --ff-only perso "$branche" || exit 1
else
  printf 'Branche absente de perso : premier push à la clôture.\n'
fi
git status --short
git branch --show-current
```

Attendu : racine Parcours, branche personnelle et remote de sauvegarde corrects. Si `status` n'est pas vide, enregistre tes fichiers, lis le diff et sélectionne ceux à commiter comme dans la clôture ci-

dessous. Ne récupère pas un jalon par-dessus du travail non commité. Si le pull échoue pour authentification, vérifie le compte et les droits dans le mécanisme GitHub proposé ; ne colle aucun token dans une commande. Un refus `--ff-only` indique des historiques divergents : fais les contrôles avec le formateur, sans reset ni push forcé.

Au signal du formateur, intègre **jalon/07** sur cette même branche avec les deux commandes suivantes. Le script effectue la fusion nécessaire des commits du cours ; `--ff-only` ci-dessus concernait uniquement la sauvegarde personnelle.

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 07
if ($LASTEXITCODE -ne 0) { throw 'Jalon non intégré : lire le diagnostic, ne pas lancer le
module.' }
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 07
if ($LASTEXITCODE -ne 0) { throw 'Vérification du jalon échouée.' }
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
git branch --show-current
```

```
bash scripts/formation/mettre_a_niveau.sh 07 || exit 1
bash scripts/formation/verifier_jalon.sh 07 || exit 1
head -n 1 FORMATION/JALON_ACTUEL.md
git branch --show-current
```

Attendu : `Jalon integre sur … : jalon/07`, puis `Jalon verifie : jalon/07`, le marqueur correspondant au tableau de progression, et ta branche personnelle. En cas de conflit, le script annule sa fusion ; lis le diagnostic avec le formateur avant un éventuel rattrapage.

Ouverture préflight scénario A

- Préflight : **Prefect est déjà dans `uv sync`** (pas de `uv add`). Vérifier : `uv run python -c "import prefect; print(prefect.__version__)"` → **3.7.6**. Tout tourne en **local** (pas de serveur Prefect requis).

Théorie ciblée pourquoi orchestrer

- 👀 Slide « Pourquoi orchestrer (et pas juste cron) » + « Idempotence & retries ».

Le piège data fuite inter machines à jouer

- Debug à jouer : symptôme (pressions incohérentes) → hypothèse en binôme → correction `merge_asof(..., by="machine", tolerance="90min", direction="nearest")` → test qui repasse.

☑ **Résultat attendu :** Résidu non-joint **≈ 1,76 %**.

TP socle flow hello design

```
# créer d'abord src/indusense/flows/hello.py + __init__.py vide (code : fiche TD 29)
uv run python -m indusense.flows.hello   # pong:indusense + run loggé, retries
```

- Dessiner le flow `ingest→feature→predict→store` + une **table I/O** (entrées/sorties/erreurs de chaque task), à figer **avant** d'implémenter.

Preuve FAQ et correction

📖 **Définition. Task** : étape unitaire. **Flow** : orchestration des tasks. **Idempotent** : rejouer ne change rien. **Backoff** : délai croissant entre réessais. ❓ « Pourquoi pas un simple cron ? » → cron lance, point ; un orchestrateur ajoute **retries, observabilité, dépendances**. ❓ « On réessaie toutes les erreurs ? » → non : seulement les **transitoires** (DB 1 s), pas les déterministes. 🔑 À ne pas oublier de dire : `merge_asof` **sans `by="machine"`** ne lève **aucune erreur** — bug silencieux.

- 🔗 « Rappelez la **fuite de données** du module 23 : ici c'est la même idée mais **entre machines** (`by="machine"`). »

Transition module 30

Pause obligatoire en visio

- Coupe le partage d'écran et ne dépanne pas pendant la pause. À la reprise, écris dans le chat le backend choisi : **PostgreSQL dans Compose** ou **SQLite sur l'hôte**.
- ☑ La preuve d'idempotence est **deux runs contre la même base neuve → même nombre de lignes**.

30 Implémentation du flow J4 matin 2 2 US3 4 C6 C7

✓ **Preuve finale visée :** flow end-to-end · **2 runs → 0 doublon** · `panne` **≈ 4,7802 %**.

Ouverture préflight scénario A

- Préflight : **SQLAlchemy + psycopg sont déjà verrouillés**. Exécute :

```
uv sync --frozen
if ($LASTEXITCODE -ne 0) { throw "uv sync a échoué : code $LASTEXITCODE" }
$env:PREFECT_PROFILE = "ephemeral"
$env:PREFECT_SERVER_ANALYTICS_ENABLED = "false"
$env:PREFECT_CLOUD_ENABLE_ORCHESTRATION_TELEMETRY = "false"
$qaStamp = Get-Date -Format "yyyyMMdd_HHmmss"
$env:COMPOSE_PROJECT_NAME = "cisia_m30_$qaStamp"
docker compose up -d --wait db
if ($LASTEXITCODE -ne 0) { throw "Démarrage DB a échoué : code $LASTEXITCODE" }
```

PostgreSQL `@db:5432` exige que le flow tourne dans Compose. Le nom de projet isole sa base des volumes existants ; sur l'hôte, utilise une base SQLite de preuve neuve sous le dossier temporaire Windows (`$env:TEMP`).

Théorie ciblée historiser upsert

- 👀 Slide « Historiser : la table predictions » + « L'idempotence par upsert ».
- Au bord de `store`, adapte `prediction_ts` au pilote : `pd.Timestamp(value).isoformat()` pour SQLite ; `pd.Timestamp(value).to_pydatetime()` pour PostgreSQL. Un `pandas.Timestamp` brut n'est pas accepté par SQLite.
- `prediction_ts` conserve l'heure métier **sans fuseau**, comme les loaders ; seul `created_at` est un instant d'audit avec fuseau. `store` lit `INDUSENSE_DB_URL` dans son environnement : ne passe ni ne journalise cette URL dans les paramètres Prefect. Ne journalise jamais les colonnes opérateur brutes des incidents.

Le piège data cible date seule à jouer

- Debug : symptôme `panne.mean()` ≈ 4,89 % → diagnostic (`time` à minuit) → correction `pd.to_datetime(date.astype(str) + " " + time.astype(str))` → retour à **4,7802 %**.

TP socle predict store upsert preuve d idempotence

- **Mode PostgreSQL.** Reconstruis l'image après création de `predict_flow.py`, monte le jeu complet en lecture seule et lance le flow dans le service `api` :

```
if (-not $env:INDUSENSE_DATA_DIR) {
  throw "Définir INDUSENSE_DATA_DIR vers le jeu complet"
}
$sourceData = (Resolve-Path -LiteralPath $env:INDUSENSE_DATA_DIR `
  -ErrorAction Stop).Path
docker compose build api
if ($LASTEXITCODE -ne 0) { throw "Build API a échoué : code $LASTEXITCODE" }
$flowArgs = @(
  "compose", "run", "--rm", "--no-deps",
  "-e", "PREFECT_PROFILE=ephemeral",
```

```
  "-e", "INDUSENSE_DATA_DIR=/app/data/run",
  "--volume", "${sourceData}:/app/data/run:ro",
  "api", "python", "-m", "indusense.flows.predict_flow"
)
docker @flowArgs
if ($LASTEXITCODE -ne 0) { throw "Flow run 1 a échoué : code $LASTEXITCODE" }
$count1 = (docker compose exec -T db psql -U indusense `
  -d indusense -tA -c "SELECT count(*) FROM predictions;").Trim()
if ($LASTEXITCODE -ne 0) { throw "Lecture count1 a échoué : code $LASTEXITCODE" }
docker @flowArgs
if ($LASTEXITCODE -ne 0) { throw "Flow run 2 a échoué : code $LASTEXITCODE" }
$count2 = (docker compose exec -T db psql -U indusense `
  -d indusense -tA -c "SELECT count(*) FROM predictions;").Trim()
if ($LASTEXITCODE -ne 0) { throw "Lecture count2 a échoué : code $LASTEXITCODE" }
if ([int]$count1 -ne [int]$count2) {
  throw "Idempotence KO : $count1 puis $count2"
}
if ([int]$count2 -ne 15) { throw "Population scorée inattendue : $count2 au lieu de 15" }
```

🛟 **Repli SQLite sur l'hôte.** Garde le même jeu complet, sinon tu changes de preuve :

```
if (-not $env:INDUSENSE_DATA_DIR) { throw "Définir INDUSENSE_DATA_DIR vers le jeu complet" }
$env:INDUSENSE_DATA_DIR = (Resolve-Path -LiteralPath $env:INDUSENSE_DATA_DIR `
  -ErrorAction Stop).Path
$env:PREFECT_PROFILE = "ephemeral"
$env:PREFECT_SERVER_ANALYTICS_ENABLED = "false"
$env:PREFECT_CLOUD_ENABLE_ORCHESTRATION_TELEMETRY = "false"
$qaStamp = (Get-Date -Format "yyyyMMdd_HHmmss") + "_" +
([guid]::NewGuid().ToString("N").Substring(0,8))
$qaDir = New-Item -ItemType Directory -Path (Join-Path $env:TEMP "cisia_m30_$qaStamp")
$dbPath = Join-Path $qaDir.FullName "predictions_flow.db"
if (Test-Path -LiteralPath $dbPath) { throw "Refus d'écraser la base : $dbPath" }
$env:INDUSENSE_DB_URL = "sqlite:///$($dbPath.Replace('\','/'))"
uv run python -m indusense.flows.predict_flow
if ($LASTEXITCODE -ne 0) { throw "Flow SQLite run 1 a échoué : code $LASTEXITCODE" }
$countScript = @'
import sqlite3, sys
cx = sqlite3.connect(sys.argv[1])
print(cx.execute("select count(*) from predictions").fetchone()[0])
'@
$count1 = uv run python -c $countScript $dbPath
if ($LASTEXITCODE -ne 0) { throw "Lecture count1 a échoué : code $LASTEXITCODE" }
uv run python -m indusense.flows.predict_flow
if ($LASTEXITCODE -ne 0) { throw "Flow SQLite run 2 a échoué : code $LASTEXITCODE" }
$count2 = uv run python -c $countScript $dbPath
if ($LASTEXITCODE -ne 0) { throw "Lecture count2 a échoué : code $LASTEXITCODE" }
if ([int]$count1 -ne [int]$count2) { throw "Idempotence KO : $count1 puis $count2" }
if ([int]$count2 -ne 15) { throw "Population scorée inattendue : $count2 au lieu de 15" }
```

☑ **Résultat attendu :** les logs distinguent `joined_rows=65625`, `joined_panne_count=3137`, `joined_panne_rate=0.04780190` de `gold_rows=64625`, `gold_panne_count=3109`, `gold_panne_rate=0.04810832`, sans publier les lignes sources ; `count1 == count2 == 15` · **0 doublon** · `store` a des retries. La base SQLite reste sous `$env:TEMP` et l'ancien `predictions.db` du dépôt n'est jamais ouvert. Le starter donne ≈ 10,42 % brut / ≈ 10,55 % entraînable : nomme toujours la population.

Autonomie tutorée courte

- Choisis une seule option : ajouter une assertion anti-doublon dans un test, ou documenter le contrat I/O de `store`. Arrête au signal du formateur même si ce bonus n'est pas fini ; la preuve socle prime.

QCM express 29 30 FAQ et preuve finale

- **Passe le QCM J4** (`kahoot_J4.xlsx`) — premier récap ; la **fin de journée 4** (avec le drift PayGuard) arrive après le module 32.

À ne pas oublier FAQ définitions

📖 **Définition. Clé naturelle** : colonnes qui identifient une ligne (`machine, prediction_ts`). **Upsert** : insère **ou** met à jour (`ON CONFLICT … DO UPDATE`). **Idempotence** : 2 runs → même `count(*)`. ❓ « `db` est introuvable / Prefect répond 401 Cloud ? » → `db` n'existe que dans Compose ; utilise le service `api`. Garde `PREFECT_PROFILE=ephemeral` pour la preuve locale. ❓ « Pourquoi mes lignes doublent à la relance ? » → `INSERT` simple → passer à l'**upsert**. ❓ « `type 'Timestamp' is not supported` ? » → le paramètre SQLite est encore un objet pandas ; convertir en ISO 8601 dans `store` sans perdre l'heure. ❓ « Pourquoi 4,89 % et pas 4,78 % ? » → cible sur **`date` seule** (heure à minuit) → recombiner `date+time`. 🔑 À ne pas oublier de dire : le flow **réutilise** la lib testée — il ne réécrit **aucune** logique métier.

- 🔗 « Le **split temporel** du 23 revient : l'horodatage `date+time` décide de la cible `panne`. »

Transition module 31

Fin de demi journée J4 matin M29 et M30

Enregistre les fichiers dans VS Code. Retourne à la **racine Parcours** avant de sauvegarder. Les chemins ci-dessous correspondent aux fichiers travaillés dans cette demi-journée : retire ceux que tu n'as pas modifiés et ajoute uniquement les nouveaux fichiers que tu as réellement créés et relus. Les dossiers temporaires, `.env`, clés, bases SQLite, captures contenant des données privées et environnements `.venv` restent hors Git. Si `git status` signale une suppression volontaire, inspecte-la avec le formateur avant son ajout.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
$brancheAttendue = Read-Host 'Nom exact de ta branche personnelle choisie en début de demi-
journée'
$branche = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheAttendue) -or $branche -cne
$brancheAttendue -or $branche -eq 'main' -or $branche -like 'jalon/*') { throw 'Branche
courante différente de ta branche personnelle attendue : arrêt avant ajout, commit et push.'
}
git status --short
git diff --stat
git diff
$fichiers = @('src/indusense/flows/hello.py', 'src/indusense/flows/__init__.py',
'src/indusense/flows/predict_flow.py', 'docs/flow_design.md', 'docs/store_contract.md')
```

```
# Relis et adapte cette liste avant de poursuivre.
$existants = @($fichiers | Where-Object { Test-Path -LiteralPath $_ })
if ($existants.Count -gt 0) {
    git add -- $existants
    if ($LASTEXITCODE -ne 0) { throw 'Ajout Git échoué : ne pas commiter.' }
}
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Corrige les erreurs signalées avant le commit.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J4 matin M29 et M30 travail et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire identité Git, hook ou conflit.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Impossible de contrôler les changements indexés.' }
git status --short
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Sauvegarde distante non configurée : commits locaux
conservés.' }
$suivi = git for-each-ref --format="%(upstream:short)" "refs/heads/$branche"
if ($suivi -cne "perso/$branche") {
    git push -u perso $branche
} else { git push perso $branche }
if ($LASTEXITCODE -ne 0) { throw 'Push échoué : ta sauvegarde distante reste à faire.' }
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso $branche
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
printf 'Nom exact de ta branche personnelle choisie en début de demi-journée : '
IFS= read -r branche_attendue
branche="$(git branch --show-current)" || exit 1
[ -n "$branche_attendue" ] && [ "$branche" = "$branche_attendue" ] || { printf 'Branche
différente de la branche personnelle attendue.\n'; exit 1; }
case "$branche" in main|jalon/*|'') printf 'Branche personnelle requise avant ajout, commit
et push.\n'; exit 1;; esac
git status --short
git diff --stat
git diff
fichiers=("src/indusense/flows/hello.py" "src/indusense/flows/__init__.py"
"src/indusense/flows/predict_flow.py" "docs/flow_design.md" "docs/store_contract.md")
# Relis et adapte cette liste avant de poursuivre.
for fichier in "${fichiers[@]}"; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
```

```
code_diff=$?
if [ "$code_diff" -eq 1 ]; then
  git commit -m "Sprint 3 J4 matin M29 et M30 travail et preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then
  printf 'Contrôle du diff impossible.\n'; exit 1
fi
git status --short
git remote get-url perso || exit 1
suivi="$(git for-each-ref --format='%(upstream:short)' "refs/heads/$branche")"
if [ "$suivi" != "perso/$branche" ]; then
  git push -u perso "$branche" || exit 1
else
  git push perso "$branche" || exit 1
fi
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso "$branche"
```

Attendu : un commit seulement si tu as des changements, un push terminé sans erreur et **le même hash** pour `HEAD` et la branche distante listée par `ls-remote`. Vérifie aussi ton dernier commit dans le dépôt personnel GitHub. `Everything up-to-date` est normal si tout était déjà envoyé. `nothing to commit` ne justifie pas un commit vide : vérifie et pousse les commits locaux éventuels. Si des fichiers restent dans `status`, conserve-les et identifie-les ; tu n'es pas encore prêt pour le prochain jalon.

Si le commit échoue à cause d'un hook qui reformate un fichier, relis le nouveau diff et indexe à nouveau le fichier avant un nouveau commit. Si le push répond `rejected` ou `non-fast-forward`, ne force pas : un autre historique distant doit être réconcilié avec le formateur. Si un conflit est déjà en cours, `git status` indique les fichiers concernés ; travaille avec le formateur avant d'ajouter et de commiter la résolution.

31 Data drift métriques concepts J4 après midi et J5 matin US3 5 C3 C8

Début de demi journée J4 après midi PayGuard

Terminal du dépôt Parcours. Même si tu y travaillais hier, redonne le chemin complet ; cela évite un `Set-Location -LiteralPath $projet` depuis un dossier déjà ouvert. Arrête d'abord le serveur qui occupe ce terminal avec Ctrl+C si nécessaire. Ne lance pas de Git dans le terminal occupé par un serveur.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
pwd
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
git status --short
git remote -v
git branch --show-current
```

```
if (git status --porcelain) { throw 'Travail non commité : fais un checkpoint avant la
récupération.' }
$branche = Read-Host 'Nom exact de ta branche personnelle du Sprint 3'
if ([string]::IsNullOrWhiteSpace($branche) -or $branche -eq 'main' -or $branche -like
'jalon/*') { throw 'Utilise ta branche personnelle.' }
git switch $branche
if ($LASTEXITCODE -ne 0) { throw 'Branche non ouverte : lire le message Git.' }
code .
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Remote perso absent : configure ta destination de
sauvegarde avant la suite.' }
$refPerso = git ls-remote --heads perso $branche
if ($LASTEXITCODE -ne 0) { throw 'Accès distant échoué : ne pas continuer comme si le pull
avait réussi.' }
if ($refPerso) {
    git pull --ff-only perso $branche
    if ($LASTEXITCODE -ne 0) { throw 'Pull refusé : conserve ton travail et appelle le
formateur.' }
} else { Write-Host 'Premier passage : branche encore absente de perso, premier push à la
clôture.' }
git status --short
git branch --show-current
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
pwd
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
git status --short
git remote -v
git branch --show-current
[ -z "$(git status --porcelain)" ] || { printf 'Fais un checkpoint avant la récupération.\n';
exit 1; }
printf 'Nom exact de ta branche personnelle du Sprint 3 : '
IFS= read -r branche
case "$branche" in main|jalon/*|'') printf 'Utilise ta branche personnelle.\n'; exit 1;; esac
git switch "$branche" || exit 1
code .
git remote get-url perso || exit 1
ref_perso="$(git ls-remote --heads perso "$branche")" || exit 1
if [ -n "$ref_perso" ]; then
  git pull --ff-only perso "$branche" || exit 1
else
  printf 'Branche absente de perso : premier push à la clôture.\n'
fi
git status --short
git branch --show-current
```

Attendu : racine Parcours, branche personnelle et remote de sauvegarde corrects. Si `status` n'est pas vide, enregistre tes fichiers, lis le diff et sélectionne ceux à commiter comme dans la clôture ci-dessous. Ne récupère pas un jalon par-dessus du travail non commité. Si le pull échoue pour

authentification, vérifie le compte et les droits dans le mécanisme GitHub proposé ; ne colle aucun token dans une commande. Un refus `--ff-only` indique des historiques divergents : fais les contrôles avec le formateur, sans reset ni push forcé.

Au signal du formateur, intègre **jalon/08** sur cette même branche avec les deux commandes suivantes. Le script effectue la fusion nécessaire des commits du cours ; `--ff-only` ci-dessus concernait uniquement la sauvegarde personnelle.

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 08
if ($LASTEXITCODE -ne 0) { throw 'Jalon non intégré : lire le diagnostic, ne pas lancer le
module.' }
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 08
if ($LASTEXITCODE -ne 0) { throw 'Vérification du jalon échouée.' }
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
git branch --show-current
```

```
bash scripts/formation/mettre_a_niveau.sh 08 || exit 1
bash scripts/formation/verifier_jalon.sh 08 || exit 1
head -n 1 FORMATION/JALON_ACTUEL.md
git branch --show-current
```

Attendu : `Jalon integre sur … : jalon/08`, puis `Jalon verifie : jalon/08`, le marqueur correspondant au tableau de progression, et ta branche personnelle. En cas de conflit, le script annule sa fusion ; lis le diagnostic avec le formateur avant un éventuel rattrapage.

Ouvrir le dossier PayGuard

Après le jalon 08, extrait `FORMATION/EXERCICES/tp_payguard_apprenants.zip` à côté de Parcours sous `tp_payguard`. Le dossier à ouvrir est **tp_payguard/tp_autonome_drift**, car l'archive contient ce sous-dossier avec pyproject.toml et uv.lock. Ne réextrais pas par-dessus un travail existant.

```
$racineParcours = (git rev-parse --show-toplevel).Trim()
$payParent = Join-Path (Split-Path -Parent $racineParcours) 'tp_payguard'
if (-not (Test-Path -LiteralPath $payParent)) { Expand-Archive -LiteralPath
.\FORMATION\EXERCICES\tp_payguard_apprenants.zip -DestinationPath $payParent }
$payguard = Join-Path $payParent 'tp_autonome_drift'
Set-Location -LiteralPath $payguard -ErrorAction Stop
pwd
Test-Path .\pyproject.toml
Test-Path .\uv.lock
code -n .
```

```
racine_parcours="$(git rev-parse --show-toplevel)" || exit 1
pay_parent="$(dirname "$racine_parcours")/tp_payguard"
if [ ! -d "$pay_parent" ]; then unzip FORMATION/EXERCICES/tp_payguard_apprenants.zip -d
"$pay_parent" || exit 1; fi
cd "$pay_parent/tp_autonome_drift" || exit 1
pwd
test -f pyproject.toml && test -f uv.lock || exit 1
code -n .
```

Si unzip manque, utilise l'explorateur pour extraire vers le même dossier. Ce TP a son environnement uv propre et le ZIP n'a pas d'historique Git. Pour sauvegarder tes exercices, utilise un **dépôt personnel PayGuard distinct et vide**, créé dans ton compte GitHub de formation. Dans le dossier extérieur, vérifie que git rev-parse ne désigne pas un dépôt parent inattendu avant l'initialisation.

```
$branchePay = Read-Host 'Nom de ta branche personnelle PayGuard'
$ancienDepot = git rev-parse --show-toplevel 2>$null
if ($LASTEXITCODE -eq 0) { throw 'Dépôt Git déjà présent : utilise la reprise, pas une
seconde initialisation.' }
git init -b $branchePay
if ($LASTEXITCODE -ne 0) { throw 'Initialisation PayGuard échouée.' }
$urlPay = Read-Host 'URL du dépôt personnel PayGuard vide sans token'
```

```
git remote add perso $urlPay
git status --short
git remote -v
git branch --show-current
```

```
printf 'Nom de ta branche personnelle PayGuard : '
IFS= read -r branche_pay
if git rev-parse --show-toplevel >/dev/null 2>&1; then printf 'Dépôt déjà présent : utilise
la reprise.\n'; exit 1; fi
git init -b "$branche_pay" || exit 1
printf 'URL du dépôt personnel PayGuard vide sans token : '
IFS= read -r url_pay
git remote add perso "$url_pay"
git status --short
git remote -v
git branch --show-current
```

Exécute cette initialisation seulement si ce TP n'est pas déjà un dépôt. Pour une reprise, exige un état propre et fais `git pull --ff-only perso` avec le nom de sa branche personnelle. Au premier passage la branche distante est absente : pull sans objet avant le premier push. Les jalons ne se fusionnent que dans Parcours.

Reprendre un dépôt PayGuard déjà initialisé

Ce bloc remplace l'initialisation lors d'une reprise. Si tu as du travail non commité, conserve-le et fais le checkpoint de clôture PayGuard avant le pull. Utilise le nom de sa propre branche, pas celui de Parcours.

```
$payguard = Read-Host 'Chemin complet de tp_payguard\tp_autonome_drift'
Set-Location -LiteralPath $payguard -ErrorAction Stop
pwd
$racinePay = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'PayGuard doit être un dépôt initialisé.' }
$payAttendu = (Resolve-Path -LiteralPath $payguard -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racinePay.Trim()).TrimEnd('\','/') -ine
$payAttendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt PayGuard.' }
if (-not (Test-Path .\pyproject.toml) -or -not (Test-Path .\uv.lock)) { throw 'Projet
PayGuard incomplet.' }
git status --short
git remote -v
$branchePayAttendue = Read-Host 'Nom exact de ta branche personnelle PayGuard'
$branchePay = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($branchePayAttendue) -or $branchePay
-cne $branchePayAttendue -or [string]::IsNullOrWhiteSpace($branchePay) -or $branchePay -eq
'main' -or $branchePay -like 'jalon/*') { throw 'Branche personnelle PayGuard requise.' }
if (git status --porcelain) { throw 'Fais un checkpoint PayGuard avant le pull.' }
code -n .
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Configure la destination personnelle PayGuard.' }
$refPay = git ls-remote --heads perso $branchePay
if ($LASTEXITCODE -ne 0) { throw 'Accès PayGuard distant échoué.' }
if ($refPay) {
    git pull --ff-only perso $branchePay
    if ($LASTEXITCODE -ne 0) { throw 'Divergence à examiner avec le formateur.' }
} else { Write-Host 'Branche PayGuard non publiée : premier push à la clôture.' }
```

```
printf 'Chemin complet de tp_payguard/tp_autonome_drift : '
IFS= read -r payguard
cd "$payguard" || exit 1
pwd
racine_pay="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_pay" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine PayGuard.\n';
exit 1; }
test -f pyproject.toml && test -f uv.lock || exit 1
```

```
git status --short
git remote -v
printf 'Nom exact de ta branche personnelle PayGuard : '
IFS= read -r branche_pay_attendue
branche_pay="$(git branch --show-current)" || exit 1
[ -n "$branche_pay_attendue" ] && [ "$branche_pay" = "$branche_pay_attendue" ] || { printf
'Branche PayGuard différente de la branche attendue.\n'; exit 1; }
case "$branche_pay" in main|jalon/*|'') printf 'Branche personnelle PayGuard requise.\n';
exit 1;; esac
[ -z "$(git status --porcelain)" ] || { printf 'Checkpoint PayGuard avant le pull.\n'; exit
1; }
code -n .
git remote get-url perso || exit 1
ref_pay="$(git ls-remote --heads perso "$branche_pay")" || exit 1
if [ -n "$ref_pay" ]; then git pull --ff-only perso "$branche_pay" || exit 1; fi
```

Attendu : racine PayGuard et branche personnelle conservées ; pull terminé ou absence distante vérifiée avant premier push. Un échec d'authentification, une divergence ou un conflit n'efface rien : garde le travail et lis le diagnostic avec le formateur.

Progression partagée M31-M32 : le formateur annonce chaque étape et chaque pause. **Passe 1** = TP autonome PayGuard (zip `05_DONNEES_ET_EXERCICES\tp_payguard_apprenants.zip`, definition of done : 12 tests verts). **Passe 2** = miroir autonome InduSense livré dans `05_DONNEES_ET_EXERCICES\tp_drift_indusense` : ouvre ce dossier dans VS Code, puis garde `PAS_A_PAS_apprenant_indusense.md` à côté du terminal. Ce miroir possède son propre `.python-version`, `pyproject.toml` et `uv.lock` ; il utilise **Python 3.13 / scikit-learn 1.9.0** et ne dépend ni du dépôt GitHub, ni de PayGuard, ni des quatre sources brutes.

Dans ce miroir, lance d'abord `uv sync --frozen --extra dev`, puis vérifie `uv run python --version` → **Python 3.13.x**. La preuve attendue est `uv run python -m pytest .\tests -q` → **11 passed** avec le modèle livré ; sans `models\model.joblib`, le résultat intermédiaire normal est **10 passed, 1 skipped** jusqu'à `uv run python .\scripts\train_model.py`. Le seuil gelé est **0,05**. Les rappels sont **F1 0,771 · F2 0,784 · F3 0,053 · janvier 0,728**. La fenêtre F2 « capteur +8 °C » donne un **PSI 6,845** ; janvier donne **6,213** contre la référence normale mais **0,001** contre la référence haute. Retenir les trois leçons : le PSI hurle sous dérive capteur ; il reste muet face au concept drift F3 malgré le rappel effondré ; une campagne différente exige une référence par régime.

⚠ Ne mélange pas ces mesures réelles avec la simulation synthétique de la fiche TD ci-dessous : `normal(70,4) + 8 °C` donne environ **3,32**. Un PSI n'a de sens qu'avec sa population, sa référence et son binning nommés.

Fin de demi journée J4 après midi PayGuard

Enregistre les fichiers dans VS Code. Retourne à la **racine Parcours** avant de sauvegarder. Les chemins ci-dessous correspondent aux fichiers travaillés dans cette demi-journée : retire ceux que tu n'as pas modifiés et ajoute uniquement les nouveaux fichiers que tu as réellement créés et relus. Les dossiers temporaires, `.env`, clés, bases SQLite, captures contenant des données privées et environnements `.venv` restent hors Git. Si `git status` signale une suppression volontaire, inspecte-la avec le formateur avant son ajout.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
```

```
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
$brancheAttendue = Read-Host 'Nom exact de ta branche personnelle choisie en début de demi-
journée'
$branche = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheAttendue) -or $branche -cne
$brancheAttendue -or $branche -eq 'main' -or $branche -like 'jalon/*') { throw 'Branche
courante différente de ta branche personnelle attendue : arrêt avant ajout, commit et push.'
}
git status --short
git diff --stat
git diff
$fichiers = @('FORMATION/EXERCICES/PAYGUARD_README.md')
# Relis et adapte cette liste avant de poursuivre.
$existants = @($fichiers | Where-Object { Test-Path -LiteralPath $_ })
if ($existants.Count -gt 0) {
    git add -- $existants
    if ($LASTEXITCODE -ne 0) { throw 'Ajout Git échoué : ne pas commiter.' }
}
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Corrige les erreurs signalées avant le commit.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J4 après midi PayGuard travail et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire identité Git, hook ou conflit.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Impossible de contrôler les changements indexés.' }
git status --short
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Sauvegarde distante non configurée : commits locaux
conservés.' }
$suivi = git for-each-ref --format="%(upstream:short)" "refs/heads/$branche"
if ($suivi -cne "perso/$branche") {
    git push -u perso $branche
} else { git push perso $branche }
if ($LASTEXITCODE -ne 0) { throw 'Push échoué : ta sauvegarde distante reste à faire.' }
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso $branche
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
```

```
printf 'Nom exact de ta branche personnelle choisie en début de demi-journée : '
IFS= read -r branche_attendue
branche="$(git branch --show-current)" || exit 1
[ -n "$branche_attendue" ] && [ "$branche" = "$branche_attendue" ] || { printf 'Branche
différente de la branche personnelle attendue.\n'; exit 1; }
case "$branche" in main|jalon/*|'') printf 'Branche personnelle requise avant ajout, commit
et push.\n'; exit 1;; esac
git status --short
git diff --stat
git diff
fichiers=("FORMATION/EXERCICES/PAYGUARD_README.md")
# Relis et adapte cette liste avant de poursuivre.
for fichier in "${fichiers[@]}"; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then
  git commit -m "Sprint 3 J4 après midi PayGuard travail et preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then
  printf 'Contrôle du diff impossible.\n'; exit 1
fi
git status --short
git remote get-url perso || exit 1
suivi="$(git for-each-ref --format='%(upstream:short)' "refs/heads/$branche")"
if [ "$suivi" != "perso/$branche" ]; then
  git push -u perso "$branche" || exit 1
else
  git push perso "$branche" || exit 1
fi
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso "$branche"
```

Attendu : un commit seulement si tu as des changements, un push terminé sans erreur et **le même hash** pour `HEAD` et la branche distante listée par `ls-remote`. Vérifie aussi ton dernier commit dans le dépôt personnel GitHub. `Everything up-to-date` est normal si tout était déjà envoyé. `nothing to commit` ne justifie pas un commit vide : vérifie et pousse les commits locaux éventuels. Si des fichiers restent dans `status`, conserve-les et identifie-les ; tu n'es pas encore prêt pour le prochain jalon.

Si le commit échoue à cause d'un hook qui reformate un fichier, relis le nouveau diff et indexe à nouveau le fichier avant un nouveau commit. Si le push répond `rejected` ou `non-fast-forward`, ne force pas : un autre historique distant doit être réconcilié avec le formateur. Si un conflit est déjà en cours, `git status` indique les fichiers concernés ; travaille avec le formateur avant d'ajouter et de commiter la résolution.

Sauvegarder aussi le travail PayGuard de cette demi journée

La clôture Parcours ci-dessus conserve les commits de jalon. Tes exercices PayGuard se trouvent dans un autre dépôt : ouvre un terminal dans `tp_payguard/tp_autonome_drift` et sauvegarde-les séparément. Choisis les fichiers de code et de preuve relus ; laisse les environnements, bases et secrets hors Git.

```
$payguard = Read-Host 'Chemin complet de tp_payguard\tp_autonome_drift'
Set-Location -LiteralPath $payguard -ErrorAction Stop
pwd
$racinePay = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'PayGuard doit être un dépôt initialisé.' }
$payAttendu = (Resolve-Path -LiteralPath $payguard -ErrorAction Stop).Path
```

```
if ([IO.Path]::GetFullPath($racinePay.Trim()).TrimEnd('\','/') -ine
$payAttendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt PayGuard.' }
if (-not (Test-Path .\pyproject.toml) -or -not (Test-Path .\uv.lock)) { throw 'Projet
PayGuard incomplet.' }
git status --short
git remote -v
$branchePayAttendue = Read-Host 'Nom exact de ta branche personnelle PayGuard'
$branchePay = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($branchePayAttendue) -or $branchePay
-cne $branchePayAttendue -or [string]::IsNullOrWhiteSpace($branchePay) -or $branchePay -eq
'main' -or $branchePay -like 'jalon/*') { throw 'Branche personnelle PayGuard requise.' }
git diff
$fichiersPay = @('.gitignore', '.python-version', 'pyproject.toml', 'uv.lock', 'README.md',
'src', 'scripts', 'tests', 'reports')
$choixPay = @($fichiersPay | Where-Object { Test-Path -LiteralPath $_ })
if ($choixPay.Count -gt 0) { git add -- $choixPay }
if ($LASTEXITCODE -ne 0) { throw 'Ajout PayGuard échoué.' }
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Diff PayGuard à corriger avant le commit.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J4 PayGuard exercices et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit PayGuard refusé : lire le message.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Diff PayGuard impossible.' }
if ($LASTEXITCODE -ne 0 -and $LASTEXITCODE -ne 1) { throw 'Contrôle ou commit échoué.' }
git push -u perso $branchePay
if ($LASTEXITCODE -ne 0) { throw 'Push PayGuard non terminé.' }
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso $branchePay
```

```
printf 'Chemin complet de tp_payguard/tp_autonome_drift : '
IFS= read -r payguard
cd "$payguard" || exit 1
pwd
racine_pay="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_pay" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine PayGuard.\n';
exit 1; }
test -f pyproject.toml && test -f uv.lock || exit 1
git status --short
git remote -v
printf 'Nom exact de ta branche personnelle PayGuard : '
IFS= read -r branche_pay_attendue
branche_pay="$(git branch --show-current)" || exit 1
[ -n "$branche_pay_attendue" ] && [ "$branche_pay" = "$branche_pay_attendue" ] || { printf
'Branche PayGuard différente de la branche attendue.\n'; exit 1; }
case "$branche_pay" in main|jalon/*|'') printf 'Branche personnelle PayGuard requise.\n';
exit 1;; esac
git diff
for fichier in .gitignore .python-version pyproject.toml uv.lock README.md src scripts tests
reports; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then git commit -m "Sprint 3 J4 PayGuard exercices et preuves" ||
exit 1; fi
git push -u perso "$branche_pay" || exit 1
git status -sb
```

```
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso "$branche_pay"
```

Le premier push fixe le suivi avec `-u` ; les suivants peuvent être `git push perso` avec le nom de branche. N'utilise pas l'URL du dépôt Parcours pour un projet PayGuard initialisé séparément. Attendu : hash local et distant identiques. `nothing to commit` signifie vérifier le push existant. Un refus d'authentification ou un push rejeté laisse tes commits locaux intacts : conserve-les et fais réconcilier la destination, sans force.

Début de demi journée J5 matin InduSense drift

Terminal du dépôt Parcours. Même si tu y travaillais hier, redonne le chemin complet ; cela évite un `Set-Location -LiteralPath $projet` depuis un dossier déjà ouvert. Arrête d'abord le serveur qui occupe ce terminal avec Ctrl+C si nécessaire. Ne lance pas de Git dans le terminal occupé par un serveur.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
pwd
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
git status --short
git remote -v
git branch --show-current
if (git status --porcelain) { throw 'Travail non commité : fais un checkpoint avant la
récupération.' }
$branche = Read-Host 'Nom exact de ta branche personnelle du Sprint 3'
if ([string]::IsNullOrWhiteSpace($branche) -or $branche -eq 'main' -or $branche -like
'jalon/*') { throw 'Utilise ta branche personnelle.' }
git switch $branche
if ($LASTEXITCODE -ne 0) { throw 'Branche non ouverte : lire le message Git.' }
code .
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Remote perso absent : configure ta destination de
sauvegarde avant la suite.' }
$refPerso = git ls-remote --heads perso $branche
if ($LASTEXITCODE -ne 0) { throw 'Accès distant échoué : ne pas continuer comme si le pull
avait réussi.' }
if ($refPerso) {
    git pull --ff-only perso $branche
    if ($LASTEXITCODE -ne 0) { throw 'Pull refusé : conserve ton travail et appelle le
formateur.' }
} else { Write-Host 'Premier passage : branche encore absente de perso, premier push à la
clôture.' }
git status --short
git branch --show-current
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
pwd
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
git status --short
git remote -v
git branch --show-current
[ -z "$(git status --porcelain)" ] || { printf 'Fais un checkpoint avant la récupération.\n';
exit 1; }
printf 'Nom exact de ta branche personnelle du Sprint 3 : '
IFS= read -r branche
case "$branche" in main|jalon/*|'') printf 'Utilise ta branche personnelle.\n'; exit 1;; esac
git switch "$branche" || exit 1
code .
git remote get-url perso || exit 1
ref_perso="$(git ls-remote --heads perso "$branche")" || exit 1
if [ -n "$ref_perso" ]; then
  git pull --ff-only perso "$branche" || exit 1
else
  printf 'Branche absente de perso : premier push à la clôture.\n'
fi
git status --short
git branch --show-current
```

Attendu : racine Parcours, branche personnelle et remote de sauvegarde corrects. Si `status` n'est pas vide, enregistre tes fichiers, lis le diff et sélectionne ceux à commiter comme dans la clôture ci-dessous. Ne récupère pas un jalon par-dessus du travail non commité. Si le pull échoue pour authentification, vérifie le compte et les droits dans le mécanisme GitHub proposé ; ne colle aucun token dans une commande. Un refus `--ff-only` indique des historiques divergents : fais les contrôles avec le formateur, sans reset ni push forcé.

Au signal du formateur, intègre **jalon/09** sur cette même branche avec les deux commandes suivantes. Le script effectue la fusion nécessaire des commits du cours ; `--ff-only` ci-dessus concernait uniquement la sauvegarde personnelle.

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 09
if ($LASTEXITCODE -ne 0) { throw 'Jalon non intégré : lire le diagnostic, ne pas lancer le
module.' }
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 09
if ($LASTEXITCODE -ne 0) { throw 'Vérification du jalon échouée.' }
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
git branch --show-current
```

```
bash scripts/formation/mettre_a_niveau.sh 09 || exit 1
bash scripts/formation/verifier_jalon.sh 09 || exit 1
head -n 1 FORMATION/JALON_ACTUEL.md
git branch --show-current
```

Attendu : `Jalon integre sur … : jalon/09`, puis `Jalon verifie : jalon/09`, le marqueur correspondant au tableau de progression, et ta branche personnelle. En cas de conflit, le script annule sa fusion ; lis le diagnostic avec le formateur avant un éventuel rattrapage.

Ouvre le miroir réellement contenu dans Parcours après le jalon 09.

```
Set-Location -LiteralPath (Read-Host 'Chemin complet du dépôt Parcours') -ErrorAction Stop
Set-Location -LiteralPath .\FORMATION\EXERCICES\tp_drift_indusense -ErrorAction Stop
pwd
git rev-parse --show-toplevel
code -n .
```

```
printf 'Chemin complet du dépôt Parcours : '
IFS= read -r projet
cd "$projet/FORMATION/EXERCICES/tp_drift_indusense" || exit 1
pwd
git rev-parse --show-toplevel
code -n .
```

Attendu : dossier uv `tp_drift_indusense`, mais racine Git **Parcours**. Ne crée pas de `.git` imbriqué. Pour la clôture J5 matin, retourne à la racine Parcours et ajoute les chemins `FORMATION/EXERCICES/tp_drift_indusense/…`.

J5 matin — séquence exacte dans le terminal du miroir :

```
# 1) Confirmer que VS Code a ouvert tp_drift_indusense, pas son dossier parent.
Test-Path -LiteralPath .\pyproject.toml
Test-Path -LiteralPath .\uv.lock
Test-Path -LiteralPath .\data\reference_normale.csv

# 2) Construire l'environnement sans recalculer le verrou.
uv sync --frozen --extra dev
uv run python --version
uv run python -c "import sklearn; print(sklearn.__version__)"

# 3) Rejouer les quatre fenêtres et nommer la référence à chaque fois.
uv run python .\scripts\drift_lab.py --fenetre 1 --reference normale
uv run python .\scripts\drift_lab.py --fenetre 2 --reference normale
uv run python .\scripts\drift_lab.py --fenetre 3 --reference normale
uv run python .\scripts\drift_lab.py --fenetre janvier --reference normale
uv run python .\scripts\drift_lab.py --fenetre janvier --reference haute

# 4) Produire la preuve M32, puis valider le kit complet.
uv run python .\scripts\alerting_demo.py --report-out .\reports\drift_report_f2.json
uv run python -m pytest .\tests -q -p no:cacheprovider
```

☑ **À montrer au formateur :** Python 3.13.x, scikit-learn 1.9.0, les deux rapports de janvier (`normale` puis `haute`), `sequence=0 -> 1 -> 0`, une seule ligne `drift_events`, le JSON généré et **11 passed**. Si `python --version` tout court affiche autre chose, ne l'utilise pas : c'est `uv run python` qui fait foi.

Réserve d entraînement ancien protocole intégré synthétique optionnel au signal du formateur

Le socle J5 est **uniquement** le miroir et les preuves 6,845 / 0,053 / 11 tests ci-dessus. Si le formateur ouvre ensuite une extension, il peut faire comparer la simulation synthétique `normal(70,4) + 8 °C` (**PSI ≈ 3,32, KS p ≈ 0**) ou l'ancien protocole intégré au dépôt fil rouge (**PSI 6,834, rappel 0,092, 8 tests**). Ces valeurs ne remplacent jamais la preuve officielle : elles illustrent que le PSI dépend de la référence, du binning et de la population.

📖 **Définition. Covariate drift** : P(X) change (entrées). **Concept drift** : P(y\|X) change (relation). **PSI** : ampleur (<0,1 / 0,1-0,25 / >0,25). **KS** : significativité (p≈0). ❓ « Faut-il des labels pour détecter le drift ? » → covariate : non ; concept : oui. 🔑 Toujours citer **population + référence + binning** avec une valeur de PSI.

32 Rapport de drift alerting contrat JSON officiel Evidently en extension US3 5 C3 C8

✓ **Preuve officielle J5 :** dans le miroir, `alerting_demo.py` génère le rapport JSON, écrit une seule ligne `drift_events` dans SQLite et prouve **`sequence=0 -> 1 -> 0`**. **Evidently n'est pas installé dans l'environnement du miroir** : un rapport Evidently HTML appartient à la démonstration ou à l'extension du dépôt fil rouge, jamais au préflight obligatoire.

📖 **Définition. Cooldown** : délai mini entre deux alertes. **Hystérésis** : réarmer seulement quand le PSI repasse sous le seuil. **drift_events** : table qui trace l'alerte. Un bon système d'alerte reste silencieux quand tout va bien.

Extension seulement, sur consigne du formateur : brancher `drift_check` après `predict`, produire un rapport Evidently avec l'extra explicitement installé/verrouillé dans le dépôt fil rouge, puis rejouer sain → 0, dérive → 1, relance → 0. Ne pas lancer cette extension dans le miroir autonome.

Fin de demi journée J5 matin InduSense drift

Enregistre les fichiers dans VS Code. Retourne à la **racine Parcours** avant de sauvegarder. Les chemins ci-dessous correspondent aux fichiers travaillés dans cette demi-journée : retire ceux que tu n'as pas modifiés et ajoute uniquement les nouveaux fichiers que tu as réellement créés et relus. Les dossiers temporaires, `.env`, clés, bases SQLite, captures contenant des données privées et environnements `.venv` restent hors Git. Si `git status` signale une suppression volontaire, inspecte-la avec le formateur avant son ajout.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
$brancheAttendue = Read-Host 'Nom exact de ta branche personnelle choisie en début de demi-
journée'
$branche = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheAttendue) -or $branche -cne
$brancheAttendue -or $branche -eq 'main' -or $branche -like 'jalon/*') { throw 'Branche
courante différente de ta branche personnelle attendue : arrêt avant ajout, commit et push.'
}
git status --short
git diff --stat
git diff
$fichiers = @('FORMATION/EXERCICES/tp_drift_indusense/scripts',
'FORMATION/EXERCICES/tp_drift_indusense/tests',
'FORMATION/EXERCICES/tp_drift_indusense/reports')
# Relis et adapte cette liste avant de poursuivre.
$existants = @($fichiers | Where-Object { Test-Path -LiteralPath $_ })
if ($existants.Count -gt 0) {
    git add -- $existants
    if ($LASTEXITCODE -ne 0) { throw 'Ajout Git échoué : ne pas commiter.' }
```

```
}
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Corrige les erreurs signalées avant le commit.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J5 matin InduSense drift travail et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire identité Git, hook ou conflit.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Impossible de contrôler les changements indexés.' }
git status --short
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Sauvegarde distante non configurée : commits locaux
conservés.' }
$suivi = git for-each-ref --format="%(upstream:short)" "refs/heads/$branche"
if ($suivi -cne "perso/$branche") {
    git push -u perso $branche
} else { git push perso $branche }
if ($LASTEXITCODE -ne 0) { throw 'Push échoué : ta sauvegarde distante reste à faire.' }
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso $branche
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
printf 'Nom exact de ta branche personnelle choisie en début de demi-journée : '
IFS= read -r branche_attendue
branche="$(git branch --show-current)" || exit 1
[ -n "$branche_attendue" ] && [ "$branche" = "$branche_attendue" ] || { printf 'Branche
différente de la branche personnelle attendue.\n'; exit 1; }
case "$branche" in main|jalon/*|'') printf 'Branche personnelle requise avant ajout, commit
et push.\n'; exit 1;; esac
git status --short
git diff --stat
git diff
fichiers=("FORMATION/EXERCICES/tp_drift_indusense/scripts"
"FORMATION/EXERCICES/tp_drift_indusense/tests"
"FORMATION/EXERCICES/tp_drift_indusense/reports")
# Relis et adapte cette liste avant de poursuivre.
for fichier in "${fichiers[@]}"; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then
  git commit -m "Sprint 3 J5 matin InduSense drift travail et preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then
```

```
  printf 'Contrôle du diff impossible.\n'; exit 1
fi
git status --short
git remote get-url perso || exit 1
suivi="$(git for-each-ref --format='%(upstream:short)' "refs/heads/$branche")"
if [ "$suivi" != "perso/$branche" ]; then
  git push -u perso "$branche" || exit 1
else
  git push perso "$branche" || exit 1
fi
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso "$branche"
```

Attendu : un commit seulement si tu as des changements, un push terminé sans erreur et **le même hash** pour `HEAD` et la branche distante listée par `ls-remote`. Vérifie aussi ton dernier commit dans le dépôt personnel GitHub. `Everything up-to-date` est normal si tout était déjà envoyé. `nothing to commit` ne justifie pas un commit vide : vérifie et pousse les commits locaux éventuels. Si des fichiers restent dans `status`, conserve-les et identifie-les ; tu n'es pas encore prêt pour le prochain jalon.

Si le commit échoue à cause d'un hook qui reformate un fichier, relis le nouveau diff et indexe à nouveau le fichier avant un nouveau commit. Si le push répond `rejected` ou `non-fast-forward`, ne force pas : un autre historique distant doit être réconcilié avec le formateur. Si un conflit est déjà en cours, `git status` indique les fichiers concernés ; travaille avec le formateur avant d'ajouter et de commiter la résolution.

33 Observabilité API Prometheus J5 après midi 1 2 US3 6 C6 C8

Début de demi journée J5 après midi M33 et M34

Terminal du dépôt Parcours. Même si tu y travaillais hier, redonne le chemin complet ; cela évite un `Set-Location -LiteralPath $projet` depuis un dossier déjà ouvert. Arrête d'abord le serveur qui occupe ce terminal avec Ctrl+C si nécessaire. Ne lance pas de Git dans le terminal occupé par un serveur.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
pwd
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
git status --short
git remote -v
git branch --show-current
if (git status --porcelain) { throw 'Travail non commité : fais un checkpoint avant la
récupération.' }
$branche = Read-Host 'Nom exact de ta branche personnelle du Sprint 3'
```

```
if ([string]::IsNullOrWhiteSpace($branche) -or $branche -eq 'main' -or $branche -like
'jalon/*') { throw 'Utilise ta branche personnelle.' }
git switch $branche
if ($LASTEXITCODE -ne 0) { throw 'Branche non ouverte : lire le message Git.' }
code .
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Remote perso absent : configure ta destination de
sauvegarde avant la suite.' }
$refPerso = git ls-remote --heads perso $branche
if ($LASTEXITCODE -ne 0) { throw 'Accès distant échoué : ne pas continuer comme si le pull
avait réussi.' }
if ($refPerso) {
    git pull --ff-only perso $branche
    if ($LASTEXITCODE -ne 0) { throw 'Pull refusé : conserve ton travail et appelle le
formateur.' }
} else { Write-Host 'Premier passage : branche encore absente de perso, premier push à la
clôture.' }
git status --short
git branch --show-current
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
pwd
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
git status --short
git remote -v
git branch --show-current
[ -z "$(git status --porcelain)" ] || { printf 'Fais un checkpoint avant la récupération.\n';
exit 1; }
printf 'Nom exact de ta branche personnelle du Sprint 3 : '
IFS= read -r branche
case "$branche" in main|jalon/*|'') printf 'Utilise ta branche personnelle.\n'; exit 1;; esac
git switch "$branche" || exit 1
code .
git remote get-url perso || exit 1
ref_perso="$(git ls-remote --heads perso "$branche")" || exit 1
if [ -n "$ref_perso" ]; then
  git pull --ff-only perso "$branche" || exit 1
else
  printf 'Branche absente de perso : premier push à la clôture.\n'
fi
git status --short
git branch --show-current
```

Attendu : racine Parcours, branche personnelle et remote de sauvegarde corrects. Si `status` n'est pas vide, enregistre tes fichiers, lis le diff et sélectionne ceux à commiter comme dans la clôture ci-dessous. Ne récupère pas un jalon par-dessus du travail non commité. Si le pull échoue pour authentification, vérifie le compte et les droits dans le mécanisme GitHub proposé ; ne colle aucun

token dans une commande. Un refus `--ff-only` indique des historiques divergents : fais les contrôles avec le formateur, sans reset ni push forcé.

Au signal du formateur, intègre **jalon/10** sur cette même branche avec les deux commandes suivantes. Le script effectue la fusion nécessaire des commits du cours ; `--ff-only` ci-dessus concernait uniquement la sauvegarde personnelle.

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 10
if ($LASTEXITCODE -ne 0) { throw 'Jalon non intégré : lire le diagnostic, ne pas lancer le
module.' }
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 10
if ($LASTEXITCODE -ne 0) { throw 'Vérification du jalon échouée.' }
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
git branch --show-current
```

```
bash scripts/formation/mettre_a_niveau.sh 10 || exit 1
bash scripts/formation/verifier_jalon.sh 10 || exit 1
head -n 1 FORMATION/JALON_ACTUEL.md
git branch --show-current
```

Attendu : `Jalon integre sur … : jalon/10`, puis `Jalon verifie : jalon/10`, le marqueur correspondant au tableau de progression, et ta branche personnelle. En cas de conflit, le script annule sa fusion ; lis le diagnostic avec le formateur avant un éventuel rattrapage.

🆕 Deux cibles à voir **UP** dans Prometheus (Status → Targets) : `indusense-api` (l'API du repo) et `indusense-drift` (lancez `uv run python scripts/export_drift_metrics.py` sur votre poste). ✓ **Preuve finale visée :** `/metrics` scrapeable · gauge obligatoire `indusense_model_loaded` visible · 5 SLI/SLO · requêtes PromQL p95 et readiness qui renvoient une valeur.

Ouverture scénario A

- Préflight : **l'instrumentator est déjà câblé** dans `api/main.py`. Pour isoler réellement les 401, l'instanciation doit être `Instrumentator(should_group_status_codes=False)` avant `.instrument(app).expose(app, endpoint="/metrics", include_in_schema=False)`. **Prometheus est déjà dans le compose**.
- Le ZIP `vision_metrics_lab_v1_20260823.zip` n'est **pas versionné dans le dépôt Git** : il est livré dans `05_DONNEES_ET_EXERCICES/` du pack apprenant. Ne déduis jamais le pack du répertoire courant.

Le préflight tente deux emplacements usuels, exige une sélection explicite en cas d'absence ou d'ambiguïté, puis vérifie l'empreinte avant toute extraction.

```
$expectedVisionSha = '49E8017F75D85412B0489E48867A132737042598584ABCADF9181F203454B5EC'
$searchRoots = @((Join-Path $env:USERPROFILE 'Downloads'), 'C:\CISIA\S3') |
    Where-Object { Test-Path -LiteralPath $_ -PathType Container }
$packCandidates = @(
    foreach ($searchRoot in $searchRoots) {
        Get-ChildItem -LiteralPath $searchRoot -Directory -Filter
'PACK_APPRENANTS_SPRINT3_CISIA_*' `
            -ErrorAction SilentlyContinue | Where-Object {
                Test-Path -LiteralPath (Join-Path $_.FullName
'05_DONNEES_ET_EXERCICES\vision_metrics_lab_v1_20260823.zip')
            }
    }
)
if ($packCandidates.Count -eq 1) {
    $packRoot = $packCandidates[0].FullName
} else {
    $packInput = Read-Host 'Chemin complet du dossier PACK_APPRENANTS_SPRINT3_CISIA_*'
    $packRoot = (Resolve-Path -LiteralPath $packInput.Trim('"')).Path
}
$projectInput = Read-Host 'Colle le chemin complet du dossier CISIA_24082026_Parcours'
$projectRoot = (Resolve-Path -LiteralPath $projectInput.Trim('"')).Path
```

```
$m33Payload = Join-Path $packRoot 'payload.json'
$m33Locustfile = Join-Path $packRoot 'perf\locustfile.py'
$m33VisionLabZip = Join-Path $packRoot
'05_DONNEES_ET_EXERCICES\vision_metrics_lab_v1_20260823.zip'
@($projectRoot, $m33Payload, $m33Locustfile, $m33VisionLabZip) | ForEach-Object {
    if (-not (Test-Path -LiteralPath $_)) { throw "Préflight M33 : chemin introuvable : $_" }
}
$actualVisionSha = (Get-FileHash -LiteralPath $m33VisionLabZip -Algorithm SHA256).Hash
if ($actualVisionSha -ne $expectedVisionSha) {
    throw "ZIP M33 inattendu : $actualVisionSha"
}
# Contrôle visuel : si une AUTRE stack tient 3000 ou 9090, appelle le formateur avant de
continuer.
docker ps --filter "publish=3000" --filter "publish=9090" --format "{{.Names}} -> {{.Ports}}"
Set-Location -LiteralPath $projectRoot -ErrorAction Stop
uv sync --frozen
if ($LASTEXITCODE -ne 0) { throw "uv sync --frozen a échoué (code $LASTEXITCODE)." }
if (-not (Test-Path -LiteralPath .\.env)) { Copy-Item -LiteralPath .\.env.example -
Destination .\.env }
docker compose config -q
if ($LASTEXITCODE -ne 0) { throw "Configuration Compose invalide." }
docker compose up -d --wait
if ($LASTEXITCODE -ne 0) { throw "Démarrage de la stack M33 impossible." }
```

macOS/Linux — auto-détection et SHA-256 (à exécuter sous Bash) :

```
set -euo pipefail
expected_vision_sha='49E8017F75D85412B0489E48867A132737042598584ABCADF9181F203454B5EC'
shopt -s nullglob
pack_candidates=()
for search_root in "$HOME/Downloads" "$HOME/CISIA/S3"; do
  [ -d "$search_root" ] || continue
  for candidate in "$search_root"/PACK_APPRENANTS_SPRINT3_CISIA_*; do
    [ -f "$candidate/05_DONNEES_ET_EXERCICES/vision_metrics_lab_v1_20260823.zip" ] &&
pack_candidates+=("$candidate")
  done
done
if [ "${#pack_candidates[@]}" -eq 1 ]; then
  pack_root="${pack_candidates[0]}"
else
  read -r -p 'Chemin complet du dossier PACK_APPRENANTS_SPRINT3_CISIA_* : ' pack_root
  pack_root="$(cd "$pack_root" && pwd -P)"
fi
m33_vision_lab_zip="$pack_root/05_DONNEES_ET_EXERCICES/vision_metrics_lab_v1_20260823.zip"
[ -f "$m33_vision_lab_zip" ] || { echo "ZIP M33 introuvable : $m33_vision_lab_zip" >&2; exit
1; }
if command -v sha256sum >/dev/null 2>&1; then
  actual_vision_sha="$(sha256sum "$m33_vision_lab_zip" | awk '{print toupper($1)}')"
else
  actual_vision_sha="$(shasum -a 256 "$m33_vision_lab_zip" | awk '{print toupper($1)}')"
fi
[ "$actual_vision_sha" = "$expected_vision_sha" ] || { echo "ZIP M33 inattendu :
$actual_vision_sha" >&2; exit 1; }
printf 'Pack M33 validé : %s\n' "$pack_root"
```

⚠ Le dépôt contient aussi un `perf\locustfile.py` homonyme : ne l'utilise pas pour M33. Le fichier canonique est celui du **pack apprenant**, désigné par `$m33Locustfile`.

- Ouvre maintenant **Terminal > Nouveau terminal** une seconde fois. Les variables PowerShell ne passent pas d'un terminal à l'autre : redemande donc le chemin du dépôt, lance l'exporteur drift et **laisse ce terminal ouvert** :

```
$projectInput = Read-Host 'Colle le chemin complet du dossier CISIA_24082026_Parcours'
$projectRoot = (Resolve-Path -LiteralPath $projectInput.Trim('"')).Path
```

```
Set-Location -LiteralPath $projectRoot -ErrorAction Stop
uv run python scripts/export_drift_metrics.py
```

- Reviens dans le **Terminal 1**, prouve que l'exporteur répond, puis laisse à Prometheus un intervalle de scrape avant d'ouvrir `http://localhost:9090/targets` :

```
$driftMetrics = curl.exe -fsS http://localhost:9109/metrics
$driftMetricsText = ($driftMetrics -join "`n")
if ($LASTEXITCODE -ne 0 -or $driftMetricsText -notmatch 'indusense_drift') {
    throw "Exporteur drift indisponible ou métrique indusense_drift absente sur le port
9109."
}
$driftMetricsText | Select-String 'indusense_drift'
Start-Sleep -Seconds 15
```

☑ **Résultat attendu :** `indusense-api` et `indusense-drift` sont toutes les deux **UP**. Si l'exporteur du Terminal 2 est arrêté, `indusense-drift` repasse **DOWN** : c'est attendu.

Théorie ciblée counter gauge histogram cardinalité

- 👀 Slide « Mesurer : counter, gauge, histogram » + « Le piège : la cardinalité ».

Tour du câblage main py prometheus yml

- 👀 `monitoring/prometheus.yml` : job `indusense-api`, `metrics_path: /metrics`, `targets: ["api:8000"]`, scrape 15 s.

TP socle métrique custom SLO charge

- Ajouter la métrique custom `indusense_predictions_total` (label `decision`, **pas** `machine_id`) et la gauge de readiness **obligatoire**. Dans `api/main.py`, déclarer la gauge au niveau module, la mettre à `1` après le chargement réussi du bundle et à `0` si le chargement échoue ou à l'arrêt :

```
from prometheus_client import Gauge

MODEL_LOADED = Gauge(
    "indusense_model_loaded",
    "1 si le modèle tabulaire est chargé, 0 sinon",
)

# Dans lifespan, avant le try :
MODEL_LOADED.set(0)
# Juste après store._BUNDLE = store.load_bundle(...) :
MODEL_LOADED.set(1)
# Dans except FileNotFoundError, après store._BUNDLE = None :
MODEL_LOADED.set(0)
# Après yield, à l'arrêt :
MODEL_LOADED.set(0)
```

- Puis exécuter les contrôles et la charge depuis `$projectRoot` en conservant les variables du préflight :

```
curl.exe -fsS http://localhost:8000/metrics   # répond ; chercher indusense_model_loaded 1
# PromQL : histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by
(le))
# Readiness instantanée robuste : scrape réussi × modèle chargé.
# Si la gauge est absente, le membre `or` ramène 0 :
# (
#   up{job="indusense-api"}
#   * on(job, instance)
#   indusense_model_loaded{job="indusense-api"}
# )
# or on(job, instance) (0 * up{job="indusense-api"})
# charge (locust HORS lock → outil éphémère épinglé, JAMAIS `uv add`) :
$m33KeyLine = Get-Content -LiteralPath (Join-Path $projectRoot '.env') |
```

```
  Where-Object { $_ -match '^\s*INDUSENSE_API_KEY\s*=' } |
  Select-Object -Last 1
if (-not $m33KeyLine) { throw "INDUSENSE_API_KEY absente de .env : interrompre avant Locust."
}
$env:INDUSENSE_API_KEY = (($m33KeyLine -split '=', 2)[1]).Trim().Trim('"').Trim("'")
if ([string]::IsNullOrWhiteSpace($env:INDUSENSE_API_KEY)) { throw "INDUSENSE_API_KEY est
vide." }
uv run --with locust==2.44.4 locust -f $m33Locustfile --headless -u 20 -r 5 -t 30s --host
http://localhost:8000
if ($LASTEXITCODE -ne 0) { throw "Locust a échoué (code $LASTEXITCODE)." }
# Version validée localement : locust 2.44.4 sous Python 3.13.14 (22/08/2026).
# Buckets par défaut : 0,1 / 0,5 / 1 s.
# Le p95 PromQL est interpolé : ce n'est pas une mesure fine.
```

☑ **Résultat attendu :** `/metrics` répond · `indusense_model_loaded 1` est visible quand `/ready` vaut 200 · la requête readiness vaut `1` (et `0` si scrape ou modèle indisponible) · l'exporteur `:9103` répond · **2 cibles UP après 15 s** · 5 SLO écrits · la p95 renvoie une valeur.

Preuve FAQ et correction

📖 **Définition. Counter** (monte), **Gauge** (monte/descend), **Histogram** (percentiles, p95). **Cardinalité** : nb de valeurs distinctes d'un label. **SLI** (mesuré) vs **SLO** (objectif chiffré). ❓ « Pourquoi `machine_id` interdit en label ? » → explosion de séries (cardinalité) **et** fuite de donnée. ❓ « `/health`, `/ready`, `/metrics` : pareil ? » → non : **vivant** / **prêt** / **métriques** scrapées. ❓ « Pourquoi multiplier `up` par `indusense_model_loaded` ? » → `up` seul prouve le scrape, la gauge seule peut rester sur son dernier échantillon ; le produit ne vaut `1` que si la cible est collectée **et** le modèle chargé. ❓ « Comment isoler le **taux de 401** ? » → `Instrumentator(should_group_status_codes=False)` — sinon les statuts sont groupés (`4xx` : 401/404/422 confondus). 🔑 À ne pas oublier de dire : mesurer **sans SLO** ne sert à rien — on fixe **5 SLO** dès le départ.

- 🔗 « La **fuite** du module 26 revient : un `machine_id` en label = donnée sensible exposée (et cardinalité). »

Pause obligatoire en visio

- Dépose la capture `/metrics`, la requête PromQL et `slo.md`, puis coupe le partage d'écran pendant la pause annoncée. Le module 34 commence au signal du formateur.

34 Dashboards runbooks Grafana J5 après midi 2 2 US3 6 C6 C8

🆕 Un dashboard **« InduSense — dérive & métriques »** est déjà provisionné dans Grafana : ouvrez-le, disséquez ses panels (seuils 0,10/0,25), puis relancez `evaluate_drift --fenetre 2` et regardez la jauge PSI basculer — avant de construire VOTRE dashboard SLO. ✓ **Preuve finale visée :** deux fichiers distincts — dashboard JSON et export JSON/YAML des règles — plus les deux incidents d'exercice observés en `Firing` et un runbook joué jusqu'au retour `Normal`.

Ouverture

- Reformule la cible : transformer les SLO M33 en panels, créer les règles de production **p95 > 300 ms** et **5xx > 1 %** avec `for: 5m`, puis utiliser deux règles d'exercice déterministes avec `for: 1m`.

Théorie ciblée du SLO au dashboard le runbook

- 👀 Slide « Du SLO au dashboard » + « Le runbook ». Une alerte = condition + durée ; le `for:` évite le clignotement au moindre pic.

Tour de la stack Grafana dans le compose

- 👀 `docker-compose.yml` : service **grafana** (port 3000, `depends_on prometheus`). Datasource = `http://prometheus:9090` (**nom de service**, pas localhost).

TP socle dashboard règles et incidents déterministes

- Construis le dashboard Service / Modèle / Données et exporte son JSON. Dans Grafana Unified Alerting, crée séparément les deux règles de production p95/5xx en `for: 5m`, puis les règles d'exercice **API morte** (`up{job="indusense-api"} < 1`) et **pic de 401** avec `for: 1m`.
- Incident A : `docker compose stop api`, observer `Pending → Firing`, puis `docker compose up -d api` et vérifier `/health` jusqu'au retour `Normal`.
- Incident B : produire pendant plus d'une minute des POST sans clé vers `/predict-tabular`, observer le taux de 401 passer en `Firing`, arrêter la boucle et vérifier le retour `Normal`.

☑ **Résultat attendu :** dashboard JSON + export séparé des règles · règles de production p95/5xx conservées en `for: 5m` · incidents API-down/401 observés en `Firing` · runbook menant à la résolution. Une latence aléatoire Locust n'est pas une preuve de déclenchement.

Preuve FAQ et autonomie tutorée

🕮 **Définition. Panel** : un graphique (requête PromQL). **Alert rule** : condition + `for:` + destination. **Runbook** : symptôme → diagnostic → action → escalade. ❓ « Pourquoi `for: 5m` ? » → **anti-flap** : l'alerte ne se déclenche que si la condition **dure**. ❓ « Grafana affiche 'No data' : pourquoi ? » → datasource = **`http://prometheus:9090`** (nom de service), pas localhost. 🔑 À ne pas oublier : un **runbook non joué** est probablement faux — on le **répète** comme un exercice incendie.

- Au choix : panel taux de 401, ou runbook drift rédigé puis joué. Dépose les deux exports et les captures `Firing`/`Normal`.

QCM 33 34 kahoot J6 xlsx en fin de J5

- **Passe le quiz M33-M34** : son fichier s'appelle `kahoot_J6.xlsx`, mais il se joue bien **en fin de J5**. Le **J6 Game Day n'a aucun classeur Kahoot** : son débrief tient lieu de QCM.

Clôture et consigne Game Day

- Note le lien vers les deux exports, la preuve de résolution et le nom du runbook. Le jalon 11 reste fermé jusqu'au brief J6.

Fin de demi journée J5 après midi M33 et M34

Enregistre les fichiers dans VS Code. Retourne à la **racine Parcours** avant de sauvegarder. Les chemins ci-dessous correspondent aux fichiers travaillés dans cette demi-journée : retire ceux que tu n'as pas modifiés et ajoute uniquement les nouveaux fichiers que tu as réellement créés et relus. Les dossiers temporaires, `.env`, clés, bases SQLite, captures contenant des données privées et environnements `.venv` restent hors Git. Si `git status` signale une suppression volontaire, inspecte-la avec le formateur avant son ajout.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
```

```
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
$brancheAttendue = Read-Host 'Nom exact de ta branche personnelle choisie en début de demi-
journée'
$branche = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheAttendue) -or $branche -cne
$brancheAttendue -or $branche -eq 'main' -or $branche -like 'jalon/*') { throw 'Branche
courante différente de ta branche personnelle attendue : arrêt avant ajout, commit et push.'
}
git status --short
git diff --stat
git diff
$fichiers = @('src/indusense/api/main.py', 'monitoring', 'docs', 'slo.md')
# Relis et adapte cette liste avant de poursuivre.
$existants = @($fichiers | Where-Object { Test-Path -LiteralPath $_ })
if ($existants.Count -gt 0) {
    git add -- $existants
    if ($LASTEXITCODE -ne 0) { throw 'Ajout Git échoué : ne pas commiter.' }
}
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Corrige les erreurs signalées avant le commit.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J5 après midi M33 et M34 travail et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire identité Git, hook ou conflit.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Impossible de contrôler les changements indexés.' }
git status --short
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Sauvegarde distante non configurée : commits locaux
conservés.' }
$suivi = git for-each-ref --format="%(upstream:short)" "refs/heads/$branche"
if ($suivi -cne "perso/$branche") {
    git push -u perso $branche
} else { git push perso $branche }
if ($LASTEXITCODE -ne 0) { throw 'Push échoué : ta sauvegarde distante reste à faire.' }
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso $branche
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
```

```
printf 'Nom exact de ta branche personnelle choisie en début de demi-journée : '
IFS= read -r branche_attendue
branche="$(git branch --show-current)" || exit 1
[ -n "$branche_attendue" ] && [ "$branche" = "$branche_attendue" ] || { printf 'Branche
différente de la branche personnelle attendue.\n'; exit 1; }
case "$branche" in main|jalon/*|'') printf 'Branche personnelle requise avant ajout, commit
et push.\n'; exit 1;; esac
git status --short
git diff --stat
git diff
fichiers=("src/indusense/api/main.py" "monitoring" "docs" "slo.md")
# Relis et adapte cette liste avant de poursuivre.
for fichier in "${fichiers[@]}"; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then
  git commit -m "Sprint 3 J5 après midi M33 et M34 travail et preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then
  printf 'Contrôle du diff impossible.\n'; exit 1
fi
git status --short
git remote get-url perso || exit 1
suivi="$(git for-each-ref --format='%(upstream:short)' "refs/heads/$branche")"
if [ "$suivi" != "perso/$branche" ]; then
  git push -u perso "$branche" || exit 1
else
  git push perso "$branche" || exit 1
fi
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso "$branche"
```

Attendu : un commit seulement si tu as des changements, un push terminé sans erreur et **le même hash** pour `HEAD` et la branche distante listée par `ls-remote`. Vérifie aussi ton dernier commit dans le dépôt personnel GitHub. `Everything up-to-date` est normal si tout était déjà envoyé. `nothing to commit` ne justifie pas un commit vide : vérifie et pousse les commits locaux éventuels. Si des fichiers restent dans `status`, conserve-les et identifie-les ; tu n'es pas encore prêt pour le prochain jalon.

Si le commit échoue à cause d'un hook qui reformate un fichier, relis le nouveau diff et indexe à nouveau le fichier avant un nouveau commit. Si le push répond `rejected` ou `non-fast-forward`, ne force pas : un autre historique distant doit être réconcilié avec le formateur. Si un conflit est déjà en cours, `git status` indique les fichiers concernés ; travaille avec le formateur avant d'ajouter et de commiter la résolution.

JOUR 6 jeudi 03 09 GAME DAY Opération lundi matin

Début de demi journée J6 matin Game Day

Terminal du dépôt Parcours. Même si tu y travaillais hier, redonne le chemin complet ; cela évite un `Set-Location -LiteralPath $projet` depuis un dossier déjà ouvert. Arrête d'abord le serveur qui occupe ce terminal avec Ctrl+C si nécessaire. Ne lance pas de Git dans le terminal occupé par un serveur.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
```

```
Set-Location -LiteralPath $projet -ErrorAction Stop
pwd
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
git status --short
git remote -v
git branch --show-current
if (git status --porcelain) { throw 'Travail non commité : fais un checkpoint avant la
récupération.' }
$branche = Read-Host 'Nom exact de ta branche personnelle du Sprint 3'
if ([string]::IsNullOrWhiteSpace($branche) -or $branche -eq 'main' -or $branche -like
'jalon/*') { throw 'Utilise ta branche personnelle.' }
git switch $branche
if ($LASTEXITCODE -ne 0) { throw 'Branche non ouverte : lire le message Git.' }
code .
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Remote perso absent : configure ta destination de
sauvegarde avant la suite.' }
$refPerso = git ls-remote --heads perso $branche
if ($LASTEXITCODE -ne 0) { throw 'Accès distant échoué : ne pas continuer comme si le pull
avait réussi.' }
if ($refPerso) {
    git pull --ff-only perso $branche
    if ($LASTEXITCODE -ne 0) { throw 'Pull refusé : conserve ton travail et appelle le
formateur.' }
} else { Write-Host 'Premier passage : branche encore absente de perso, premier push à la
clôture.' }
git status --short
git branch --show-current
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
pwd
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
git status --short
git remote -v
git branch --show-current
```

```
[ -z "$(git status --porcelain)" ] || { printf 'Fais un checkpoint avant la récupération.\n';
exit 1; }
printf 'Nom exact de ta branche personnelle du Sprint 3 : '
IFS= read -r branche
case "$branche" in main|jalon/*|'') printf 'Utilise ta branche personnelle.\n'; exit 1;; esac
git switch "$branche" || exit 1
code .
git remote get-url perso || exit 1
ref_perso="$(git ls-remote --heads perso "$branche")" || exit 1
if [ -n "$ref_perso" ]; then
  git pull --ff-only perso "$branche" || exit 1
else
  printf 'Branche absente de perso : premier push à la clôture.\n'
fi
git status --short
git branch --show-current
```

Attendu : racine Parcours, branche personnelle et remote de sauvegarde corrects. Si `status` n'est pas vide, enregistre tes fichiers, lis le diff et sélectionne ceux à commiter comme dans la clôture ci-dessous. Ne récupère pas un jalon par-dessus du travail non commité. Si le pull échoue pour authentification, vérifie le compte et les droits dans le mécanisme GitHub proposé ; ne colle aucun token dans une commande. Un refus `--ff-only` indique des historiques divergents : fais les contrôles avec le formateur, sans reset ni push forcé.

Au signal du formateur, intègre **jalon/11** sur cette même branche avec les deux commandes suivantes. Le script effectue la fusion nécessaire des commits du cours ; `--ff-only` ci-dessus concernait uniquement la sauvegarde personnelle.

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 11
if ($LASTEXITCODE -ne 0) { throw 'Jalon non intégré : lire le diagnostic, ne pas lancer le
module.' }
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 11
if ($LASTEXITCODE -ne 0) { throw 'Vérification du jalon échouée.' }
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
git branch --show-current
```

```
bash scripts/formation/mettre_a_niveau.sh 11 || exit 1
bash scripts/formation/verifier_jalon.sh 11 || exit 1
head -n 1 FORMATION/JALON_ACTUEL.md
git branch --show-current
```

Attendu : `Jalon integre sur … : jalon/11`, puis `Jalon verifie : jalon/11`, le marqueur correspondant au tableau de progression, et ta branche personnelle. En cas de conflit, le script annule sa fusion ; lis le diagnostic avec le formateur avant un éventuel rattrapage.

Votre mission : le bundle Git du jalon 11 contient une branche **`J6-gameday`** volontairement « maintenue » vendredi soir… et plus rien ne marche. Le lanceur crée un clone séparé et une branche `reparation-<binome>`. Vous la ramenez à l'état certifié (tag `v1.0-sain`) : `uv run pytest` vert, chiffres de référence retrouvés, API saine, stack Prometheus/Grafana vivante, dépôt propre — **post-mortem + restitution synthétique** en fin de journée. **Votre support du jour :** **`FORMATION/EXERCICES/J6/gameday_apprenant.pdf`** (6 phases guidées, chiffres de référence, questions à se poser, bonnes pratiques, théorie à retenir). Règle d'or : **diagnostiqué → corrigé → prouvé → commité**.

Progression du Game Day : le formateur donne le signal de départ, les pauses et le passage à chaque phase. Suis l'ordre du support sans anticiper la phase suivante.

- 🔑 **Premier geste Windows PowerShell, au signal du formateur et depuis la racine du dépôt Parcours :**

```
Write-Host 'Identifiant : lettres, chiffres, point, tiret ou underscore.'
$binome = (Read-Host 'Exemple : equipe01').Trim()
```

```
$binomeValide = $binome -match '^[A-Za-z0-9][A-Za-z0-9._-]*$'
if (-not $binomeValide) {
    throw 'Identifiant invalide.'
}
powershell -ExecutionPolicy Bypass -File .\scripts\formation\demarrer_gameday.ps1 -Binome
$binome
if ($LASTEXITCODE -ne 0) { throw 'Clone Game Day non créé : lire le diagnostic.' }
Set-Location -LiteralPath (Join-Path (Split-Path -Parent (git rev-parse --show-toplevel))
"CISIA_J6_GAMEDAY_$binome") -ErrorAction Stop
git branch --show-current
git diff --stat v1.0-sain J6-gameday
```

- 🔑 **Premier geste macOS zsh ou Linux bash, au signal du formateur et depuis la racine du dépôt Parcours :**

```
printf 'Identifiant du binôme, sans espace (ex. equipe01) : '
read -r binome
bash scripts/formation/demarrer_gameday.sh "$binome" || exit 1
cd "../CISIA_J6_GAMEDAY_$binome" || exit 1
git branch --show-current
git diff --stat v1.0-sain J6-gameday
```

- ⚠ On ne travaille JAMAIS directement sur `J6-gameday` ni `main` ; les tests se restaurent depuis `v1.0-sain`, le code se répare en le comprenant. Le remote `bundle-local` est local et son push est désactivé.

- ☑ Definition of done de la journée : pytest vert (tests identiques à `v1.0-sain`) · pipeline drift conforme (seuil 0,03 · PSI fenêtre 2 = 6,834 · rappel fenêtre 3 = 0,092) · 2 cibles Prometheus UP · Grafana peuplé · commits locaux propres · post-mortem rendu. Un push n'est fait que si le formateur fournit explicitement un remote de collaboration et sa consigne le jour J.

Fin de demi journée J6 matin Game Day

Enregistre les fichiers dans VS Code. Retourne à la **racine Parcours** avant de sauvegarder. Les chemins ci-dessous correspondent aux fichiers travaillés dans cette demi-journée : retire ceux que tu n'as pas modifiés et ajoute uniquement les nouveaux fichiers que tu as réellement créés et relus. Les dossiers temporaires, `.env`, clés, bases SQLite, captures contenant des données privées et environnements `.venv` restent hors Git. Si `git status` signale une suppression volontaire, inspecte-la avec le formateur avant son ajout.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
$brancheAttendue = Read-Host 'Nom exact de ta branche personnelle choisie en début de demi-
journée'
$branche = git branch --show-current
```

```
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheAttendue) -or $branche -cne
$brancheAttendue -or $branche -eq 'main' -or $branche -like 'jalon/*') { throw 'Branche
courante différente de ta branche personnelle attendue : arrêt avant ajout, commit et push.'
}
git status --short
git diff --stat
git diff
$fichiers = @('FORMATION/EXERCICES/J6')
# Relis et adapte cette liste avant de poursuivre.
$existants = @($fichiers | Where-Object { Test-Path -LiteralPath $_ })
if ($existants.Count -gt 0) {
    git add -- $existants
    if ($LASTEXITCODE -ne 0) { throw 'Ajout Git échoué : ne pas commiter.' }
}
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Corrige les erreurs signalées avant le commit.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J6 matin Game Day travail et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire identité Git, hook ou conflit.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Impossible de contrôler les changements indexés.' }
git status --short
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Sauvegarde distante non configurée : commits locaux
conservés.' }
$suivi = git for-each-ref --format="%(upstream:short)" "refs/heads/$branche"
if ($suivi -cne "perso/$branche") {
    git push -u perso $branche
} else { git push perso $branche }
if ($LASTEXITCODE -ne 0) { throw 'Push échoué : ta sauvegarde distante reste à faire.' }
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso $branche
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
printf 'Nom exact de ta branche personnelle choisie en début de demi-journée : '
IFS= read -r branche_attendue
branche="$(git branch --show-current)" || exit 1
[ -n "$branche_attendue" ] && [ "$branche" = "$branche_attendue" ] || { printf 'Branche
différente de la branche personnelle attendue.\n'; exit 1; }
case "$branche" in main|jalon/*|'') printf 'Branche personnelle requise avant ajout, commit
et push.\n'; exit 1;; esac
git status --short
git diff --stat
git diff
fichiers=("FORMATION/EXERCICES/J6")
```

```
# Relis et adapte cette liste avant de poursuivre.
for fichier in "${fichiers[@]}"; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then
  git commit -m "Sprint 3 J6 matin Game Day travail et preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then
  printf 'Contrôle du diff impossible.\n'; exit 1
fi
git status --short
git remote get-url perso || exit 1
suivi="$(git for-each-ref --format='%(upstream:short)' "refs/heads/$branche")"
if [ "$suivi" != "perso/$branche" ]; then
  git push -u perso "$branche" || exit 1
else
  git push perso "$branche" || exit 1
fi
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso "$branche"
```

Attendu : un commit seulement si tu as des changements, un push terminé sans erreur et **le même hash** pour `HEAD` et la branche distante listée par `ls-remote`. Vérifie aussi ton dernier commit dans le dépôt personnel GitHub. `Everything up-to-date` est normal si tout était déjà envoyé. `nothing to commit` ne justifie pas un commit vide : vérifie et pousse les commits locaux éventuels. Si des fichiers restent dans `status`, conserve-les et identifie-les ; tu n'es pas encore prêt pour le prochain jalon.

Si le commit échoue à cause d'un hook qui reformate un fichier, relis le nouveau diff et indexe à nouveau le fichier avant un nouveau commit. Si le push répond `rejected` ou `non-fast-forward`, ne force pas : un autre historique distant doit être réconcilié avec le formateur. Si un conflit est déjà en cours, `git status` indique les fichiers concernés ; travaille avec le formateur avant d'ajouter et de commiter la résolution.

Sauvegarder le clone Game Day J6 matin

Ouvre le terminal du clone `CISIA_J6_GAMEDAY_<binome>`, distinct de Parcours. Tu dois rester sur `reparation-<binome>`. Le remote `bundle-local` a son push désactivé : le travail se conserve d'abord par commits locaux.

```
$game = Read-Host 'Chemin complet du clone CISIA_J6_GAMEDAY de ton binôme'
Set-Location -LiteralPath $game -ErrorAction Stop
pwd
$racineGame = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Clone Game Day absent.' }
$gameAttendu = (Resolve-Path -LiteralPath $game -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGame.Trim()).TrimEnd('\','/') -ine
$gameAttendu.TrimEnd('\','/')) { throw 'Ouvre la racine du clone Game Day.' }
$bundleLocal = git remote get-url bundle-local
if ($LASTEXITCODE -ne 0) { throw 'Ce dépôt n est pas le clone du bundle Game Day.' }
git rev-parse --verify v1.0-sain
if ($LASTEXITCODE -ne 0) { throw 'Tag de référence Game Day absent.' }
git status --short
git remote -v
$brancheGameAttendue = Read-Host 'Nom exact de la branche reparation du binôme'
$brancheGame = git branch --show-current
```

```
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheGameAttendue) -or
$brancheGame -cne $brancheGameAttendue -or $brancheGame -notlike 'reparation-*') { throw
'Ouvre la branche de réparation du binôme.' }
git diff
$fichiersGame = @('src', 'tests', 'docs', 'postmortem.md', 'postmortem_template.md')
$choixGame = @($fichiersGame | Where-Object { Test-Path -LiteralPath $_ })
# Adapte la liste aux fichiers réellement réparés, y compris les tests restaurés.
if ($choixGame.Count -gt 0) { git add -- $choixGame }
if ($LASTEXITCODE -ne 0) { throw 'Ajout Game Day échoué.' }
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Diff à corriger.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J6 matin Game Day réparations et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire le diagnostic.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Diff Game Day impossible : arrêt avant push.' }
git log -1 --oneline
git status -sb
git remote get-url collab 2>$null
if ($LASTEXITCODE -eq 0) {
    git push -u collab $brancheGame
    if ($LASTEXITCODE -ne 0) { throw 'Push de collaboration échoué.' }
    git rev-parse HEAD
    git ls-remote --heads collab $brancheGame
} else { Write-Host 'Sans remote collab fourni par le formateur : commits locaux conservés,
aucun push bundle-local.' }
```

```
printf 'Chemin complet du clone Game Day du binôme : '
IFS= read -r game
cd "$game" || exit 1
pwd
racine_game="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_game" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Game Day.\n';
exit 1; }
git remote get-url bundle-local >/dev/null || exit 1
git rev-parse --verify v1.0-sain >/dev/null || exit 1
git status --short
git remote -v
printf 'Nom exact de la branche reparation du binôme : '
IFS= read -r branche_game_attendue
branche_game="$(git branch --show-current)" || exit 1
[ -n "$branche_game_attendue" ] && [ "$branche_game" = "$branche_game_attendue" ] || { printf
'Branche Game Day différente de la branche attendue.\n'; exit 1; }
case "$branche_game" in reparation-*) ;; *) printf 'Branche de réparation requise.\n'; exit
1;; esac
git diff
for fichier in src tests docs postmortem.md postmortem_template.md; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then git commit -m "Sprint 3 J6 matin Game Day réparations et
preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then printf 'Diff Game Day impossible.\n'; exit 1
fi
git log -1 --oneline
git status -sb
if git remote get-url collab >/dev/null 2>&1; then
  git push -u collab "$branche_game" || exit 1
  git rev-parse HEAD
```

```
  git ls-remote --heads collab "$branche_game"
else
  printf 'Commits locaux conservés ; aucun push bundle-local.\n'
fi
```

Si le formateur te fournit une URL de collaboration autorisée, ajoute-la **une seule fois** : PowerShell `$urlCollab = Read-Host 'URL fournie par le formateur sans token'; git remote add collab $urlCollab` ; Bash `printf 'URL fournie par le formateur : '; IFS= read -r url_collab; git remote add collab "$url_collab"`. Après le premier `push -u`, les envois suivants sont `git push collab` avec la même branche. En cas de `nothing to commit`, n'ajoute pas de commit vide. Authentification refusée, divergence ou conflit : garde les commits et fais résoudre le point avec le formateur ; jamais de push forcé.

Début de demi journée J6 après midi Game Day et retex

Terminal du dépôt Parcours. Même si tu y travaillais hier, redonne le chemin complet ; cela évite un `Set-Location -LiteralPath $projet` depuis un dossier déjà ouvert. Arrête d'abord le serveur qui occupe ce terminal avec Ctrl+C si nécessaire. Ne lance pas de Git dans le terminal occupé par un serveur.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
pwd
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
git status --short
git remote -v
git branch --show-current
if (git status --porcelain) { throw 'Travail non commité : fais un checkpoint avant la
récupération.' }
$branche = Read-Host 'Nom exact de ta branche personnelle du Sprint 3'
if ([string]::IsNullOrWhiteSpace($branche) -or $branche -eq 'main' -or $branche -like
'jalon/*') { throw 'Utilise ta branche personnelle.' }
git switch $branche
if ($LASTEXITCODE -ne 0) { throw 'Branche non ouverte : lire le message Git.' }
code .
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Remote perso absent : configure ta destination de
sauvegarde avant la suite.' }
$refPerso = git ls-remote --heads perso $branche
if ($LASTEXITCODE -ne 0) { throw 'Accès distant échoué : ne pas continuer comme si le pull
avait réussi.' }
if ($refPerso) {
    git pull --ff-only perso $branche
    if ($LASTEXITCODE -ne 0) { throw 'Pull refusé : conserve ton travail et appelle le
formateur.' }
} else { Write-Host 'Premier passage : branche encore absente de perso, premier push à la
clôture.' }
```

```
git status --short
git branch --show-current
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
pwd
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
git status --short
git remote -v
git branch --show-current
[ -z "$(git status --porcelain)" ] || { printf 'Fais un checkpoint avant la récupération.\n';
exit 1; }
printf 'Nom exact de ta branche personnelle du Sprint 3 : '
IFS= read -r branche
case "$branche" in main|jalon/*|'') printf 'Utilise ta branche personnelle.\n'; exit 1;; esac
git switch "$branche" || exit 1
code .
git remote get-url perso || exit 1
ref_perso="$(git ls-remote --heads perso "$branche")" || exit 1
if [ -n "$ref_perso" ]; then
  git pull --ff-only perso "$branche" || exit 1
else
  printf 'Branche absente de perso : premier push à la clôture.\n'
fi
git status --short
git branch --show-current
```

Attendu : racine Parcours, branche personnelle et remote de sauvegarde corrects. Si `status` n'est pas vide, enregistre tes fichiers, lis le diff et sélectionne ceux à commiter comme dans la clôture ci-dessous. Ne récupère pas un jalon par-dessus du travail non commité. Si le pull échoue pour authentification, vérifie le compte et les droits dans le mécanisme GitHub proposé ; ne colle aucun token dans une commande. Un refus `--ff-only` indique des historiques divergents : fais les contrôles avec le formateur, sans reset ni push forcé.

Au signal du formateur, intègre **jalon/12** sur cette même branche avec les deux commandes suivantes. Le script effectue la fusion nécessaire des commits du cours ; `--ff-only` ci-dessus concernait uniquement la sauvegarde personnelle.

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 12
if ($LASTEXITCODE -ne 0) { throw 'Jalon non intégré : lire le diagnostic, ne pas lancer le
module.' }
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 12
if ($LASTEXITCODE -ne 0) { throw 'Vérification du jalon échouée.' }
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
git branch --show-current
```

```
bash scripts/formation/mettre_a_niveau.sh 12 || exit 1
bash scripts/formation/verifier_jalon.sh 12 || exit 1
head -n 1 FORMATION/JALON_ACTUEL.md
git branch --show-current
```

Attendu : `Jalon integre sur … : jalon/12`, puis `Jalon verifie : jalon/12`, le marqueur correspondant au tableau de progression, et ta branche personnelle. En cas de conflit, le script annule sa fusion ; lis le diagnostic avec le formateur avant un éventuel rattrapage.

Reprendre le clone Game Day après le jalon 12 de Parcours

Laisse le jalon 12 uniquement dans Parcours. Ouvre ensuite le clone du binôme, vérifie ses propres remotes et sa branche, puis récupère **sa branche de collaboration** seulement si le formateur a fourni ce remote.

```
$game = Read-Host 'Chemin complet du clone CISIA_J6_GAMEDAY de ton binôme'
Set-Location -LiteralPath $game -ErrorAction Stop
pwd
$racineGame = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Clone Game Day absent.' }
$gameAttendu = (Resolve-Path -LiteralPath $game -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGame.Trim()).TrimEnd('\','/') -ine
$gameAttendu.TrimEnd('\','/')) { throw 'Ouvre la racine du clone Game Day.' }
$bundleLocal = git remote get-url bundle-local
if ($LASTEXITCODE -ne 0) { throw 'Ce dépôt n est pas le clone du bundle Game Day.' }
git rev-parse --verify v1.0-sain
if ($LASTEXITCODE -ne 0) { throw 'Tag de référence Game Day absent.' }
git status --short
git remote -v
$brancheGameAttendue = Read-Host 'Nom exact de la branche reparation du binôme'
$brancheGame = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheGameAttendue) -or
$brancheGame -cne $brancheGameAttendue -or $brancheGame -notlike 'reparation-*') { throw
'Branche de réparation requise.' }
if (git status --porcelain) { throw 'Conserve et commite les réparations du matin avant un
pull.' }
code -n .
git remote get-url collab 2>$null
if ($LASTEXITCODE -eq 0) {
    $refGame = git ls-remote --heads collab $brancheGame
    if ($LASTEXITCODE -ne 0) { throw 'Accès collaboration échoué.' }
    if ($refGame) {
        git pull --ff-only collab $brancheGame
        if ($LASTEXITCODE -ne 0) { throw 'Divergence à examiner avec le formateur.' }
    } else { Write-Host 'Branche de collaboration pas encore publiée : pull sans objet.' }
} else { Write-Host 'Reprise sur commits locaux ; bundle-local ne sert pas au push.' }
```

```
printf 'Chemin complet du clone Game Day du binôme : '
IFS= read -r game
cd "$game" || exit 1
pwd
racine_game="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_game" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Game Day.\n';
exit 1; }
git remote get-url bundle-local >/dev/null || exit 1
git rev-parse --verify v1.0-sain >/dev/null || exit 1
git status --short
git remote -v
printf 'Nom exact de la branche reparation du binôme : '
IFS= read -r branche_game_attendue
branche_game="$(git branch --show-current)" || exit 1
[ -n "$branche_game_attendue" ] && [ "$branche_game" = "$branche_game_attendue" ] || { printf
'Branche Game Day différente de la branche attendue.\n'; exit 1; }
case "$branche_game" in reparation-*) ;; *) exit 1;; esac
[ -z "$(git status --porcelain)" ] || { printf 'Commite les réparations du matin avant un
pull.\n'; exit 1; }
code -n .
if git remote get-url collab >/dev/null 2>&1; then
```

```
  ref_game="$(git ls-remote --heads collab "$branche_game")" || exit 1
  if [ -n "$ref_game" ]; then git pull --ff-only collab "$branche_game" || exit 1; fi
else
  printf 'Reprise locale sans push bundle-local.\n'
fi
```

Débrief Game Day tient lieu de QCM du jour

- Pas de QCM « module » aujourd'hui : le **débrief** sert de récap (qu'est-ce qui a cassé, quelle brique du sprint l'a résolu). Option ludique : 6 questions piochées dans les QCM J1→J5.

Clôture du Sprint 3

- 👀 La grille d'évaluation transverse : par apprenant, les **preuves accumulées** (tests, endpoints, dashboards) → matière pour la **soutenance** (C1→C9) et le **notebook journal de bord**.

Débrief clôture retex et ouverture Sprint 4

- Partage un point qui a marché et un point à améliorer pendant le retex guidé par le formateur.

Fin de demi journée J6 après midi Game Day et retex

Enregistre les fichiers dans VS Code. Retourne à la **racine Parcours** avant de sauvegarder. Les chemins ci-dessous correspondent aux fichiers travaillés dans cette demi-journée : retire ceux que tu n'as pas modifiés et ajoute uniquement les nouveaux fichiers que tu as réellement créés et relus. Les dossiers temporaires, `.env`, clés, bases SQLite, captures contenant des données privées et environnements `.venv` restent hors Git. Si `git status` signale une suppression volontaire, inspecte-la avec le formateur avant son ajout.

Windows PowerShell :

```
$projet = Read-Host 'Chemin complet du dossier CISIA_24082026_Parcours'
Set-Location -LiteralPath $projet -ErrorAction Stop
$racineGit = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Ce dossier n est pas un dépôt Git.' }
$attendu = (Resolve-Path -LiteralPath $projet -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGit.Trim()).TrimEnd('\','/') -ine
$attendu.TrimEnd('\','/')) { throw 'Ouvre la racine du dépôt, pas un sous-dossier.' }
$origineCours = git remote get-url origin
if ($LASTEXITCODE -ne 0 -or $origineCours -notin
@('https://github.com/thomasfesq/CISIA_24082026_Parcours.git',
'https://github.com/thomasfesq/CISIA_24082026_Parcours',
'git@github.com:thomasfesq/CISIA_24082026_Parcours.git',
'git@github.com:thomasfesq/CISIA_24082026_Parcours')) { throw 'Ce dépôt ne correspond pas au
cours Parcours.' }
if (-not (Test-Path -LiteralPath .\scripts\formation\mettre_a_niveau.ps1)) { throw 'Scripts
Parcours absents.' }
$brancheAttendue = Read-Host 'Nom exact de ta branche personnelle choisie en début de demi-
journée'
$branche = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheAttendue) -or $branche -cne
$brancheAttendue -or $branche -eq 'main' -or $branche -like 'jalon/*') { throw 'Branche
courante différente de ta branche personnelle attendue : arrêt avant ajout, commit et push.'
}
git status --short
git diff --stat
git diff
$fichiers = @('FORMATION/EXERCICES/J6')
# Relis et adapte cette liste avant de poursuivre.
$existants = @($fichiers | Where-Object { Test-Path -LiteralPath $_ })
if ($existants.Count -gt 0) {
    git add -- $existants
    if ($LASTEXITCODE -ne 0) { throw 'Ajout Git échoué : ne pas commiter.' }
```

```
}
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Corrige les erreurs signalées avant le commit.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J6 après midi Game Day et retex travail et preuves"
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire identité Git, hook ou conflit.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Impossible de contrôler les changements indexés.' }
git status --short
git remote get-url perso
if ($LASTEXITCODE -ne 0) { throw 'Sauvegarde distante non configurée : commits locaux
conservés.' }
$suivi = git for-each-ref --format="%(upstream:short)" "refs/heads/$branche"
if ($suivi -cne "perso/$branche") {
    git push -u perso $branche
} else { git push perso $branche }
if ($LASTEXITCODE -ne 0) { throw 'Push échoué : ta sauvegarde distante reste à faire.' }
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso $branche
```

macOS ou Linux dans Bash :

```
printf 'Chemin complet du dossier CISIA_24082026_Parcours : '
IFS= read -r projet
cd "$projet" || exit 1
racine_git="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_git" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Parcours.\n';
exit 1; }
origine_cours="$(git remote get-url origin)" || exit 1
case "$origine_cours" in

https://github.com/thomasfesq/CISIA_24082026_Parcours.git|https://github.com/thomasfesq/CISIA
_24082026_Parcours|git@github.com:thomasfesq/CISIA_24082026_Parcours.git|git@github.com:thoma
sfesq/CISIA_24082026_Parcours) ;;
  *) printf 'Ce dépôt ne correspond pas au cours Parcours.\n'; exit 1;;
esac
test -f scripts/formation/mettre_a_niveau.sh || exit 1
printf 'Nom exact de ta branche personnelle choisie en début de demi-journée : '
IFS= read -r branche_attendue
branche="$(git branch --show-current)" || exit 1
[ -n "$branche_attendue" ] && [ "$branche" = "$branche_attendue" ] || { printf 'Branche
différente de la branche personnelle attendue.\n'; exit 1; }
case "$branche" in main|jalon/*|'') printf 'Branche personnelle requise avant ajout, commit
et push.\n'; exit 1;; esac
git status --short
git diff --stat
git diff
fichiers=("FORMATION/EXERCICES/J6")
# Relis et adapte cette liste avant de poursuivre.
for fichier in "${fichiers[@]}"; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then
  git commit -m "Sprint 3 J6 après midi Game Day et retex travail et preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then
  printf 'Contrôle du diff impossible.\n'; exit 1
fi
```

```
git status --short
git remote get-url perso || exit 1
suivi="$(git for-each-ref --format='%(upstream:short)' "refs/heads/$branche")"
if [ "$suivi" != "perso/$branche" ]; then
  git push -u perso "$branche" || exit 1
else
  git push perso "$branche" || exit 1
fi
git status -sb
git log -1 --oneline
git rev-parse HEAD
git ls-remote --heads perso "$branche"
```

Attendu : un commit seulement si tu as des changements, un push terminé sans erreur et **le même hash** pour `HEAD` et la branche distante listée par `ls-remote`. Vérifie aussi ton dernier commit dans le dépôt personnel GitHub. `Everything up-to-date` est normal si tout était déjà envoyé. `nothing to commit` ne justifie pas un commit vide : vérifie et pousse les commits locaux éventuels. Si des fichiers restent dans `status`, conserve-les et identifie-les ; tu n'es pas encore prêt pour le prochain jalon.

Si le commit échoue à cause d'un hook qui reformate un fichier, relis le nouveau diff et indexe à nouveau le fichier avant un nouveau commit. Si le push répond `rejected` ou `non-fast-forward`, ne force pas : un autre historique distant doit être réconcilié avec le formateur. Si un conflit est déjà en cours, `git status` indique les fichiers concernés ; travaille avec le formateur avant d'ajouter et de commiter la résolution.

Sauvegarder le clone Game Day J6 après midi

Ouvre le terminal du clone `CISIA_J6_GAMEDAY_<binome>`, distinct de Parcours. Tu dois rester sur `reparation-<binome>`. Le remote `bundle-local` a son push désactivé : le travail se conserve d'abord par commits locaux.

```
$game = Read-Host 'Chemin complet du clone CISIA_J6_GAMEDAY de ton binôme'
Set-Location -LiteralPath $game -ErrorAction Stop
pwd
$racineGame = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Clone Game Day absent.' }
$gameAttendu = (Resolve-Path -LiteralPath $game -ErrorAction Stop).Path
if ([IO.Path]::GetFullPath($racineGame.Trim()).TrimEnd('\','/') -ine
$gameAttendu.TrimEnd('\','/')) { throw 'Ouvre la racine du clone Game Day.' }
$bundleLocal = git remote get-url bundle-local
if ($LASTEXITCODE -ne 0) { throw 'Ce dépôt n est pas le clone du bundle Game Day.' }
git rev-parse --verify v1.0-sain
if ($LASTEXITCODE -ne 0) { throw 'Tag de référence Game Day absent.' }
git status --short
git remote -v
$brancheGameAttendue = Read-Host 'Nom exact de la branche reparation du binôme'
$brancheGame = git branch --show-current
if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($brancheGameAttendue) -or
$brancheGame -cne $brancheGameAttendue -or $brancheGame -notlike 'reparation-*') { throw
'Ouvre la branche de réparation du binôme.' }
git diff
$fichiersGame = @('src', 'tests', 'docs', 'postmortem.md', 'postmortem_template.md')
$choixGame = @($fichiersGame | Where-Object { Test-Path -LiteralPath $_ })
# Adapte la liste aux fichiers réellement réparés, y compris les tests restaurés.
if ($choixGame.Count -gt 0) { git add -- $choixGame }
if ($LASTEXITCODE -ne 0) { throw 'Ajout Game Day échoué.' }
git diff --cached --check
if ($LASTEXITCODE -ne 0) { throw 'Diff à corriger.' }
git diff --cached
git diff --cached --quiet
if ($LASTEXITCODE -eq 1) {
    git commit -m "Sprint 3 J6 après midi Game Day réparations et preuves"
```

```
    if ($LASTEXITCODE -ne 0) { throw 'Commit refusé : lire le diagnostic.' }
} elseif ($LASTEXITCODE -ne 0) { throw 'Diff Game Day impossible : arrêt avant push.' }
git log -1 --oneline
git status -sb
git remote get-url collab 2>$null
if ($LASTEXITCODE -eq 0) {
    git push -u collab $brancheGame
    if ($LASTEXITCODE -ne 0) { throw 'Push de collaboration échoué.' }
    git rev-parse HEAD
    git ls-remote --heads collab $brancheGame
} else { Write-Host 'Sans remote collab fourni par le formateur : commits locaux conservés,
aucun push bundle-local.' }
```

```
printf 'Chemin complet du clone Game Day du binôme : '
IFS= read -r game
cd "$game" || exit 1
pwd
racine_game="$(git rev-parse --show-toplevel)" || exit 1
[ "$(cd "$racine_game" && pwd -P)" = "$(pwd -P)" ] || { printf 'Ouvre la racine Game Day.\n';
exit 1; }
git remote get-url bundle-local >/dev/null || exit 1
git rev-parse --verify v1.0-sain >/dev/null || exit 1
git status --short
git remote -v
printf 'Nom exact de la branche reparation du binôme : '
IFS= read -r branche_game_attendue
branche_game="$(git branch --show-current)" || exit 1
[ -n "$branche_game_attendue" ] && [ "$branche_game" = "$branche_game_attendue" ] || { printf
'Branche Game Day différente de la branche attendue.\n'; exit 1; }
case "$branche_game" in reparation-*) ;; *) printf 'Branche de réparation requise.\n'; exit
1;; esac
git diff
for fichier in src tests docs postmortem.md postmortem_template.md; do
  [ ! -e "$fichier" ] || git add -- "$fichier" || exit 1
done
git diff --cached --check || exit 1
git diff --cached
git diff --cached --quiet
code_diff=$?
if [ "$code_diff" -eq 1 ]; then git commit -m "Sprint 3 J6 après midi Game Day réparations et
preuves" || exit 1
elif [ "$code_diff" -ne 0 ]; then printf 'Diff Game Day impossible.\n'; exit 1
fi
git log -1 --oneline
git status -sb
if git remote get-url collab >/dev/null 2>&1; then
  git push -u collab "$branche_game" || exit 1
  git rev-parse HEAD
  git ls-remote --heads collab "$branche_game"
else
  printf 'Commits locaux conservés ; aucun push bundle-local.\n'
fi
```

Si le formateur te fournit une URL de collaboration autorisée, ajoute-la **une seule fois** : PowerShell `$urlCollab = Read-Host 'URL fournie par le formateur sans token'; git remote add collab $urlCollab` ; Bash `printf 'URL fournie par le formateur : '; IFS= read -r url_collab; git remote add collab "$url_collab"`. Après le premier `push -u`, les envois suivants sont `git push collab` avec la même branche. En cas de `nothing to commit`, n'ajoute pas de commit vide. Authentification refusée, divergence ou conflit : garde les commits et fais résoudre le point avec le formateur ; jamais de push forcé.

Glossaire annexe à distribuer si utile

- **Package / module** : dossier importable / fichier `.py`. **Refactoring** : réorganiser sans changer le comportement. **Lockfile** : versions exactes figées.
- **Fuite de données** : information du futur entrée dans l'entraînement. **CI/CD** : intégration et livraison continues. **DVC / MLflow** : versionner données et modèles.
- **REST / endpoint** : API HTTP / URL+verbe. **Pydantic** valide le contrat I/O ; **Swagger** (`/docs`) documente l'API.
- **Liveness / readiness** : processus vivant / modèle chargé. **lifespan** : chargement au démarrage.
- **STRIDE / adversarial** : menaces et entrées truquées. Codes clés : **401**, **413**, **422**, **429**, **503**.
- **Image / conteneur** : paquet immuable / instance en cours. Le **multi-stage** sépare build et runtime ; la Variante A inclut le modèle. Un **healthcheck** sonde le service ; entre conteneurs, viser `db:5432`, pas `localhost`.